

Digital Design & Computer Arch.

Lecture 22: Caches

Dr. Mohammad Sadrosadati

Prof. Onur Mutlu

ETH Zürich

Spring 2025

16 May 2025

Readings for This Lecture

- Memory Hierarchy and Caches

- Highly Recommended
 - H&H Chapters 8.1-8.3
 - Refresh: P&P Chapter 3.5
 - Kim & Mutlu, “**Memory Systems**,” Computing Handbook, 2014.
 - https://people.inf.ethz.ch/omutlu/pub/memory-systems-introduction_computing-handbook14.pdf

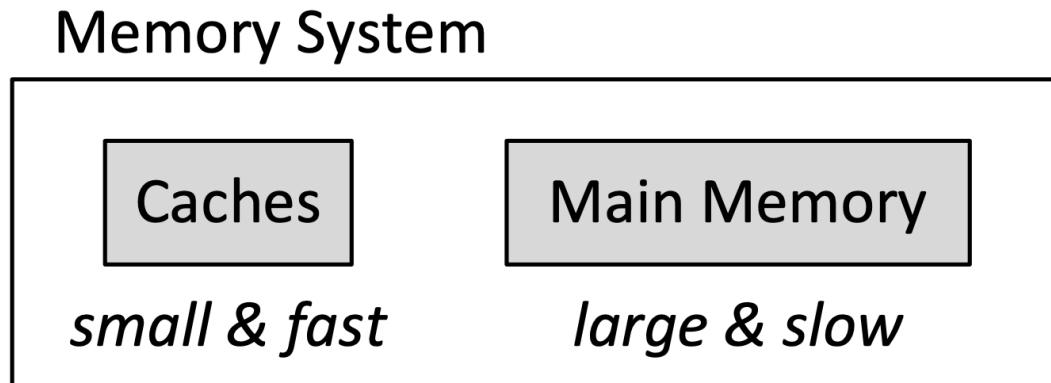
- Recommended
 - An early cache paper by Maurice Wilkes
 - Wilkes, “**Slave Memories and Dynamic Storage Allocation**,” IEEE Trans. On Electronic Computers, 1965.

Recall: The Problem

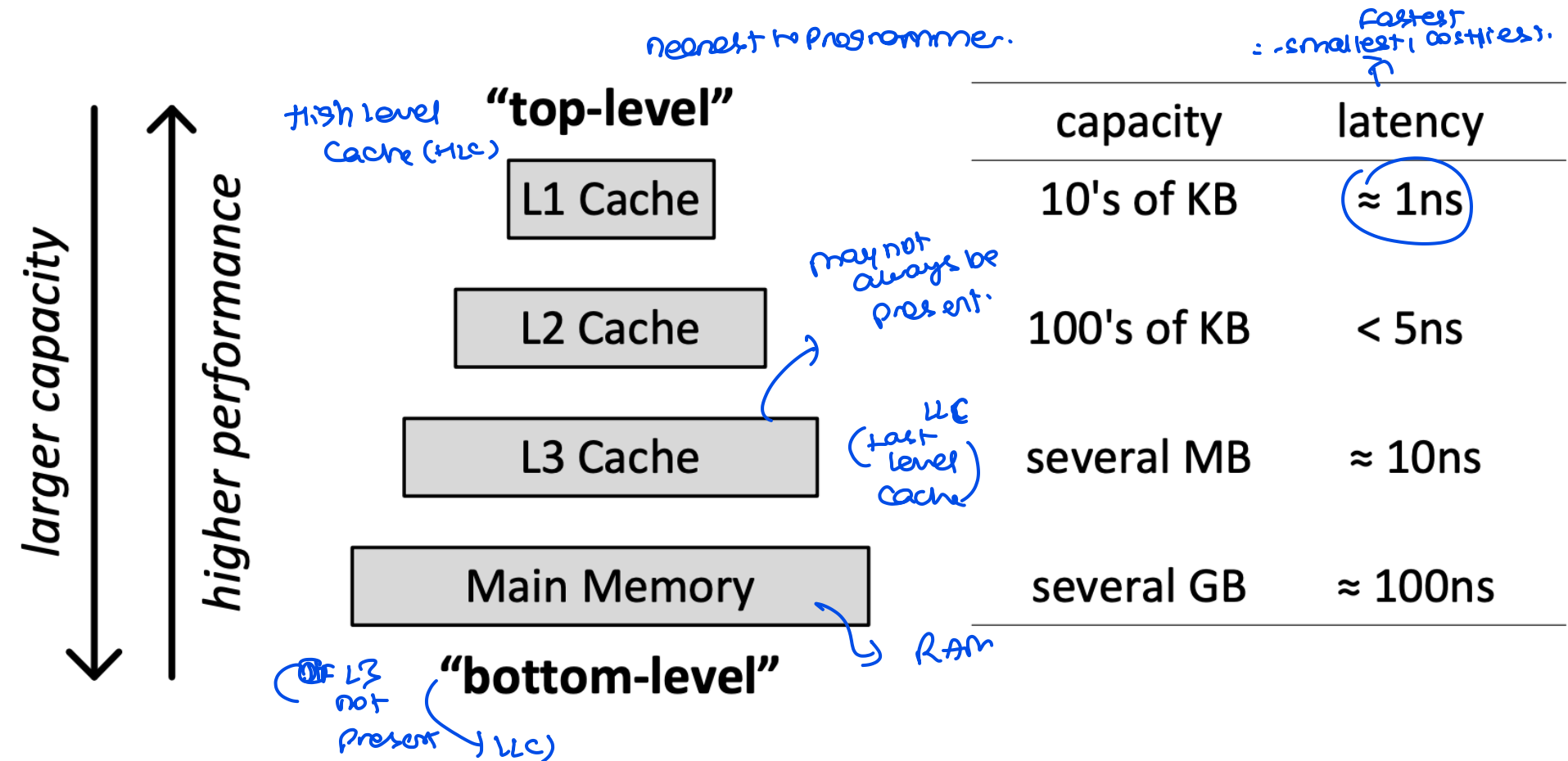
- Ideal memory's requirements oppose each other
- Bigger is slower
 - Bigger → Takes longer to determine the location
- Faster is more expensive
 - Memory technology: SRAM vs. DRAM vs. SSD vs. Disk vs. Tape
- Higher bandwidth is more expensive
 - Need more banks, more ports, more channels, higher frequency or faster technology

Recall: Why Memory Hierarchy?

- We want **both fast and large**
- But, we cannot achieve both with a single level of memory
- Idea: **Have multiple levels of storage** (progressively bigger and slower as the levels are farther from the processor) and **ensure most of the data the processor needs is kept in the fast(er) level(s)**



Recall: Memory Hierarchy Example



Kim & Mutlu, “[Memory Systems](#),” Computing Handbook, 2014

https://people.inf.ethz.ch/omutlu/pub/memory-systems-introduction_computing-handbook14.pdf

Recall: Memory Locality

- A "typical" program has a lot of locality in memory references

- typical programs are composed of "loops"

for loops.
↑
(with a high prob.)

(∴ Temporal → you access more what you currently accessed

- **Temporal**: A program tends to reference the same memory location many times and all within a small window of time (Δt)

(you access a memory location then with high prob you will access nearby memory location in nearby future)

- **Spatial**: A program tends to reference nearby memory locations within a window of time

On array element accessing.

- most notable examples:

1. instruction memory references → mostly sequential/streaming

2. references to arrays/vectors → often streaming/strided

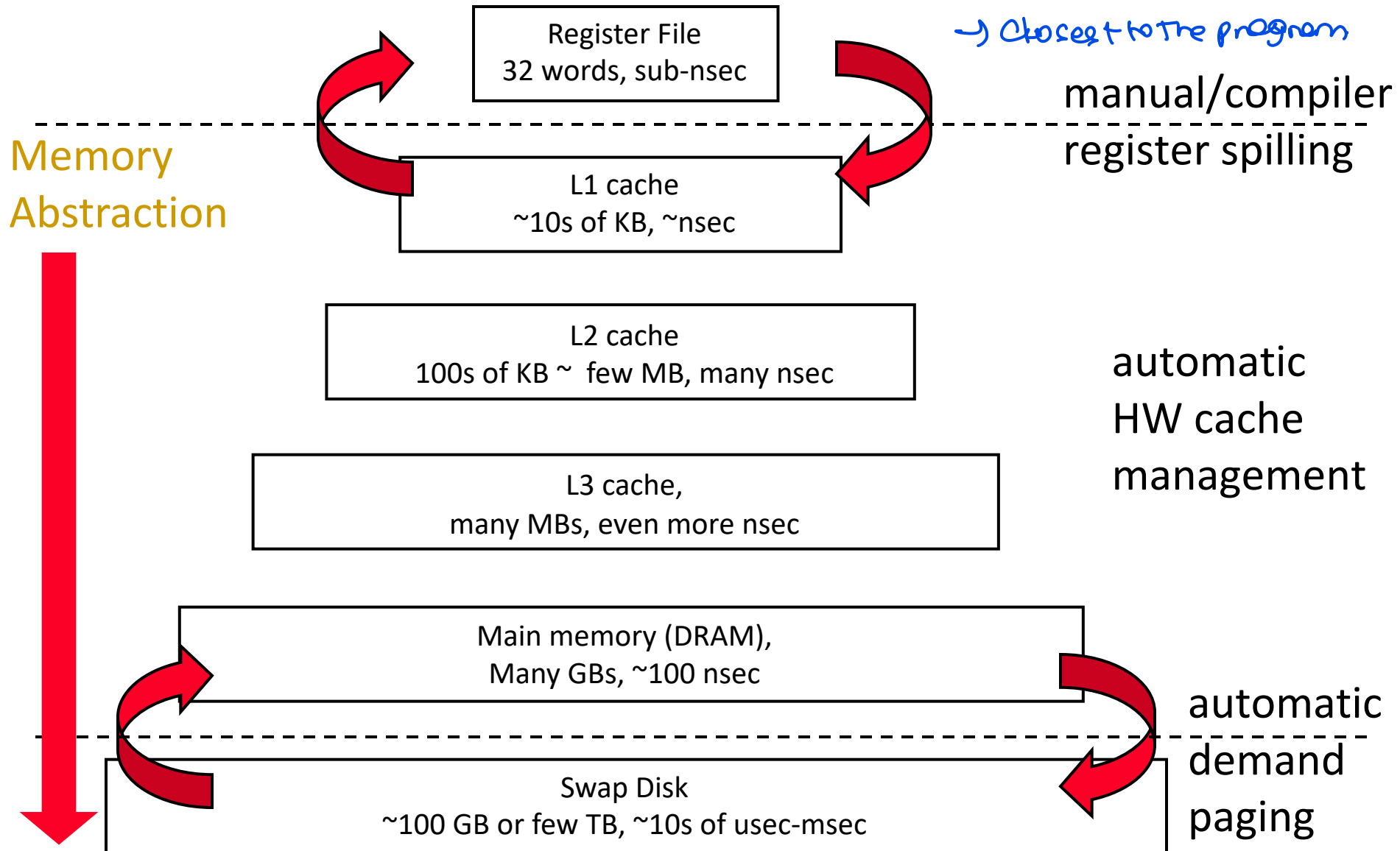
Based
on spatial
locality.

Spatial locality → can be attained by prefetch.

i.e. in loop $x = x(i) + y(i)$

at $i=0$ we prefetch $x(1)$ bet with high prob we fetch it next

Recall: A Modern Memory Hierarchy

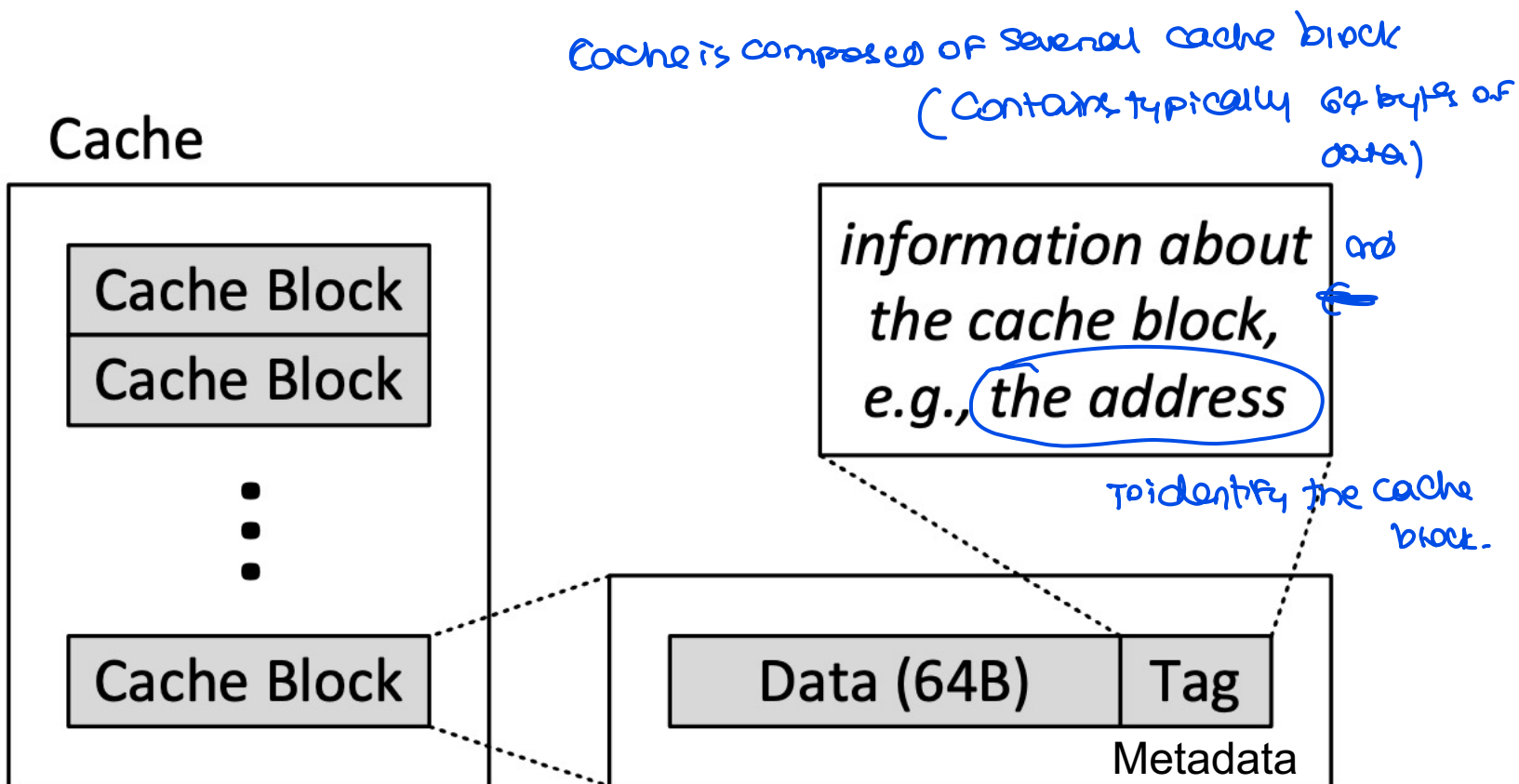


Cache Basics and Operation

Cache

- Any structure that “memoizes” used (or produced) data
 - to avoid repeating the long-latency operations required to reproduce/fetch the data from scratch
 - e.g., a web cache
- Most commonly in the processor design context:
an automatically-managed memory structure
 - e.g., memoize in fast SRAM the most frequently or recently accessed DRAM memory locations to avoid repeatedly paying for the DRAM access latency

Conceptual Picture of a Cache

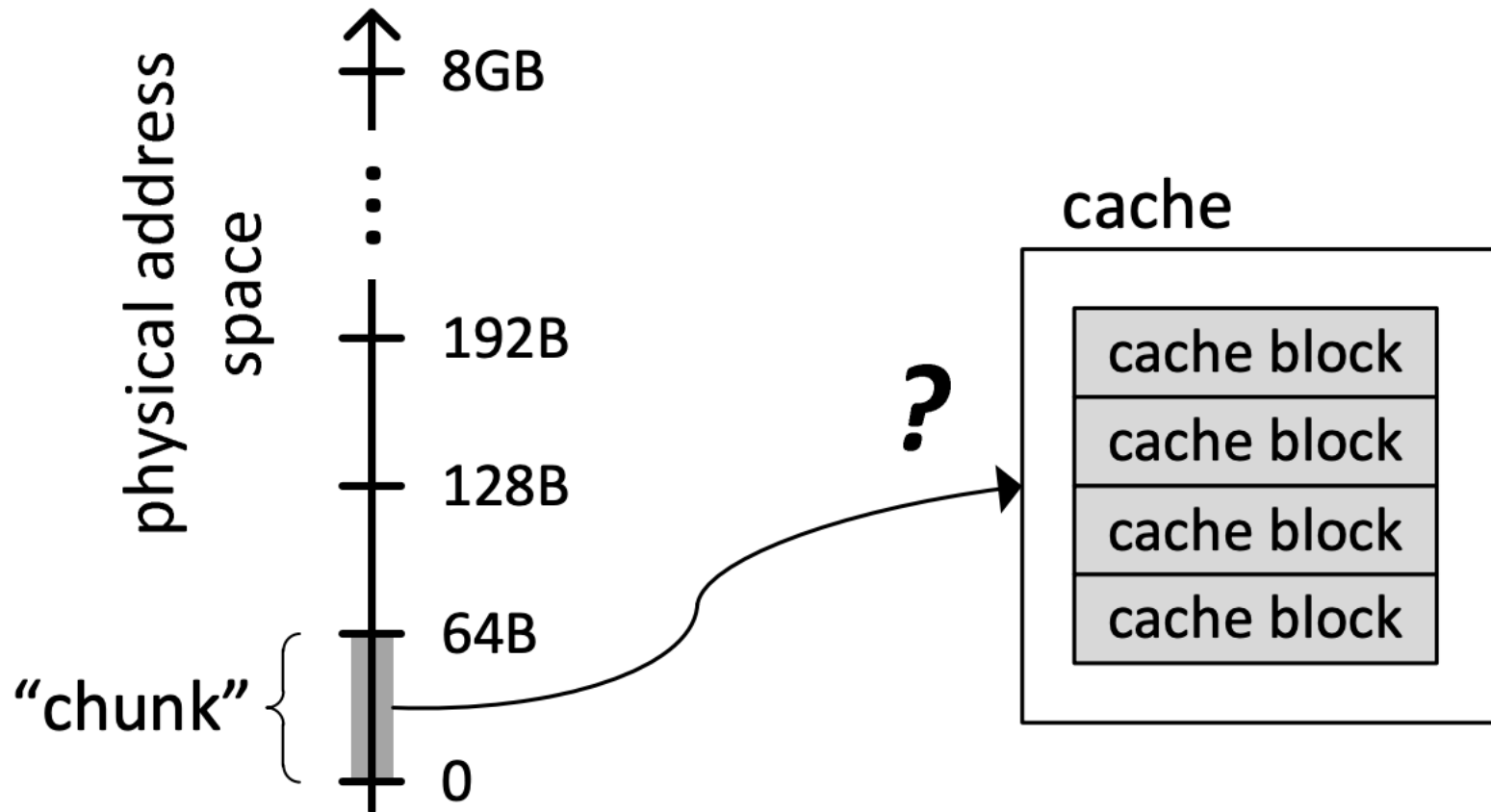


Kim & Mutlu, “[Memory Systems](#),” Computing Handbook, 2014

https://people.inf.ethz.ch/omutlu/pub/memory-systems-introduction_computing-handbook14.pdf

Logical Organization of a Cache (I)

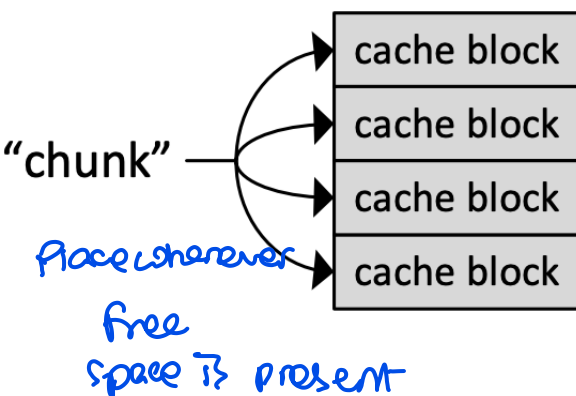
- A key question: How to map chunks of the main memory address space to blocks in the cache?
 - Which location in cache can a given “main memory chunk” be placed in?



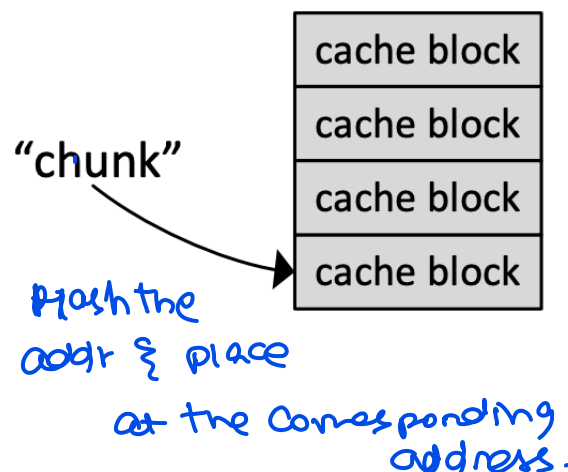
Logical Organization of a Cache (II)

- A key question: How to map chunks of the main memory address space to blocks in the cache?
 - Which location in cache can a given “main memory chunk” be placed in? (How to place cache block in cache)

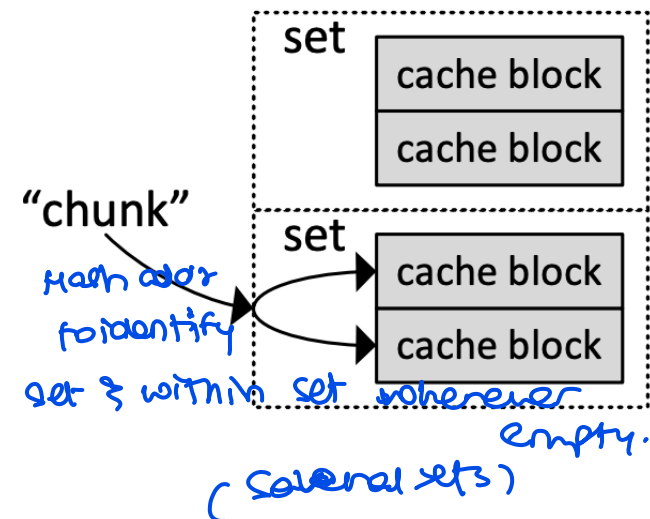
fully-associative



direct-mapped



set-associative

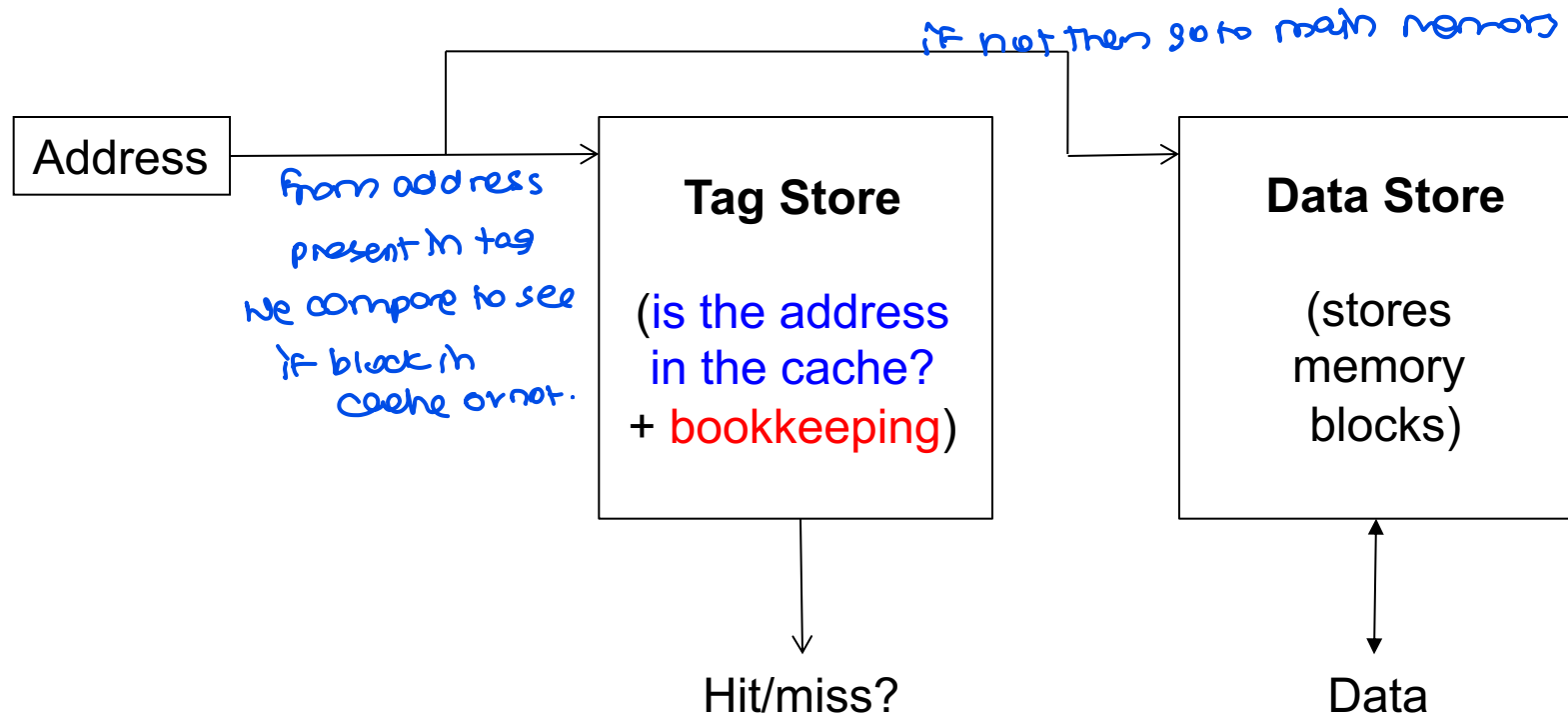


Caching Basics

cache block / cache line is same -

- **Block (line):** Unit of storage in the cache
 - Memory is logically divided into blocks that map to potential locations in the cache
- On a reference:
 - **HIT:** If in cache, use cached data instead of accessing memory ✓
 - **MISS:** If not in cache, bring block into cache
 - May have to evict some other block
- Some important cache design decisions ✓
 - **Placement:** where and how to place/find a block in cache? ✓
 - **Replacement:** what data to remove to make room in cache? ✓
 - **Granularity of management:** large or small blocks? Subblocks?
 - **Write policy:** what do we do about writes?
 - **Instructions/data:** do we treat them separately?

Cache Abstraction and Metrics



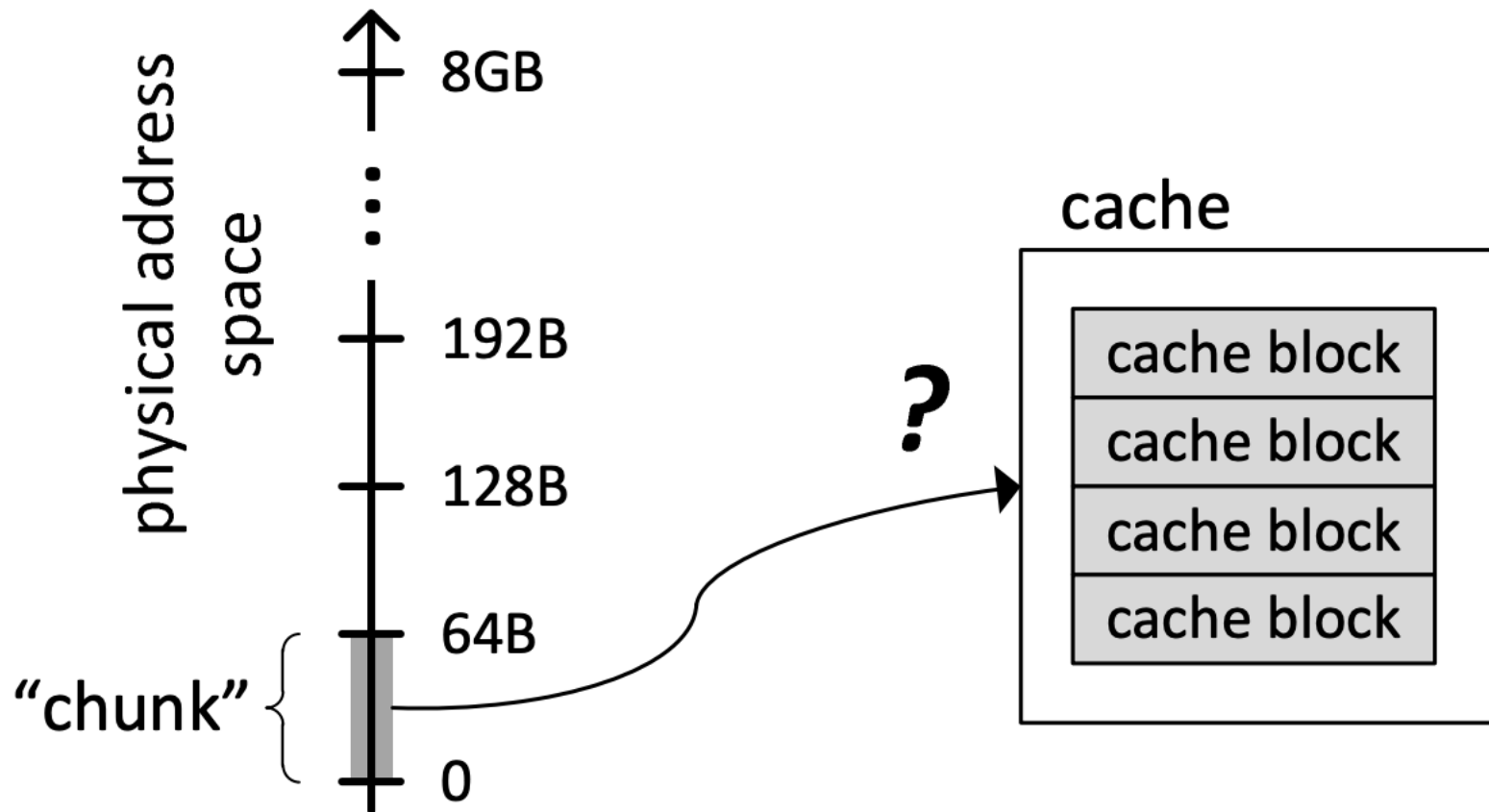
- Cache hit rate = $(\# \text{ hits}) / (\# \text{ hits} + \# \text{ misses}) = (\# \text{ hits}) / (\# \text{ accesses})$
- Average memory access time (AMAT)
= $(\text{hit-rate} * \text{hit-latency}) + (\text{miss-rate} * \text{miss-latency})$
- Important Aside: *Is reducing AMAT always beneficial for performance?*

A Basic Hardware Cache Design

- We will start with a basic hardware cache design
- Then, we will examine a multitude of ideas to make it better (i.e., higher performance)

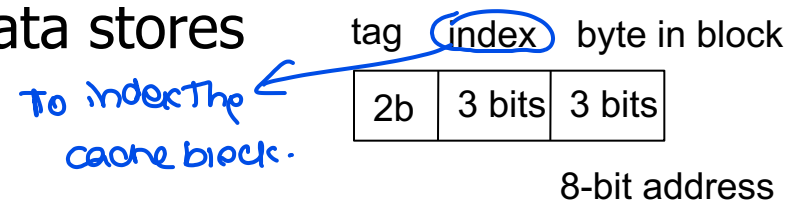
Blocks and Addressing the Cache

- Main memory logically divided into fixed-size chunks (**blocks**)
- **Cache** can house only a **limited** number of blocks



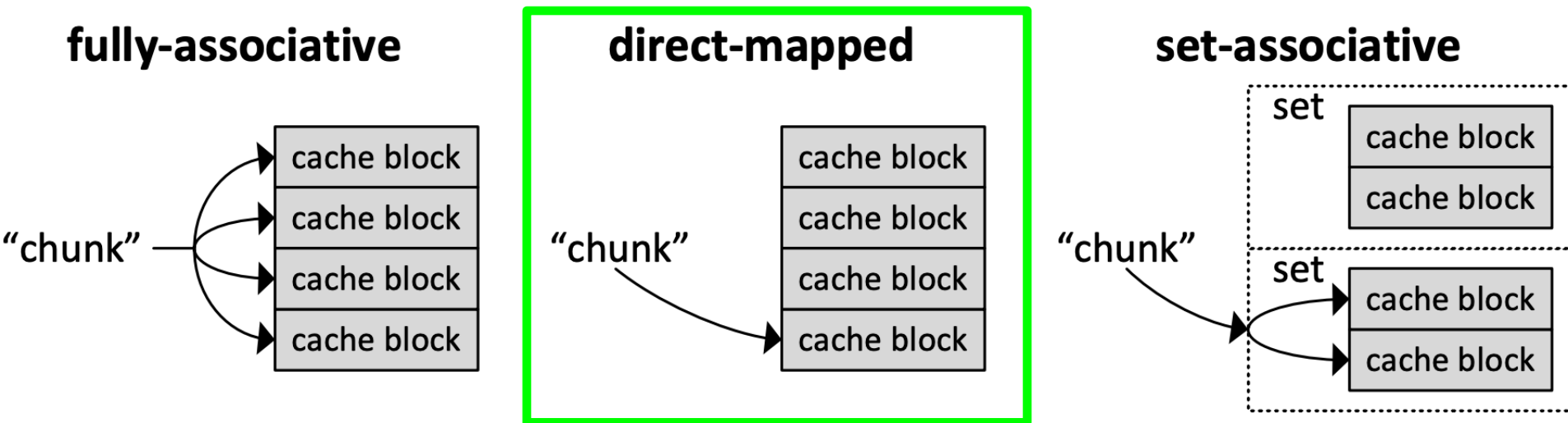
Blocks and Addressing the Cache

- Main memory logically divided into fixed-size chunks (**blocks**)
- **Cache** can house only a **limited** number of blocks
- Each **block address** maps to a potential location in the cache, determined by the **index bits** in the address
 - used to index into the tag and data stores
- Cache access:
 - 1) index into the tag and data stores with index bits in address
 - 2) check valid bit in tag store
 - 3) compare tag bits in address with the stored tag in tag store
- If the stored tag is valid and matches the tag of the block, then the block is in the cache (cache hit)



Let's See A Toy Example

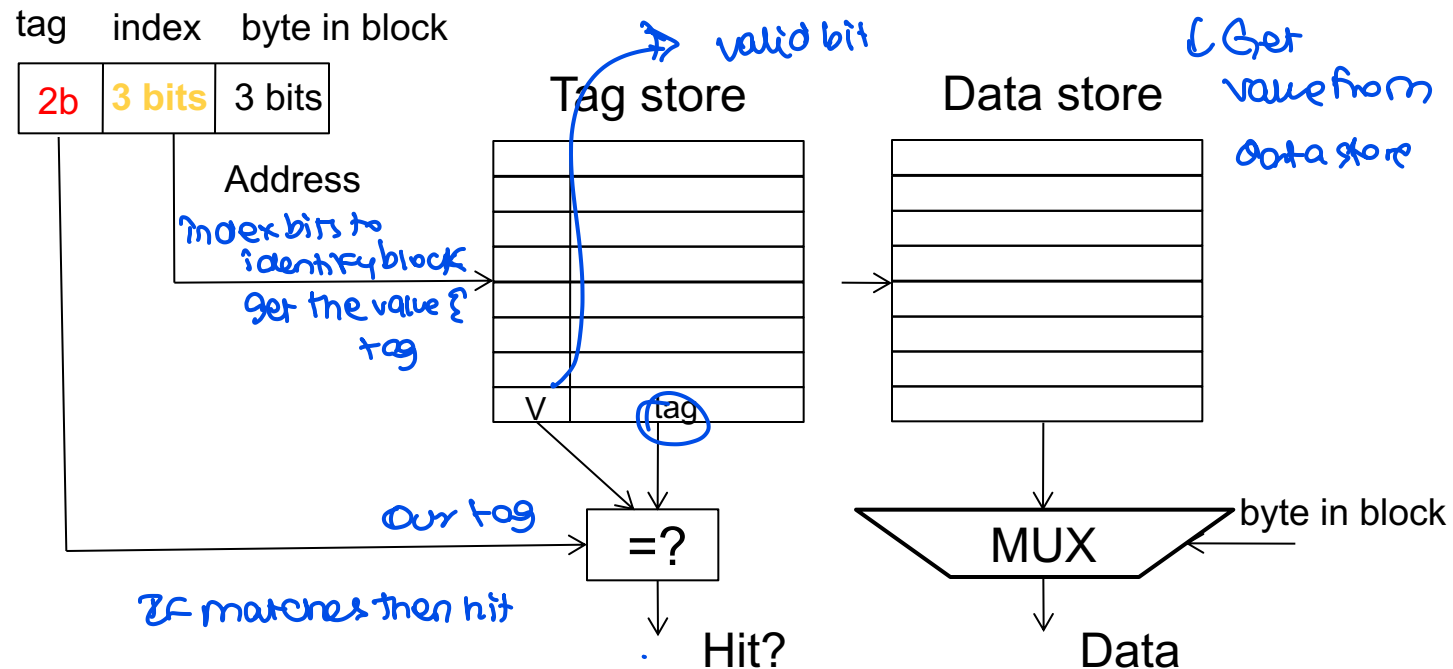
- We will examine a direct-mapped cache first
- **Direct-mapped:** A given main memory block can be placed in **only one possible location** in the cache
- Toy example: 256-byte memory, 64-byte cache, 8-byte blocks



Direct-Mapped Cache: Placement and Access

Block: 00000
Block: 00001
Block: 00010
Block: 00011
Block: 00100
Block: 00101
Block: 00110
Block: 00111
Block: 01000
Block: 01001
Block: 01010
Block: 01011
Block: 01100
Block: 01101
Block: 01110
Block: 01111
Block: 10000
Block: 10001
Block: 10010
Block: 10011
Block: 10100
Block: 10101
Block: 10110
Block: 10111
Block: 11000
Block: 11001
Block: 11010
Block: 11011
Block: 11100
Block: 11101
Block: 11110
Block: 11111

- Assume byte-addressable main memory:
256 bytes, 8-byte blocks → 32 blocks in mem
- Assume cache: 64 bytes, 8 blocks
 - Direct-mapped: A block can go to only one location



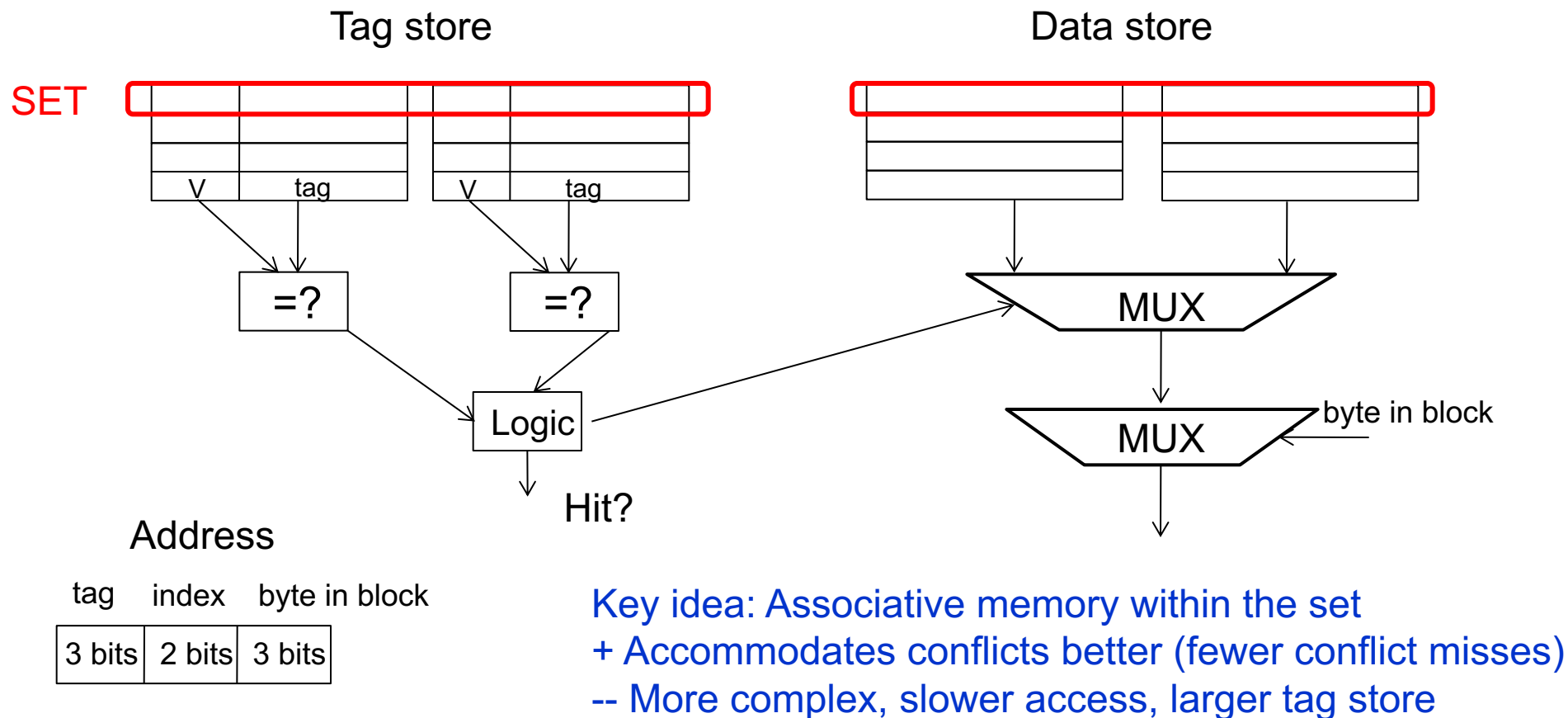
- Blocks with same index contend for the same cache location
 - Cause conflict misses when accessed consecutively

Direct-Mapped Caches

- **Direct-mapped cache:** Two blocks in memory that map to the same index in the cache cannot be present in the cache at the same time
 - One index → one entry
- Can lead to 0% hit rate if more than one block accessed in an interleaved manner map to the same index
 - Assume addresses A and B have the same index bits but different tag bits
 - A, B, A, B, A, B, A, B, ... → conflict in the cache index
 - All accesses are **conflict misses**

Set Associativity

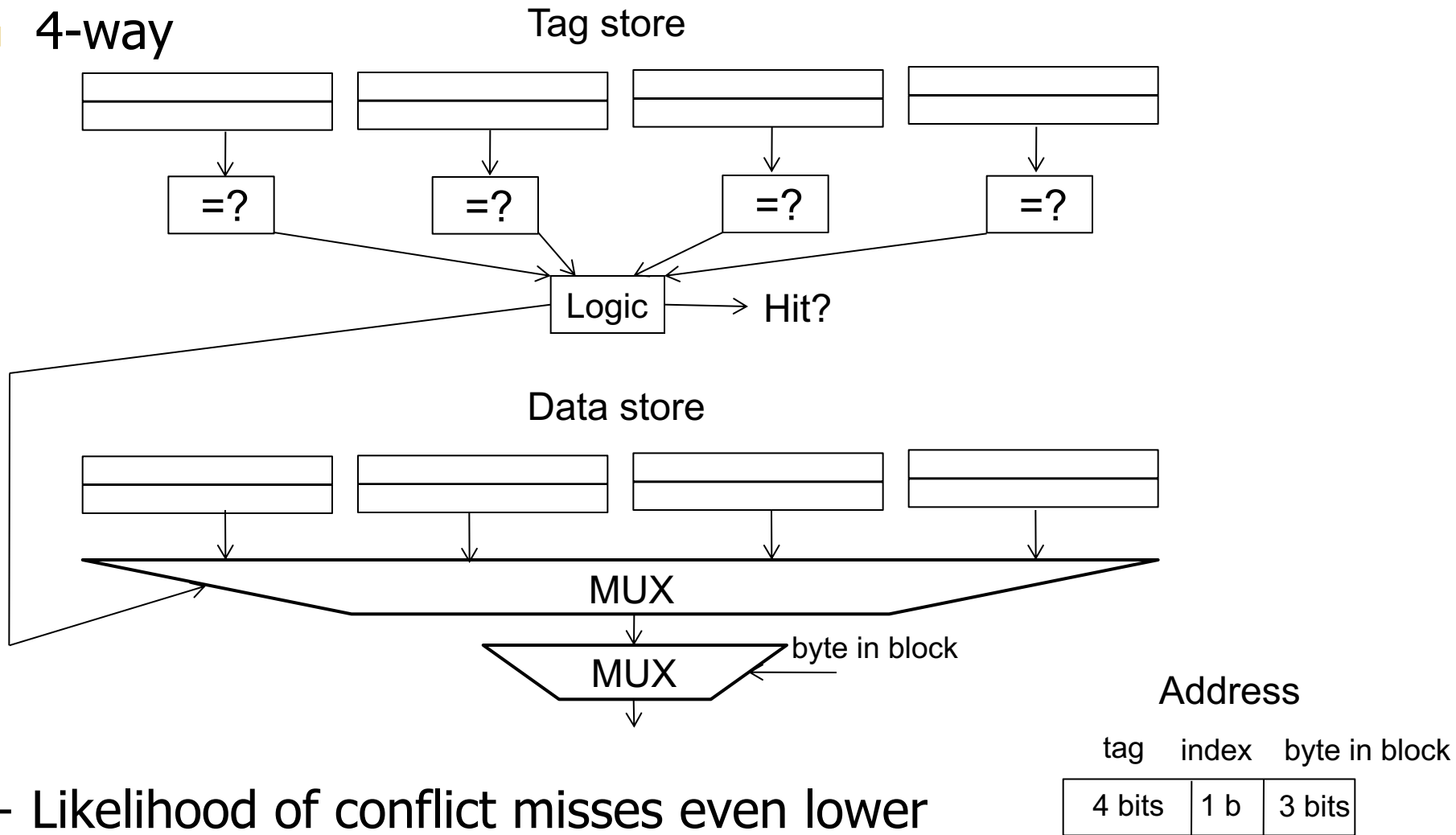
- Problem: Addresses N and $N+8$ always conflict in direct mapped cache
- Idea: enable blocks with the same index to map to > 1 cache location
- Example: Instead of having one column of 8, have 2 columns of 4 blocks



2-way set associative cache: Blocks with the same index can map to 2 locations

Higher Associativity

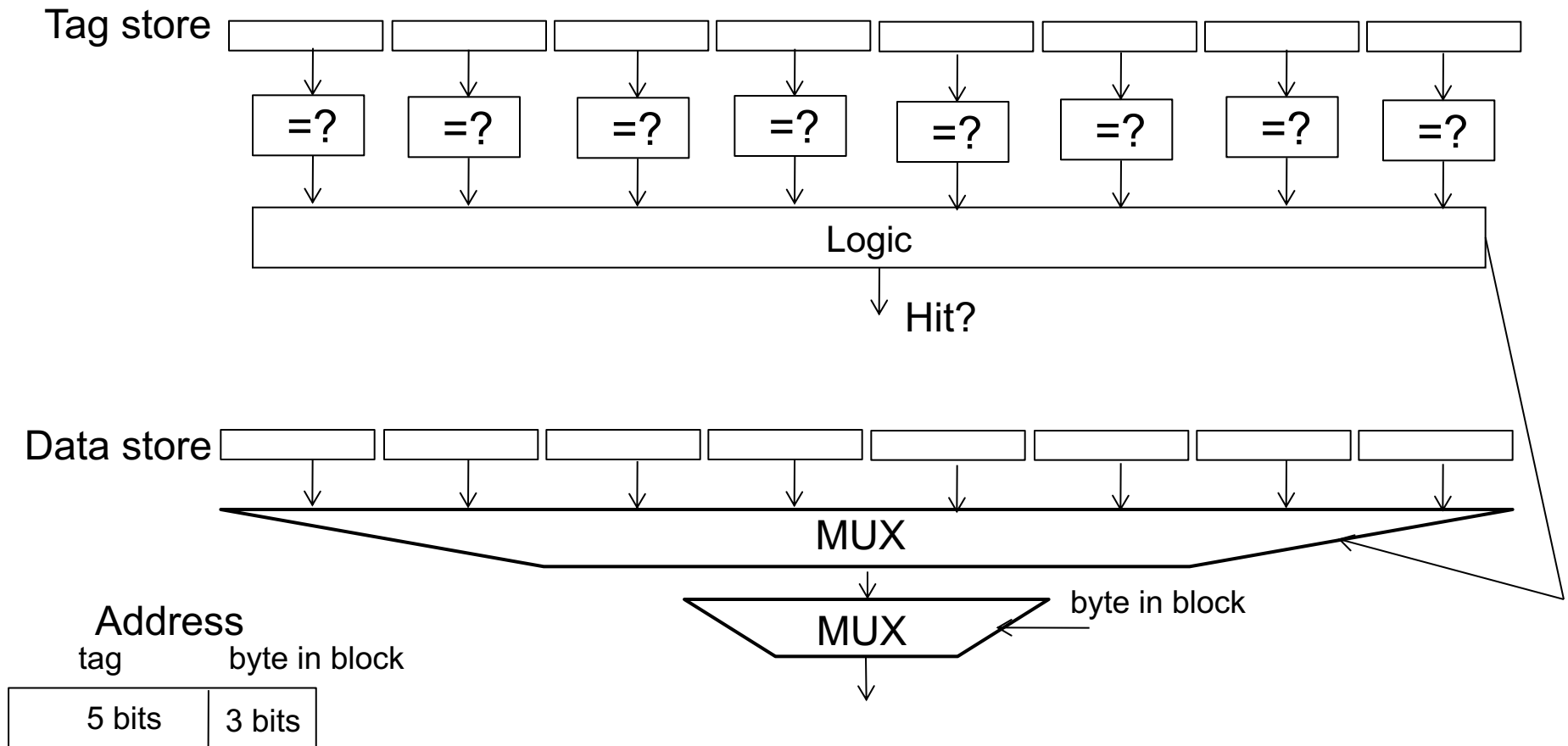
■ 4-way



4-way set associative cache: Blocks with the same index can map to 4 locations

Full Associativity

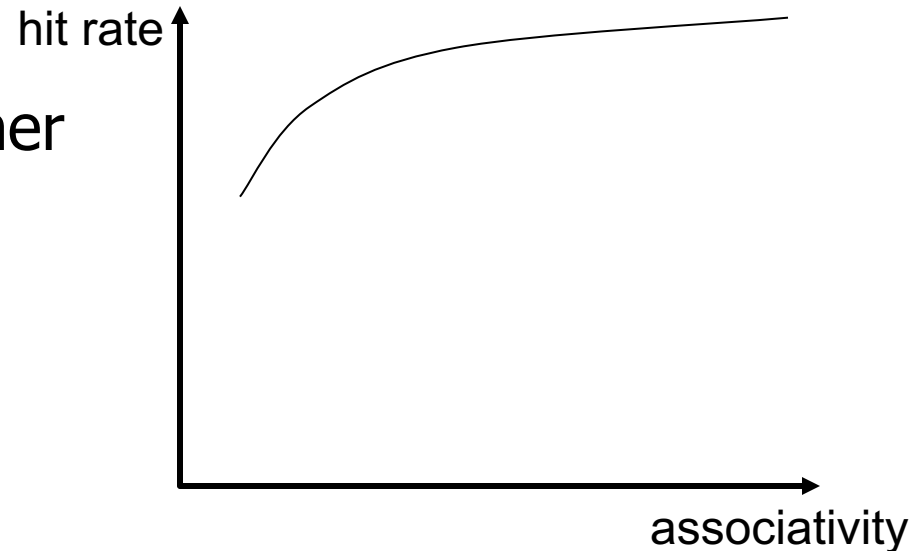
- Fully associative cache
 - A block can be placed in **any** cache location



Fully associative cache: Any block can map to any location in the cache

Associativity (and Tradeoffs)

- **Degree of associativity**: How many blocks can map to the same index (or set)?
- Higher associativity
 - ++ Higher hit rate
 - Slower cache access time (hit latency and data access latency)
 - More expensive hardware (more comparators)
- Diminishing returns from higher associativity



Issues in Set-Associative Caches

- Think of each block in a set having a “priority”
 - Indicating how important it is to keep the block in the cache
- Key issue: How do you determine/adjust block priorities?
- There are three key decisions in a set:
 - Insertion, promotion, eviction (replacement)
- Insertion: What happens to priorities on a cache fill?
 - Where to insert the incoming block; whether or not to insert the block
- Promotion: What happens to priorities on a cache hit?
 - Whether and how to change block priority
- Eviction/replacement: What happens to priorities on a cache miss?
 - Which block to evict and how to adjust priorities

Eviction/Replacement Policy

- Which block in the set to replace on a cache miss?
 - Any invalid block first
 - If all are valid, consult the replacement policy
 - Random
 - FIFO
 - Least recently used (how to implement?)
 - Not most recently used
 - Least frequently used?
 - Least costly to re-fetch?
 - Why would memory accesses have different cost?
 - Hybrid replacement policies
 - Optimal replacement policy?

Implementing LRU

- Idea: Evict the least recently accessed block
- Problem: Need to keep track of access order of blocks
- Question: 2-way set associative cache:
 - What do you minimally need to implement LRU perfectly?
- Question: 4-way set associative cache:
 - What do you minimally need to implement LRU perfectly?
 - How many different access orders are possible for the 4 blocks in the set?
 - How many bits needed to encode the LRU order of a block?
 - What is the logic needed to determine the LRU victim?
- Repeat for N-way set associative cache

Approximations of LRU

- Most modern processors do **not** implement “true LRU” (also called “perfect LRU”) in highly-associative caches
- Why?
 - True LRU is complex
 - LRU is an approximation to predict locality anyway (i.e., not the best possible cache management policy)
- Examples:
 - Not MRU (not most recently used)
 - Hierarchical LRU: divide the N-way set into M “groups”, track the MRU group and the MRU way in each group
 - Victim-NextVictim Replacement: Only keep track of the victim and the next victim

Cache Replacement Policy: LRU or Random

- LRU vs. Random: Which one is better?
 - Example: 4-way cache, cyclic references to A, B, C, D, E
 - 0% hit rate with LRU policy
- **Set thrashing:** When the “program working set” in a set is larger than set associativity
 - Random replacement policy is better when thrashing occurs
- In practice:
 - Performance of replacement policy depends on workload
 - Average hit rate of LRU and Random are similar
- Best of both Worlds: Hybrid of LRU and Random
 - How to choose between the two? **Set sampling**
 - See Qureshi et al., “**A Case for MLP-Aware Cache Replacement**,” ISCA 2006.

What Is the Optimal Replacement Policy?

- Belady's OPT
 - Replace the block that is going to be referenced furthest in the future by the program
 - Belady, “A study of replacement algorithms for a virtual-storage computer,” IBM Systems Journal, 1966.
 - How do we implement this? Simulate?
- Is this optimal for minimizing miss rate?
- Is this optimal for minimizing execution time?
 - No. Cache miss latency/cost varies from block to block!
 - Two reasons: Where miss is serviced from and miss overlapping
 - Qureshi et al. “A Case for MLP-Aware Cache Replacement,” ISCA 2006.

Recommended Reading

- Key observation: **Some misses more costly than others** as their latency is exposed as stall time. **Reducing miss rate is not always good for performance.** Cache replacement should take into account cost of misses.
- Moinuddin K. Qureshi, Daniel N. Lynch, Onur Mutlu, and Yale N. Patt, **"A Case for MLP-Aware Cache Replacement"**
Proceedings of the 33rd International Symposium on Computer Architecture (ISCA), pages 167-177, Boston, MA, June 2006. [Slides \(ppt\)](#)

A Case for MLP-Aware Cache Replacement

Moinuddin K. Qureshi Daniel N. Lynch Onur Mutlu Yale N. Patt
Department of Electrical and Computer Engineering
The University of Texas at Austin
{moin, lynch, onur, patt}@hps.utexas.edu

What's In A Tag Store Entry?

- Valid bit
- Tag
- Replacement policy bits

- Dirty bit?
 - Write back vs. write through caches

Handling Writes (I)

■ When do we write the modified data in a cache to the next level?

■ Write through: At the time the write happens (∴ in write through once written in LLC, then

■ Write back: When the block is evicted

(In write back, when we write into LLC, don't write into next level mark it as dirty in LLC when the block in LLC abt to be evicted we then write into the cache at next level) itself write through caches in next level too).

■ Write-back cache

+ Can combine multiple writes to the same block before eviction

■ Potentially saves bandwidth between cache levels + saves energy

-- Need a bit in the tag store indicating the block is "dirty/modified"

∴ we check if both valid = 1 & tag match.

valid bit = 1 → cache block contains original data.

■ Write-through cache

then only cache hit ~~if~~ 0 ⇒ store data.

+ Simpler design

+ All levels are up to date & consistent → Simpler cache coherence: no need to check close-to-processor caches' tag stores for presence

-- More bandwidth intensive; no combining of writes

Handling Writes (II)

■ Do we allocate a cache block on a write miss?

□ Allocate on write miss: Yes

→ write allocate policy. ↗

□ No-allocate on write miss: No

→ Write around (No write allocate policy)

↳ When a cache block to be written is not present

don't bring it to L1C in main memory) instead write it directly

■ Allocate on write miss

+ Can combine writes instead of writing each individually to next level
(write at cache)

↑ if in read miss or miss we bring block from MM to cache

+ Simpler because write misses can be treated the same way as read misses
↳ read it from cache.

-- Requires transfer of the whole cache block

■ No-allocate

↗ ∴ the block is not being brought ? stored in cache.

+ Conserves cache space if locality of written blocks is low
(potentially better cache hit rate)

When block to be written is not in cache then bring it from next level and write it.

Handling Writes (III)

- (What if the processor writes to an entire block over a small amount of time?) ~~no~~
- Is there any need to bring the block into the cache from memory in the first place?
- Why do we not simply write to only a *portion* of the block, i.e., subblock?
 - E.g., 4 bytes out of 64 bytes
 - Problem (Valid and dirty bits are associated with the entire 64 bytes, not with each individual 4 bytes)

∴ ~~Through~~ only some partition of the block

is written ∴ $v = 0$ ^{treat} ⇒ whole cache block is dirty.

Subblocked (Sectored) Caches

- Idea: Divide a block into subblocks (or sectors)
 - Have separate valid and dirty bits for each subblock (sector)
 - Allocate only a subblock (or a subset of subblocks) on a request

++ No need to transfer the entire cache block into the cache
(A write simply validates and updates a subblock)

++ More freedom in transferring subblocks into the cache (a cache block does not need to be in the cache fully)
(How many subblocks do you transfer on a read?)

-- More complex design; more valid and dirty bits

-- May not exploit spatial locality fully

Block divided into
subblocks



Each has its own valid & dirty bit.

Instruction vs. Data Caches

- **Separate or Unified?**
- **Pros and Cons of Unified:**
 - + Dynamic sharing of cache space → **better overall cache utilization**: no overprovisioning that might happen with static partitioning of cache space (i.e., separate I and D caches)
 - Instructions and data can evict/thrash each other (i.e., no guaranteed space for either)
 - I and D are accessed in different places in the pipeline. Where do we place the unified cache for fast access?
- First level caches are almost always split
 - Mainly for the last reason above – pipeline constraints
- Outer level caches are almost always unified

Multi-level Cache Design & Management

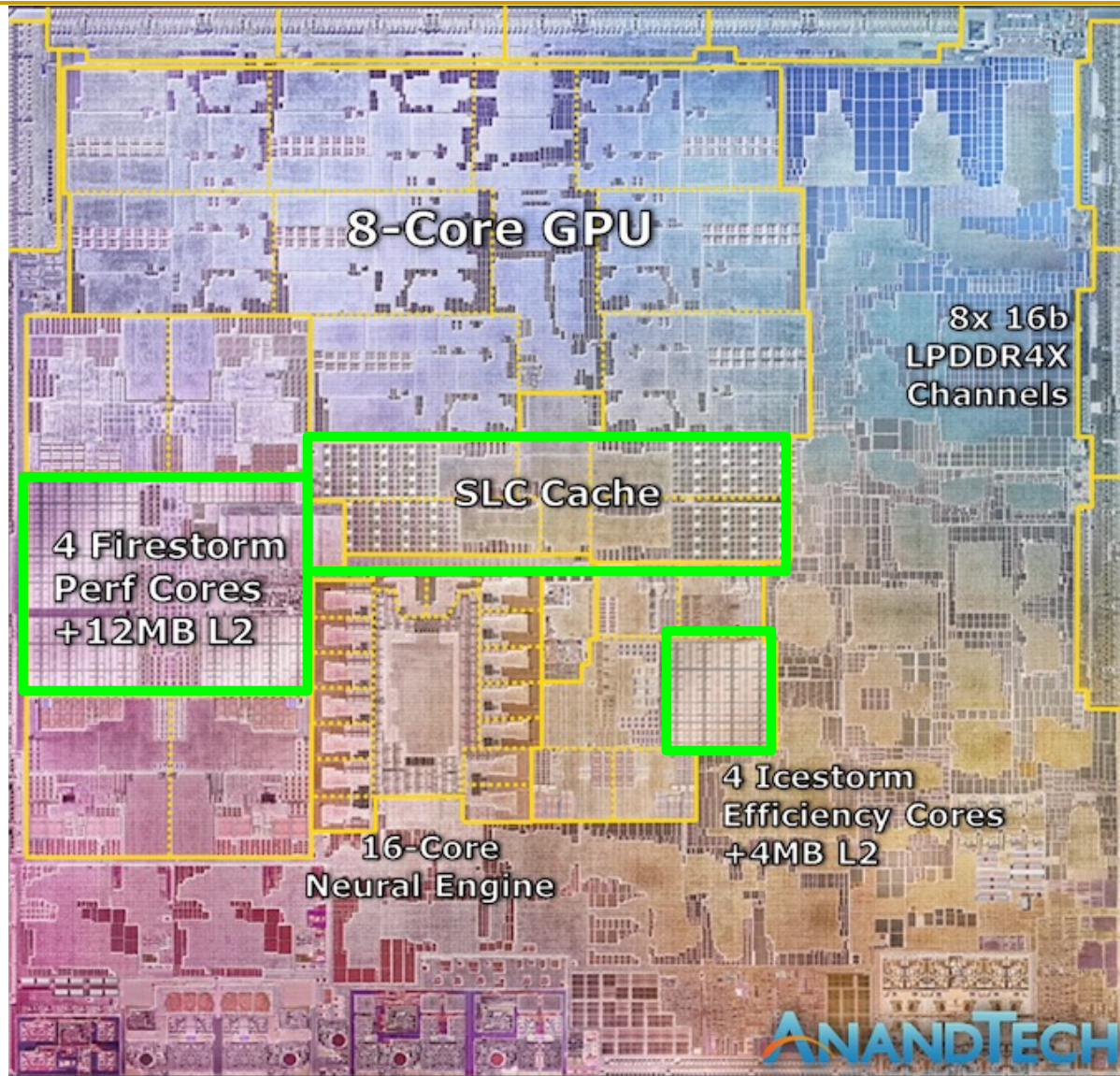
- Cache level greatly affects cache design & management
- First-level caches (instruction and data)
 - Decisions very much affected by cycle time & pipeline structure
 - Small, lower associativity; latency is critical
 - Tag store and data store are usually accessed in parallel
- Second-level caches
 - Decisions need to balance hit rate and access latency
 - Usually large and highly associative; latency not as important
 - Tag store and data store can be accessed serially
- Further-level (larger) caches
 - Access energy is a larger problem due to cache sizes
 - Tag store and data store are usually accessed serially

Serial vs. Parallel Access of Cache Levels

- Parallel: Next level cache accessed in parallel with the previous level → a form of speculative access
 - + Faster access to data if previous level misses
 - Unnecessary accesses to next level if previous level hits

- Serial: Next level cache accessed only if previous-level misses
 - Slower access to data if previous level misses
 - + No wasted accesses to next level if previous level hits
 - Next level does not see the same accesses as the previous
 - Previous level acts as a filter (filters some temporal & spatial locality)
 - **Management policies are different across cache levels**

Deeper and Larger Cache Hierarchies



Apple M1,
2021

Deeper and Larger Cache Hierarchies

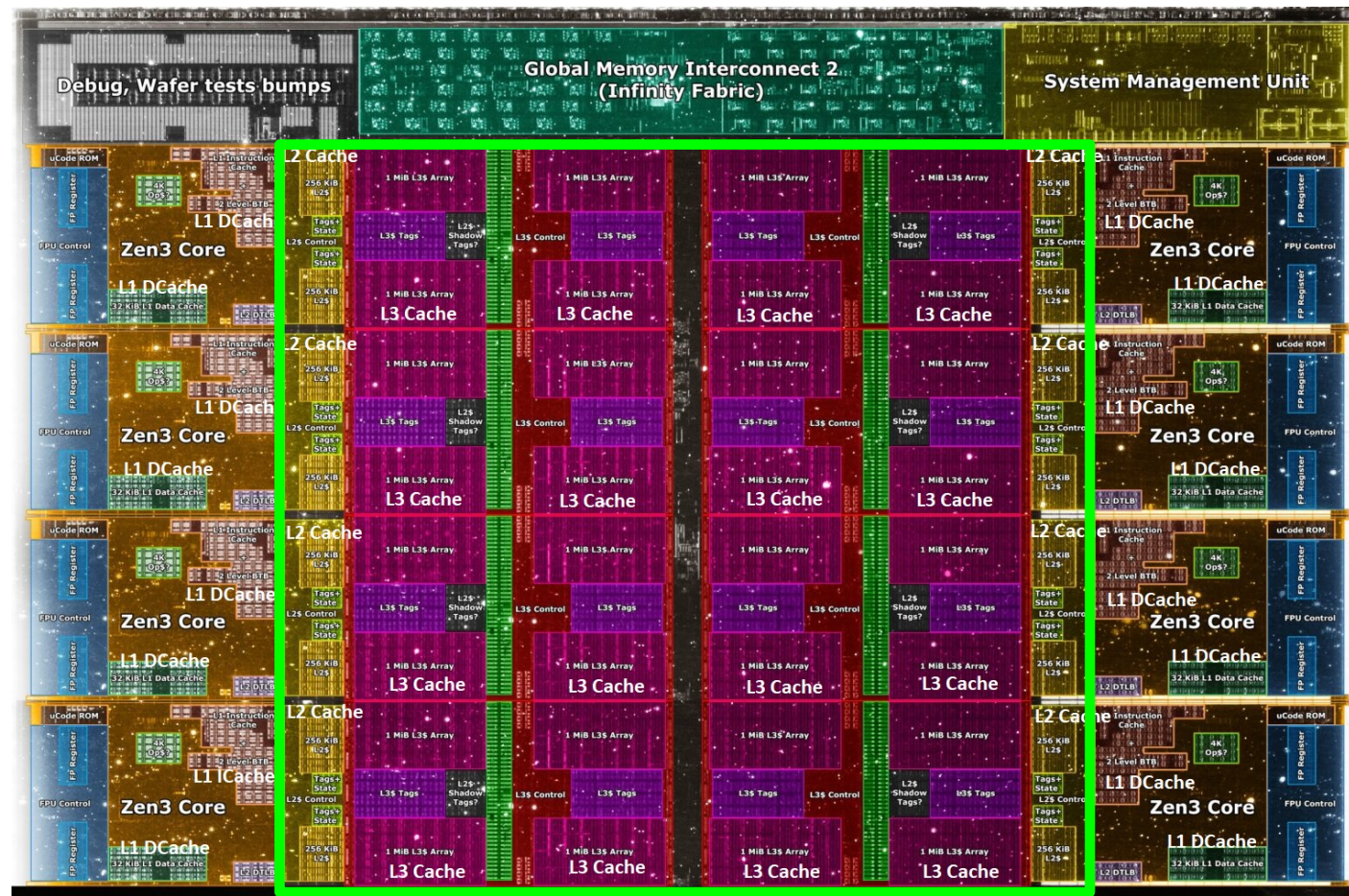


10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>

Die shot interpretation by Locuza, October 2021

Intel Alder Lake,
2021

Deeper and Larger Cache Hierarchies



Core Count:
8 cores/16 threads

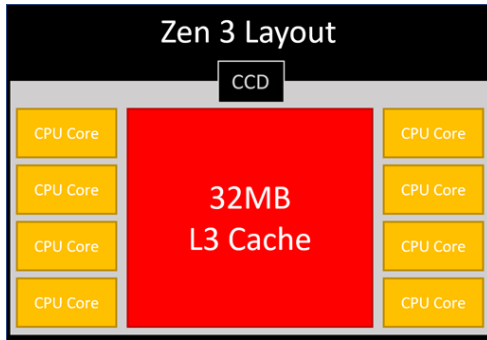
L1 Caches:
32 KB per core

L2 Caches:
512 KB per core

L3 Cache:
32 MB shared

AMD Ryzen 5000, 2020

AMD's 3D Last Level Cache (2021)

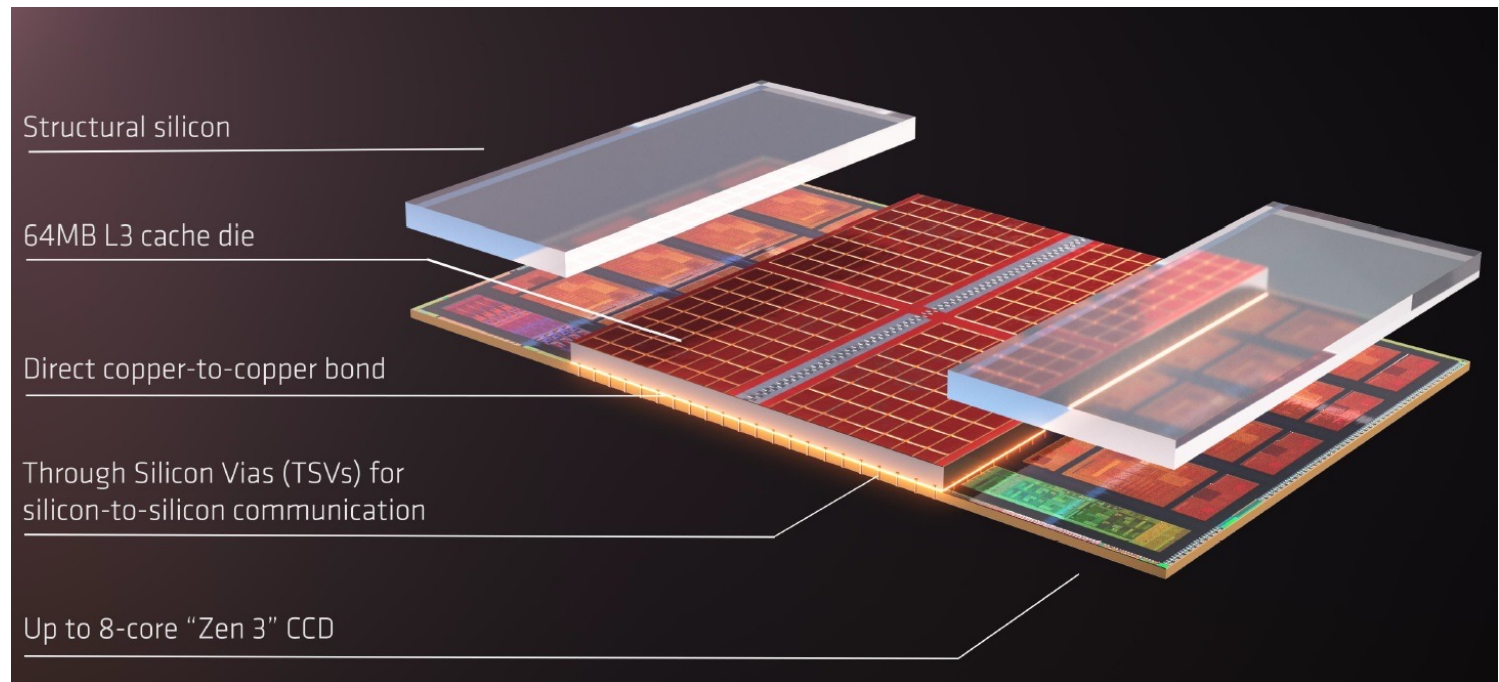


<https://community.microcenter.com/discussion/5134/comparing-zen-3-to-zen-2>

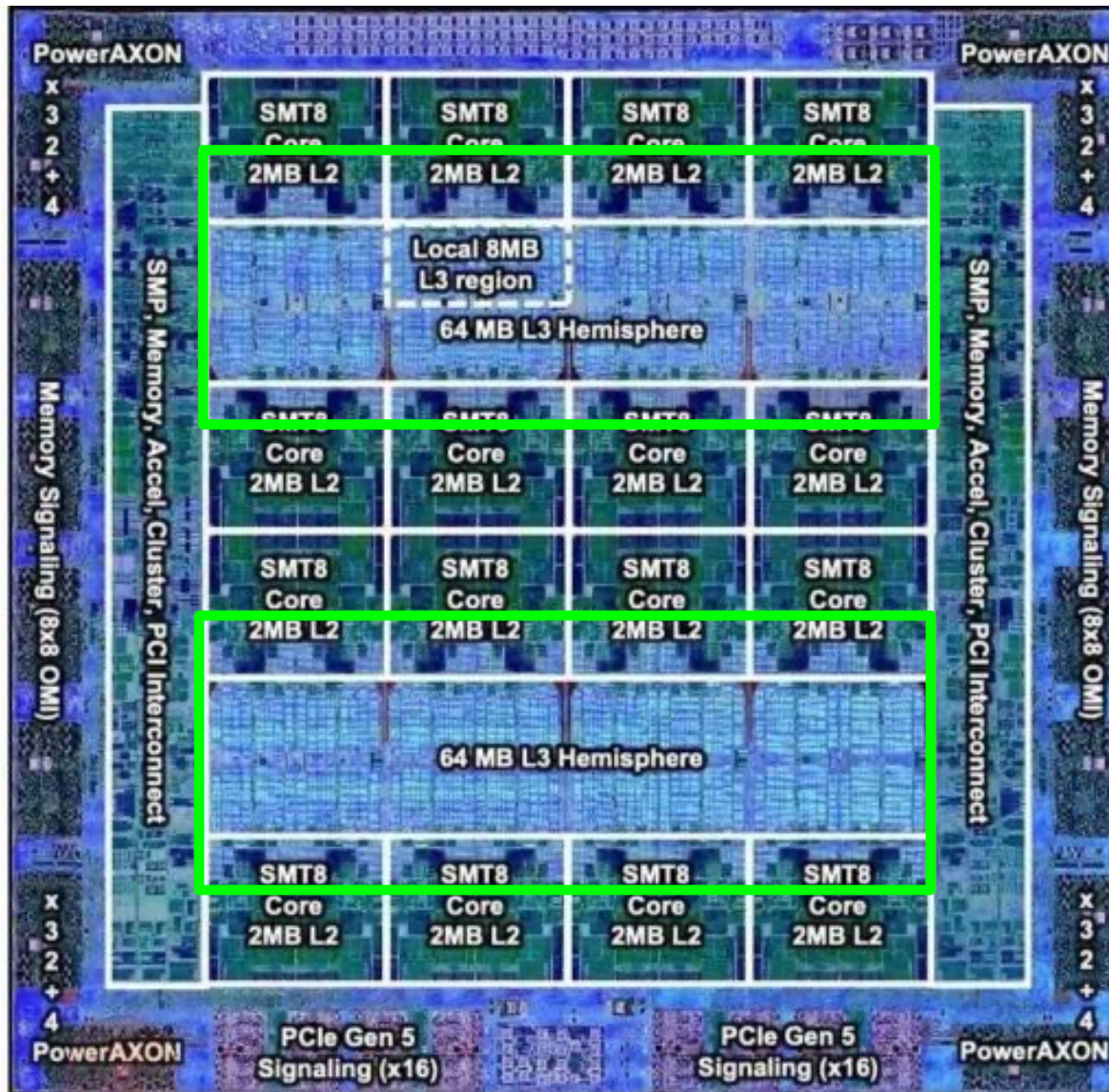
AMD increases the L3 size of their 8-core Zen 3 processors from 32 MB to 96 MB

Additional 64 MB L3 cache die
stacked on top of the processor die

- Connected using Through Silicon Vias (TSVs)
- Total of 96 MB L3 cache



Deeper and Larger Cache Hierarchies



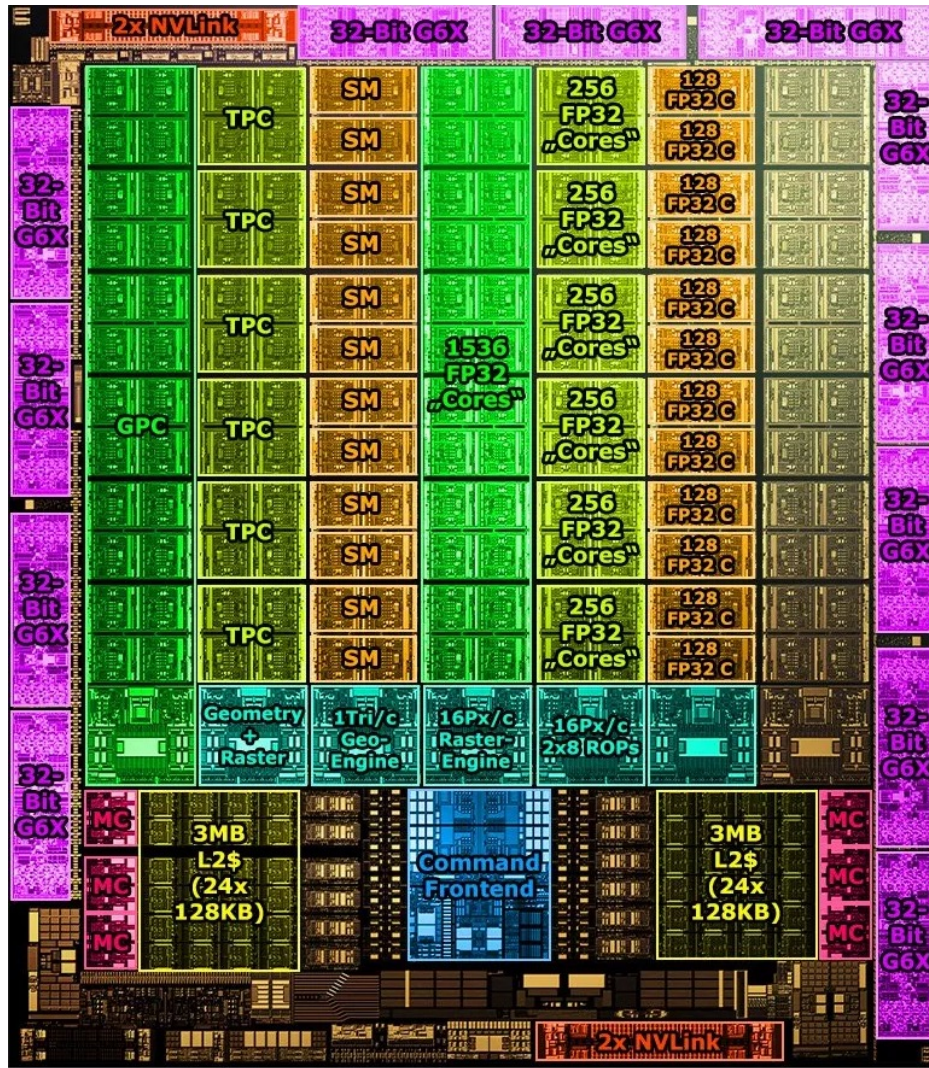
IBM POWER10,
2020

Cores:
15-16 cores,
8 threads/core

L2 Caches:
2 MB per core

L3 Cache:
120 MB shared

Deeper and Larger Cache Hierarchies



Nvidia Ampere, 2020

Cores:

128 Streaming Multiprocessors

L1 Cache or Scratchpad:

192KB per SM

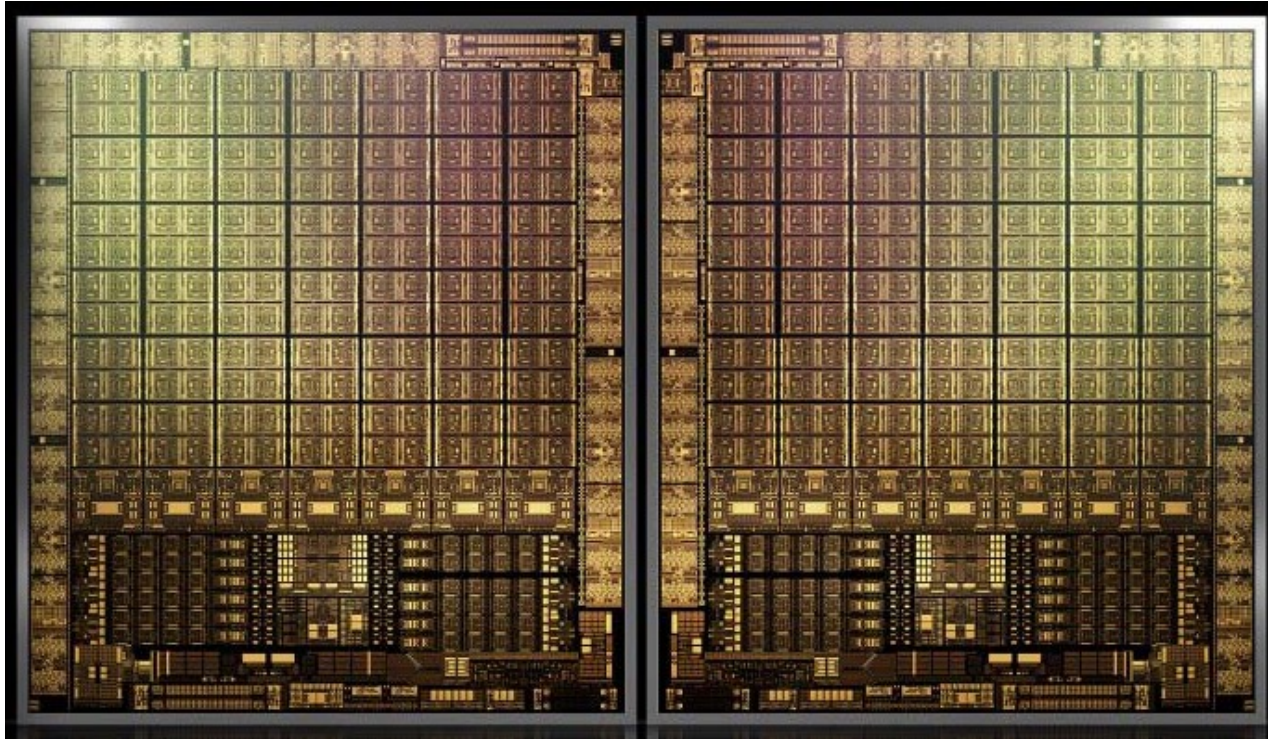
Can be used as L1 Cache and/or Scratchpad

L2 Cache:

40 MB shared

Deeper and Larger Cache Hierarchies

Nvidia Hopper, 2022



Cores:

144 Streaming
Multiprocessors

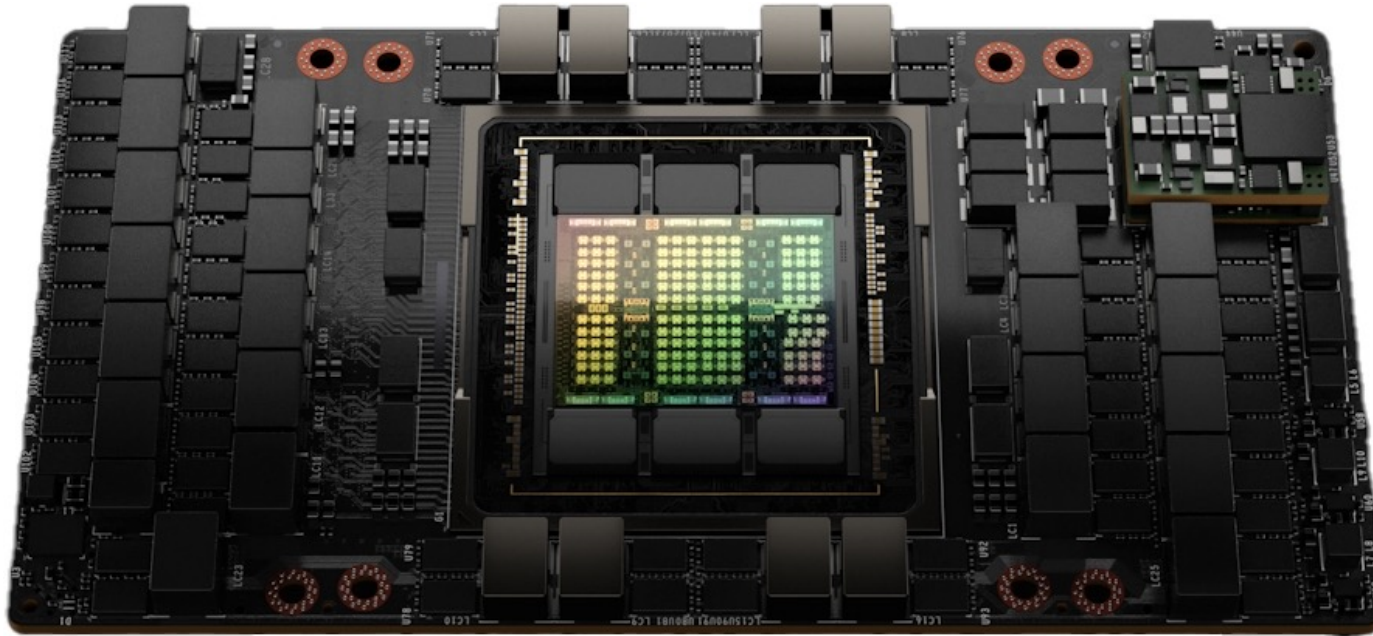
L1 Cache or Scratchpad:

256KB per SM
Can be used as L1 Cache
and/or Scratchpad

L2 Cache:

60 MB shared

Deeper and Larger Cache Hierarchies



Nvidia Hopper,
2022

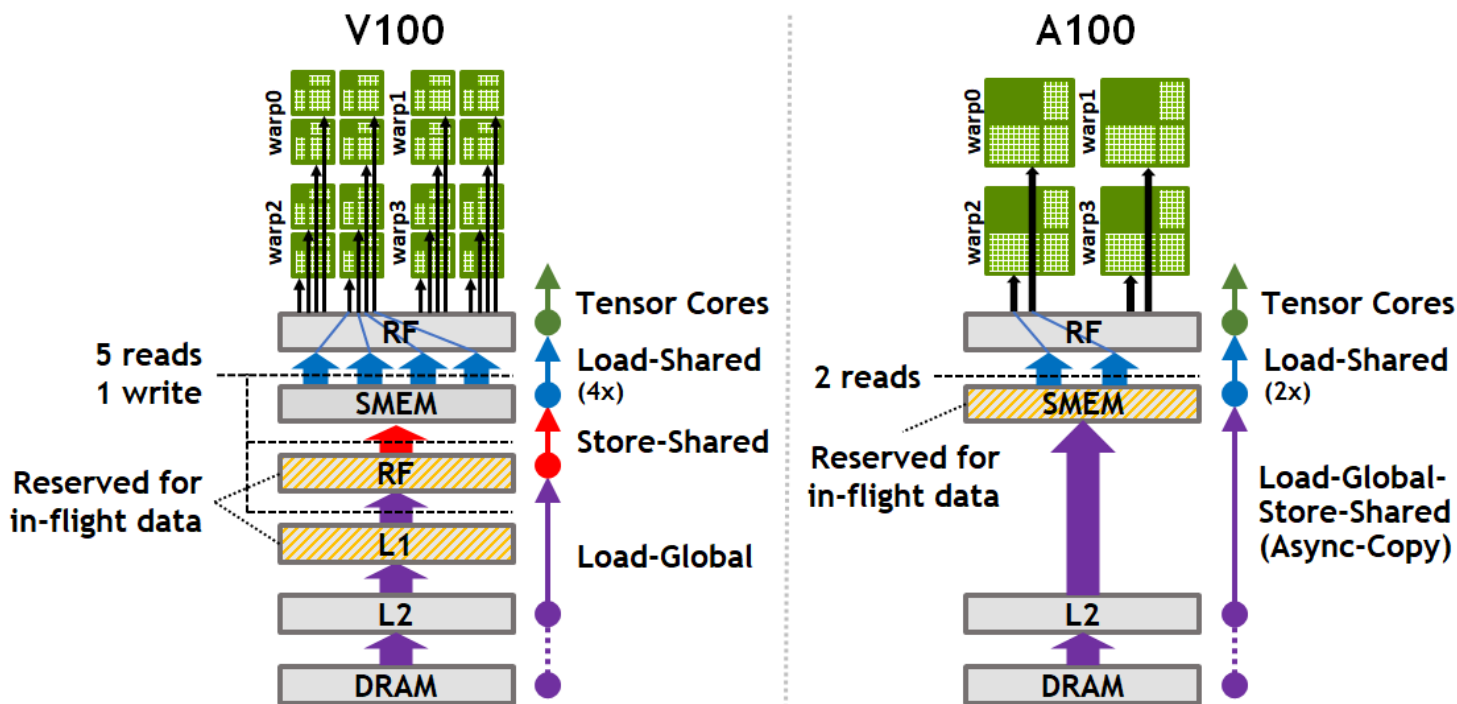
Cores:
144 Streaming
Multiprocessors

**L1 Cache or
Scratchpad:**
256KB per SM
Can be used as L1 Cache
and/or Scratchpad

L2 Cache:
60 MB shared

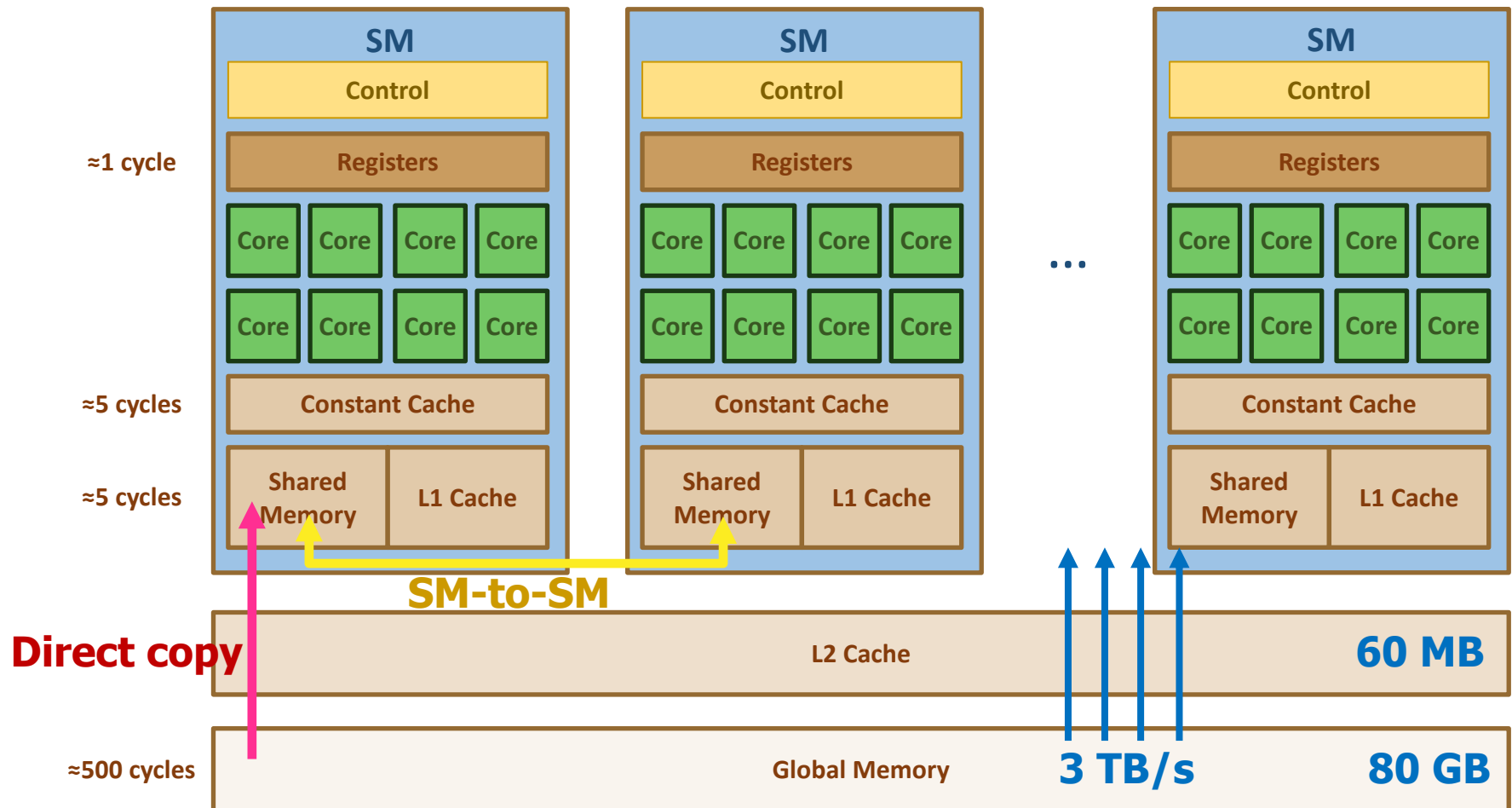
NVIDIA V100 & A100 Memory Hierarchy

- Example of data movement between GPU global memory (DRAM) and GPU cores.



A100 feature:
Direct copy from L2
to scratchpad,
bypassing L1 and
register file.

Memory in the NVIDIA H100 GPU



Multi-Level Cache Design Decisions

- Which level(s) to place a block into (from memory)?
- Which level(s) to evict a block to (from an inner level)?
- Bypassing vs. non-bypassing levels
- Inclusive, exclusive, non-inclusive hierarchies
 - **Inclusive:** a block in an inner level is always included also in an outer level → simplifies cache coherence
 - **Exclusive:** a block in an inner level does not exist in an outer level → better utilizes space in the entire hierarchy
 - **Non-inclusive:** a block in an inner level may or may not be included in an outer level → relaxes design decisions

We Covered Until Here in Lecture

Digital Design & Computer Arch.

Lecture 22: Caches

Dr. Mohammad Sadrosadati

Prof. Onur Mutlu

ETH Zürich

Spring 2025

16 May 2025

**Further Slides for Your Own Study
(May Be Covered in Future Lectures)**

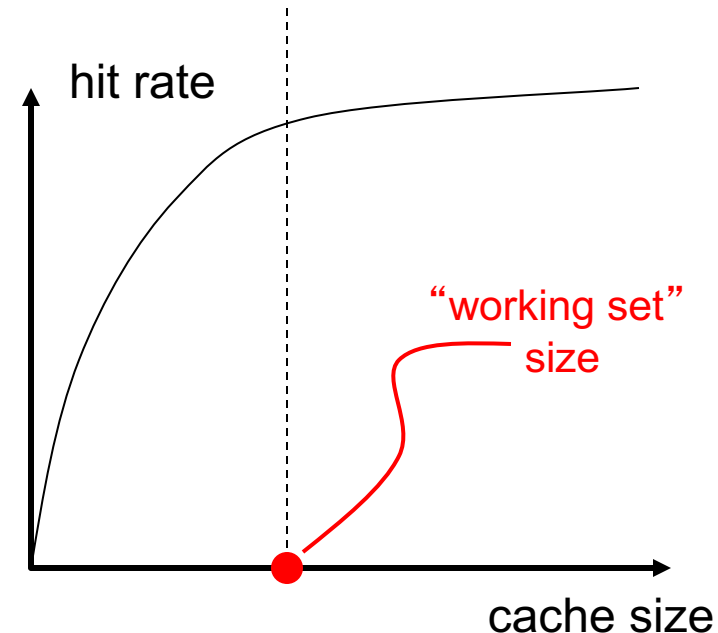
Cache Performance

Cache Parameters vs. Miss/Hit Rate

- Cache size
- Block size
- Associativity
- Replacement policy
- Insertion/Placement policy
- Promotion Policy

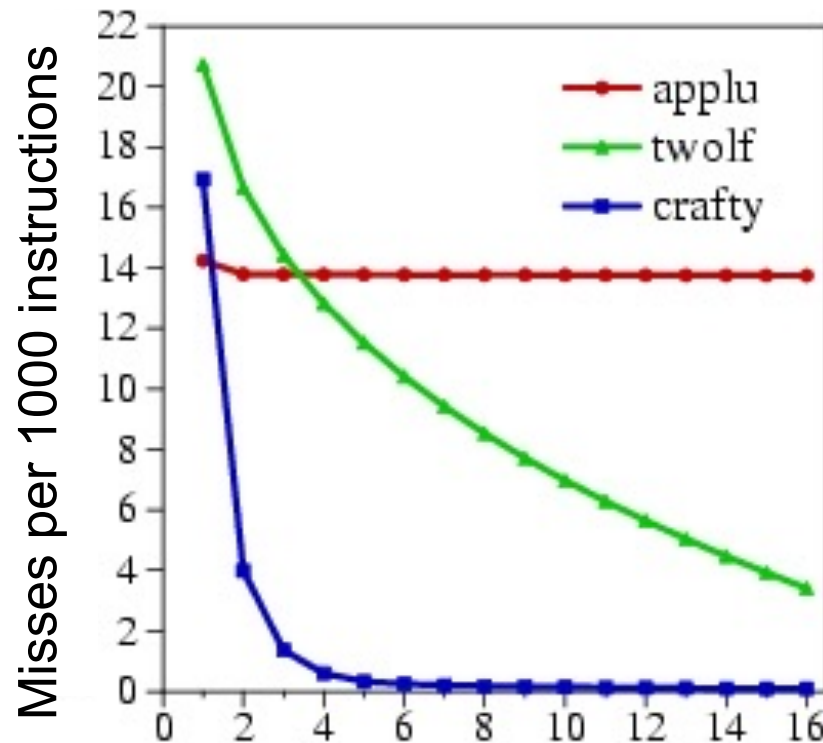
Cache Size

- Cache size: total data capacity (not including tag store)
 - bigger cache can exploit temporal locality better
- **Too large** a cache adversely affects hit and miss latency
 - bigger is slower
- **Too small** a cache
 - does not exploit temporal locality well
 - useful data replaced often
- **Working set**: entire set of data the executing application references
 - Within a time interval



Benefits of Larger Caches Widely Varies

- Benefits of cache size widely varies across applications



Number of ways from 16-way 1MB L2

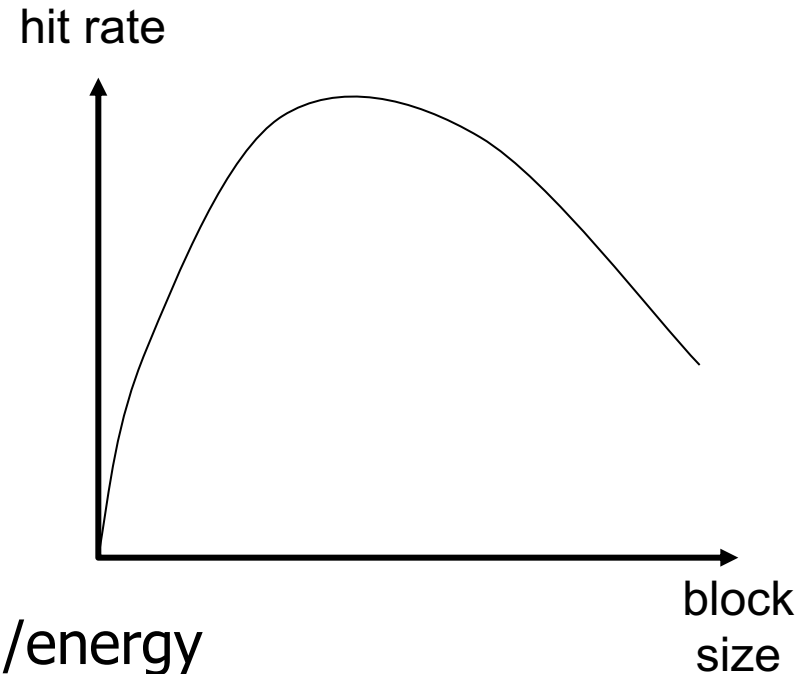
Low Cache Utility

High Cache Utility

Saturating Cache Utility

Block Size

- Block size is the data that is associated with an address tag
 - not necessarily the unit of transfer between hierarchies
 - Sub-blocking: A block divided into multiple pieces (each w/ V/D bits)
- Too small blocks
 - do not exploit spatial locality well
 - have larger tag overhead
- Too large blocks
 - too few total blocks $\rightarrow$ do not exploit temporal locality well
 - waste cache space and bandwidth/energy if spatial locality is not high



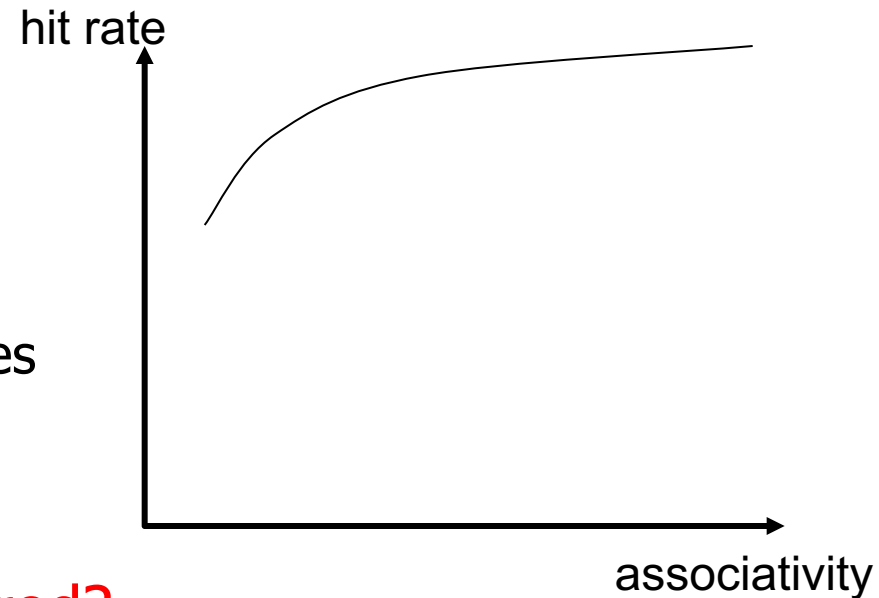
Large Blocks: Critical-Word and Subblocking

- Large cache blocks can take a long time to fill into the cache
 - ❑ Idea: Fill cache block **critical-word first**
 - ❑ Supply the critical data to the processor immediately
- Large cache blocks can waste bus bandwidth
 - ❑ Idea: Divide a block into **subblocks**
 - ❑ Associate separate valid and dirty bits for each subblock
 - ❑ **Recall: When is this useful?**



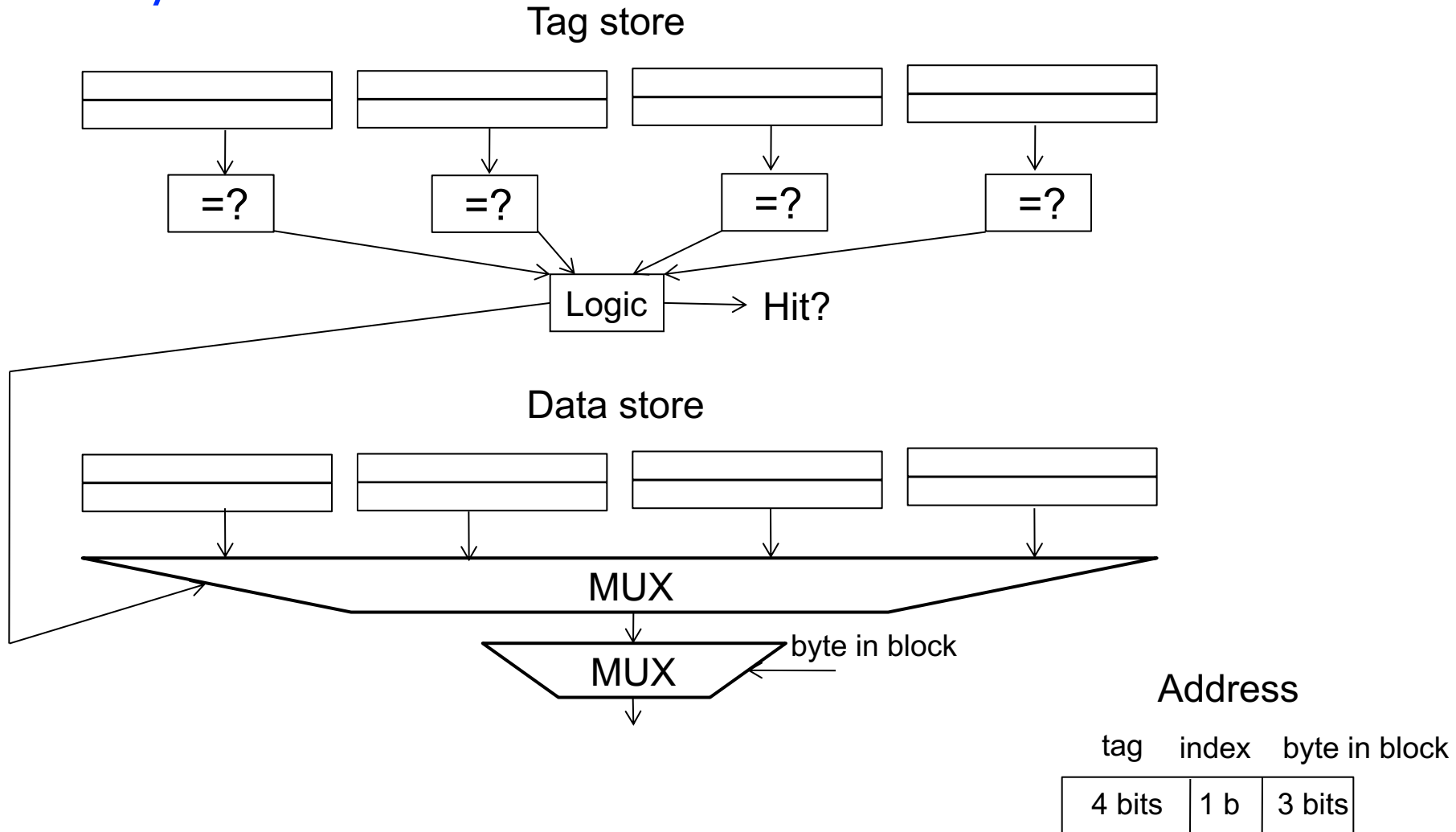
Associativity

- How many blocks can be present in the same index (i.e., set)?
- Larger associativity
 - ❑ lower miss rate (reduced conflicts)
 - ❑ higher hit latency and area cost
- Smaller associativity
 - ❑ lower cost
 - ❑ lower hit latency
 - Especially important for L1 caches
- Is power of 2 associativity required?



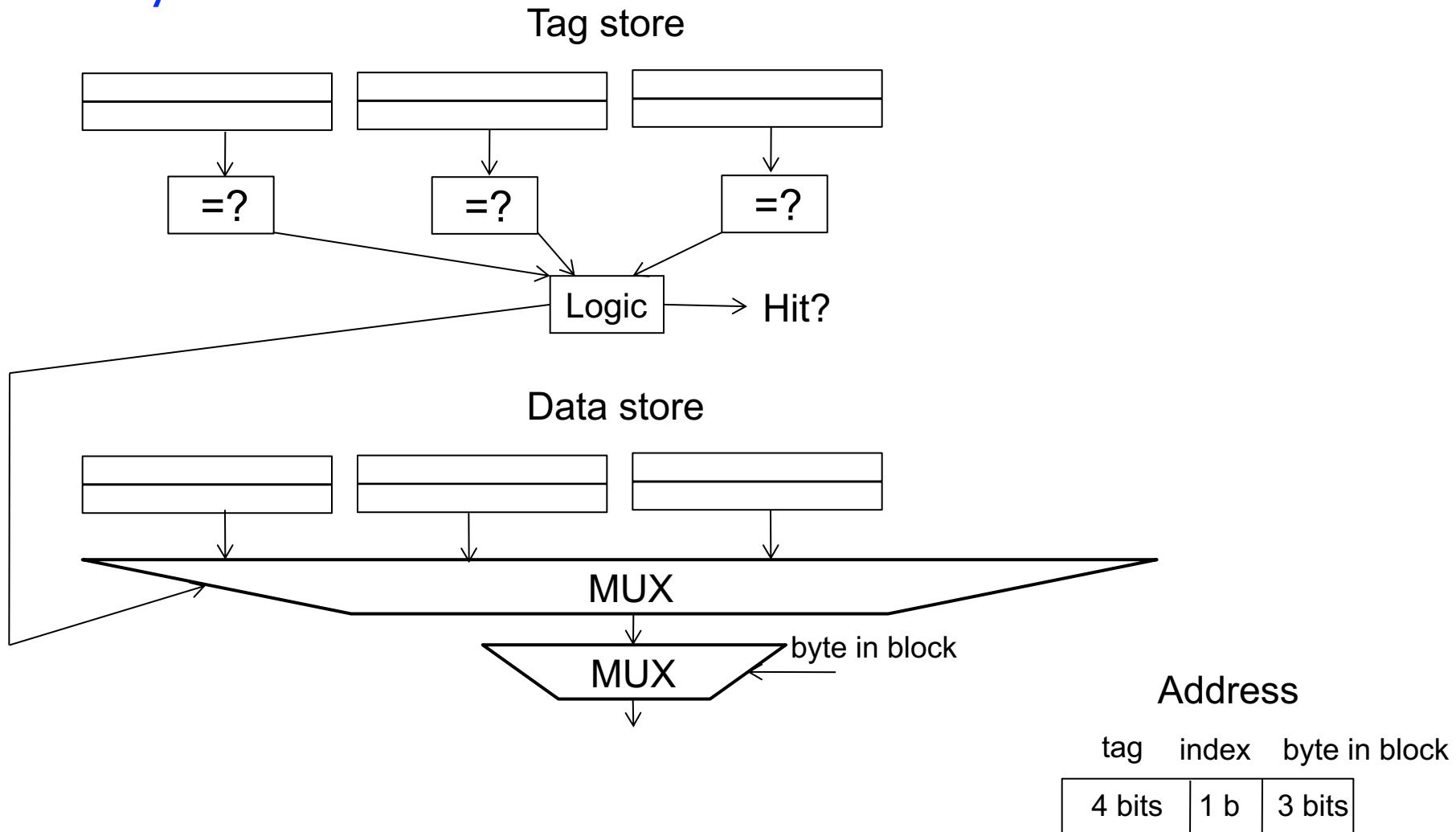
Recall: Higher Associativity (4-way)

■ 4-way

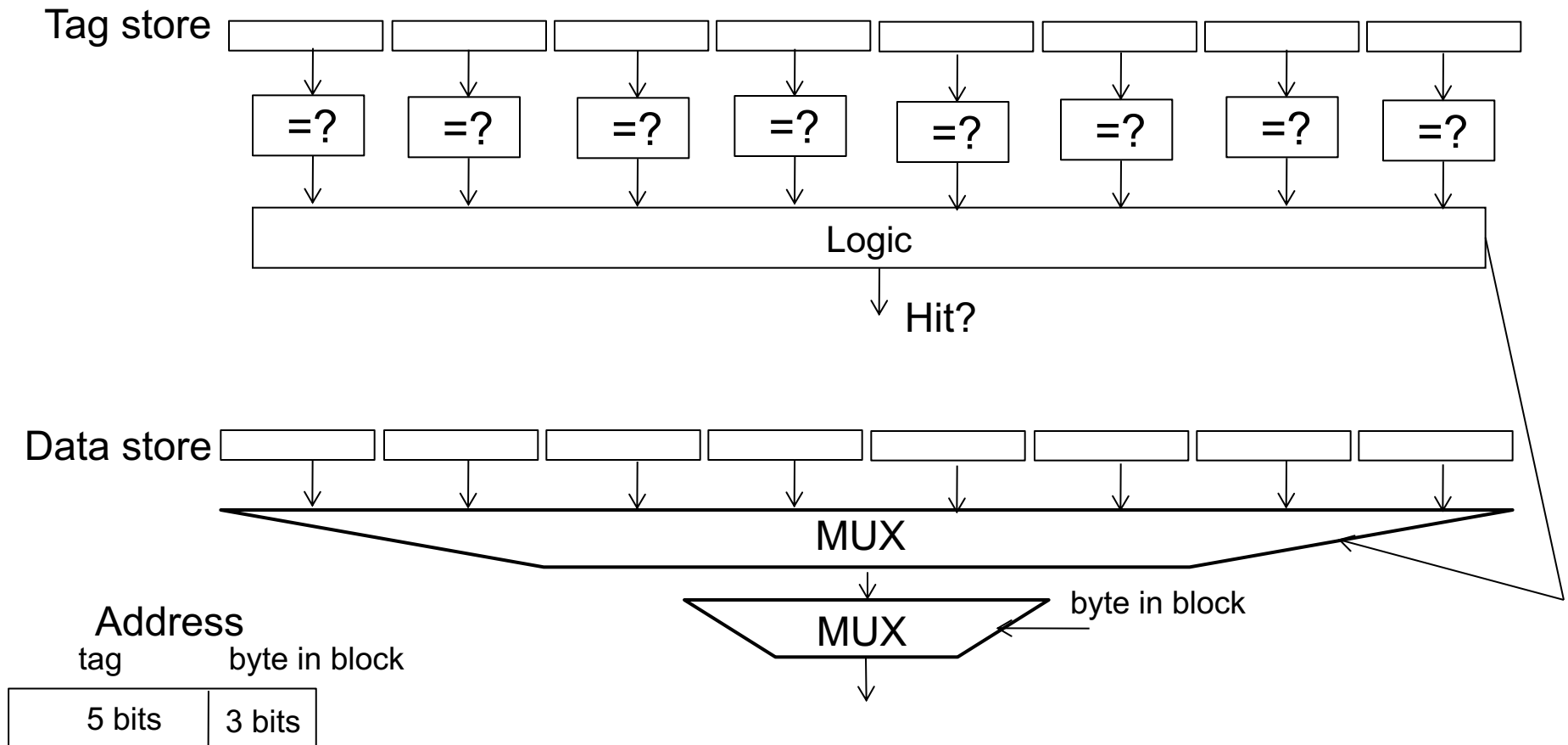


Higher Associativity (3-way)

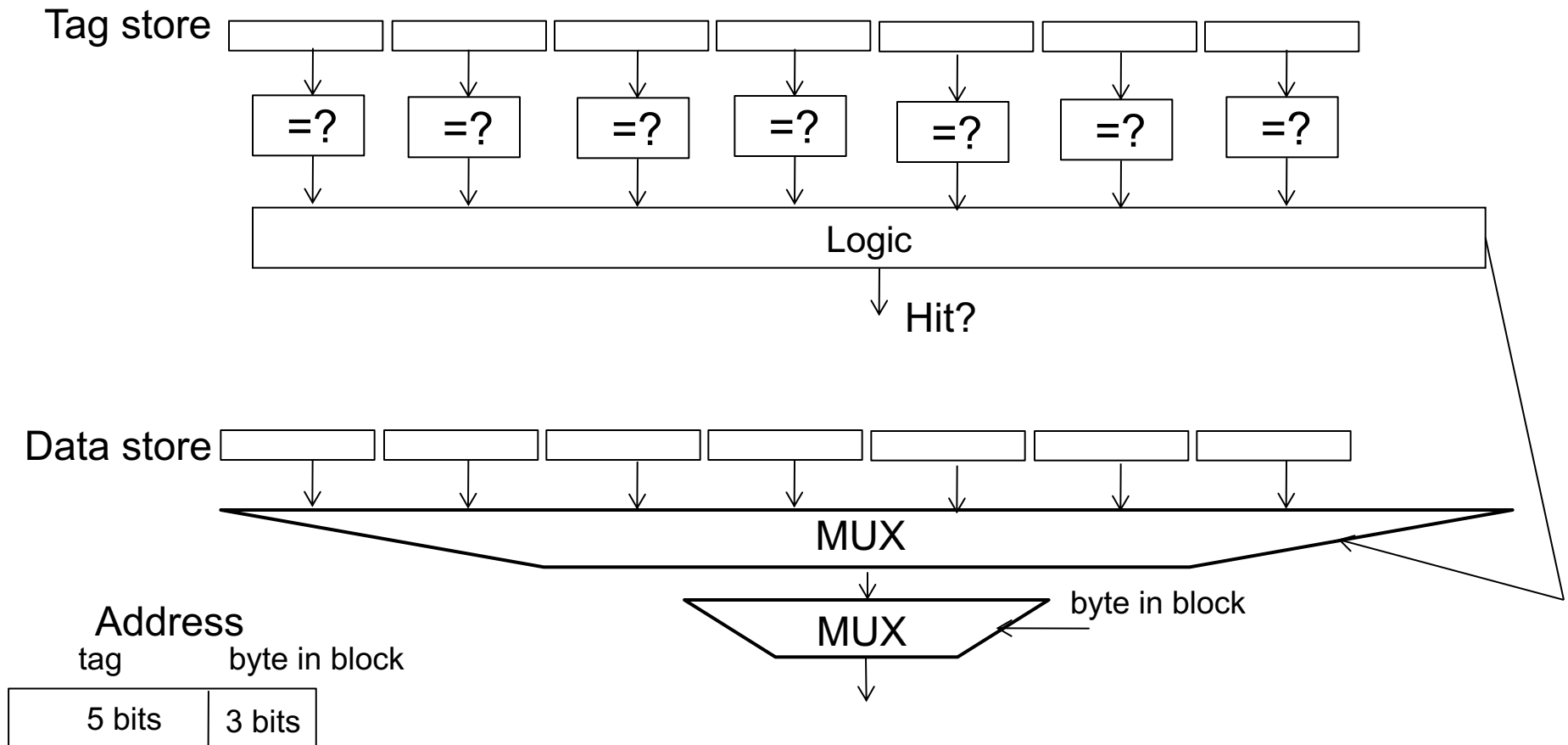
■ 3-way



Recall: 8-way Fully Associative Cache



7-way Fully Associative Cache



Classification of Cache Misses

■ Compulsory miss

- ❑ first reference to an address (block) always results in a miss
- ❑ subsequent references to the block should hit in cache unless the block is displaced from cache for the reasons below

■ Capacity miss

- ❑ cache is too small to hold all needed data
- ❑ defined as the misses that would occur even in a fully-associative cache (with optimal replacement) of the same capacity

■ Conflict miss

- ❑ defined as any miss that is neither a compulsory nor a capacity miss

How to Reduce Each Miss Type

■ Compulsory

- ❑ Caching (only accessed data) cannot help; larger blocks can
- ❑ Prefetching helps: Anticipate which blocks will be needed soon

■ Conflict

- ❑ More associativity
- ❑ Other ways to get more associativity without making the cache associative
 - Victim cache
 - Better, randomized indexing into the cache
 - Software hints for eviction/replacement/promotion

■ Capacity

- ❑ Utilize cache space better: keep blocks that will be referenced
- ❑ Software management: divide working set and computation such that each “computation phase” fits in cache

How to Improve Cache Performance

- Three fundamental goals
- Reducing miss rate
 - Caveat: reducing miss rate can reduce performance if more costly-to-refetch blocks are evicted
- Reducing miss latency or miss cost
- Reducing hit latency or hit cost
- The above three **together** affect performance

Improving Basic Cache Performance

■ Reducing miss rate

- ❑ More associativity
- ❑ Alternatives/enhancements to associativity
 - Victim caches, hashing, pseudo-associativity, skewed associativity
- ❑ Better replacement/insertion policies
- ❑ **Software approaches**

■ Reducing miss latency/cost

- ❑ Multi-level caches
- ❑ Critical word first
- ❑ Subblocking/sectoring
- ❑ **Better replacement/insertion policies**
- ❑ Non-blocking caches (multiple cache misses in parallel)
- ❑ Multiple accesses per cycle
- ❑ **Software approaches**

Software Approaches for Higher Hit Rate

- Restructuring data access patterns
- Restructuring data layout

- Loop interchange
- Data structure separation/merging
- Blocking
- ...

Restructuring Data Access Patterns (I)

- Idea: Restructure data layout or data access patterns
- Example: If column-major
 - $x[i+1,j]$ follows $x[i,j]$ in memory
 - $x[i,j+1]$ is far away from $x[i,j]$

Poor code

```
for i = 1, rows
  for j = 1, columns
    sum = sum +  $x[i,j]$ 
```

Better code

```
for j = 1, columns
  for i = 1, rows
    sum = sum +  $x[i,j]$ 
```

- This is called **loop interchange**
- Other optimizations can also increase hit rate
 - Loop fusion, array merging, ...

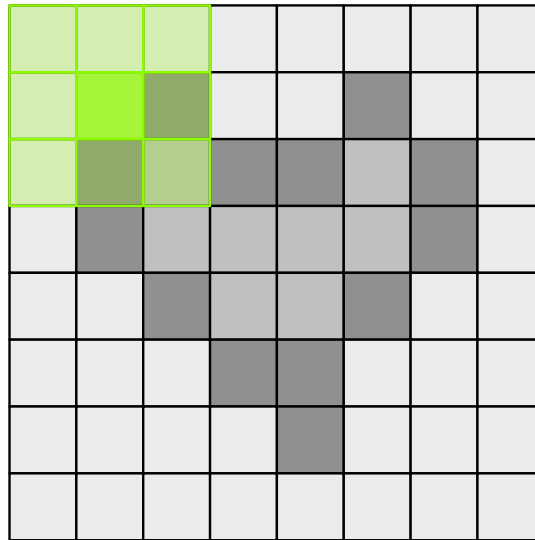
Restructuring Data Access Patterns (II)

- **Blocking**
 - ❑ Divide loops operating on arrays into computation chunks so that each chunk can hold its data in the cache
 - ❑ Avoids cache conflicts between different chunks of computation
 - ❑ Essentially: Divide the working set so that each piece fits in the cache
- Also called Tiling

Data Reuse: An Example from GPU Computing

- Same memory locations accessed by neighboring threads

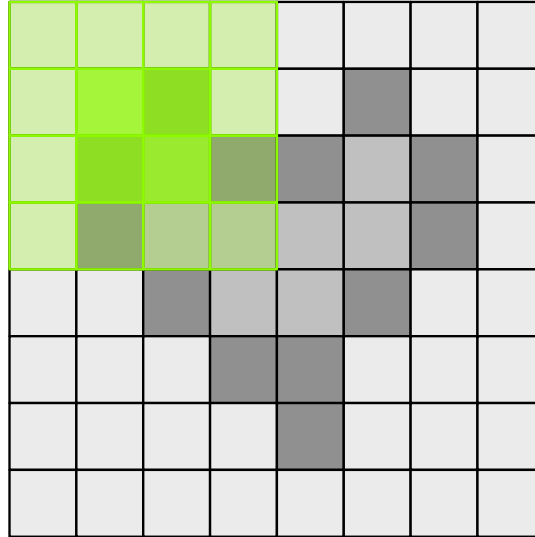
Gaussian filter applied on every pixel of an image



```
for (int i = 0; i < 3; i++){  
    for (int j = 0; j < 3; j++){  
        sum += gauss[i][j] * Image[(i+row-1)*width + (j+col-1)];  
    }  
}
```

Data Reuse: Tiling in GPU Computing

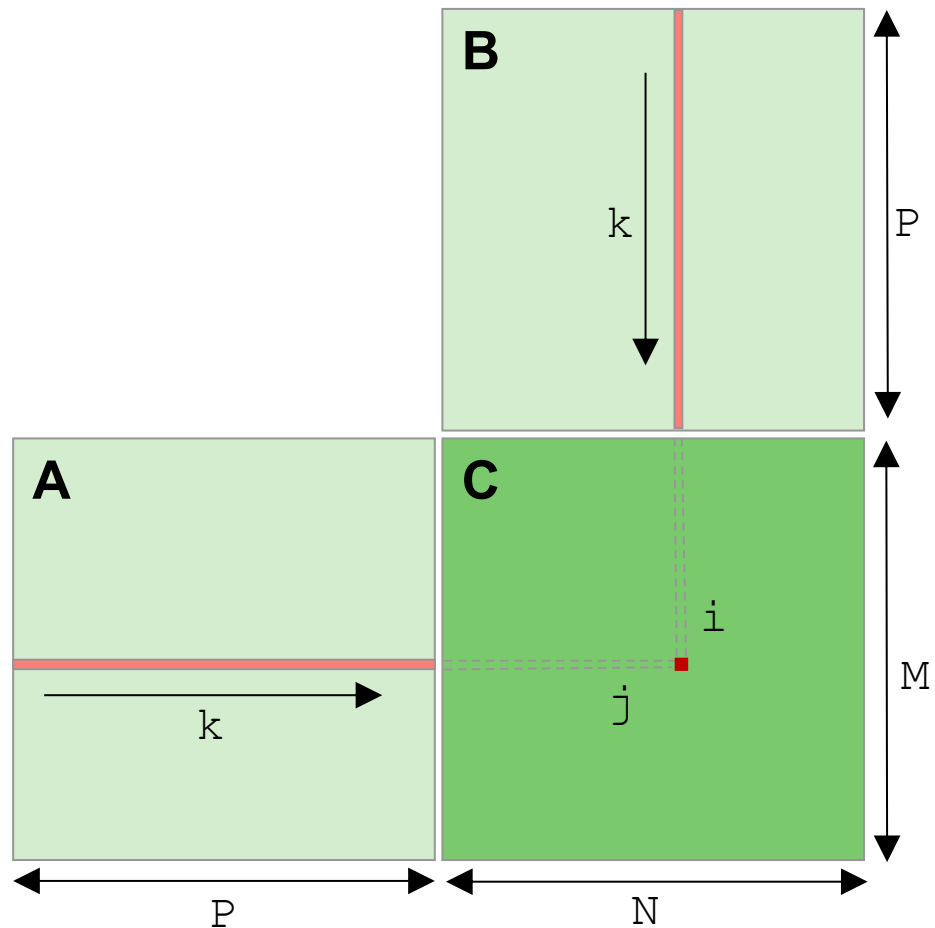
- To take advantage of data reuse, we divide the input into **tiles** that can be loaded into **shared memory (scratchpad memory)**



```
__shared__ int l_data[(L_SIZE+2)*(L_SIZE+2)];  
...  
Load tile into shared memory  
__syncthreads();  
for (int i = 0; i < 3; i++){  
    for (int j = 0; j < 3; j++){  
        sum += gauss[i][j] * l_data[(i+l_row-1)*(L_SIZE+2)+j+l_col-1];  
    }  
}
```


Naïve Matrix Multiplication (I)

- Matrix multiplication: $C = A \times B$
- Consider two input matrices A and B in row-major layout
 - A size is $M \times P$
 - B size is $P \times N$
 - C size is $M \times N$



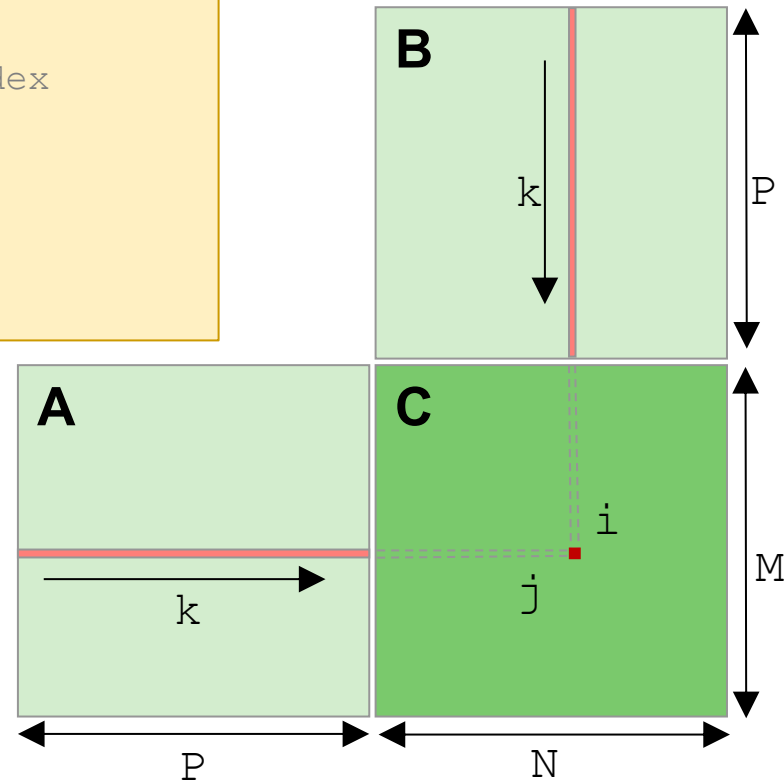
Naïve Matrix Multiplication (II)

- Naïve implementation of matrix multiplication has **poor cache locality**

```
#define A(i,j) matrix_A[i * P + j]
#define B(i,j) matrix_B[i * N + j]
#define C(i,j) matrix_C[i * N + j]

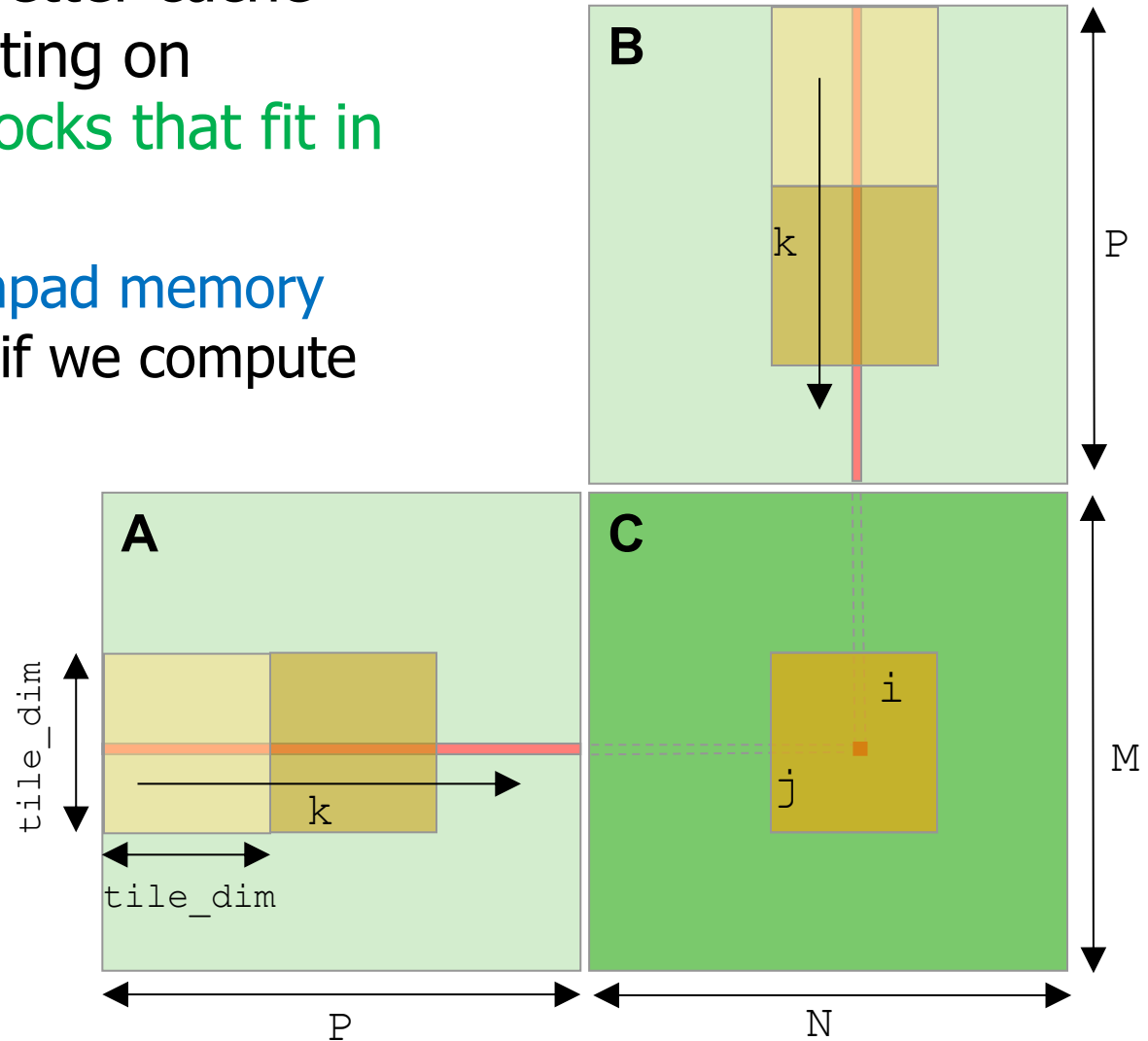
for (i = 0; i < M; i++){ // i = row index
    for (j = 0; j < N; j++){ // j = column index
        C(i, j) = 0; // Set to zero
        for (k = 0; k < P; k++) // Row x Col
            C(i, j) += A(i, k) * B(k, j);
    }
}
```

Consecutive accesses to B are far from each other, in **different cache lines**.
Every access to B is likely to cause a cache miss



Tiled Matrix Multiplication (I)

- We can achieve better cache locality by computing on smaller tiles or blocks that fit in the cache
 - Or in the **scratchpad memory** and **register file** if we compute on a GPU



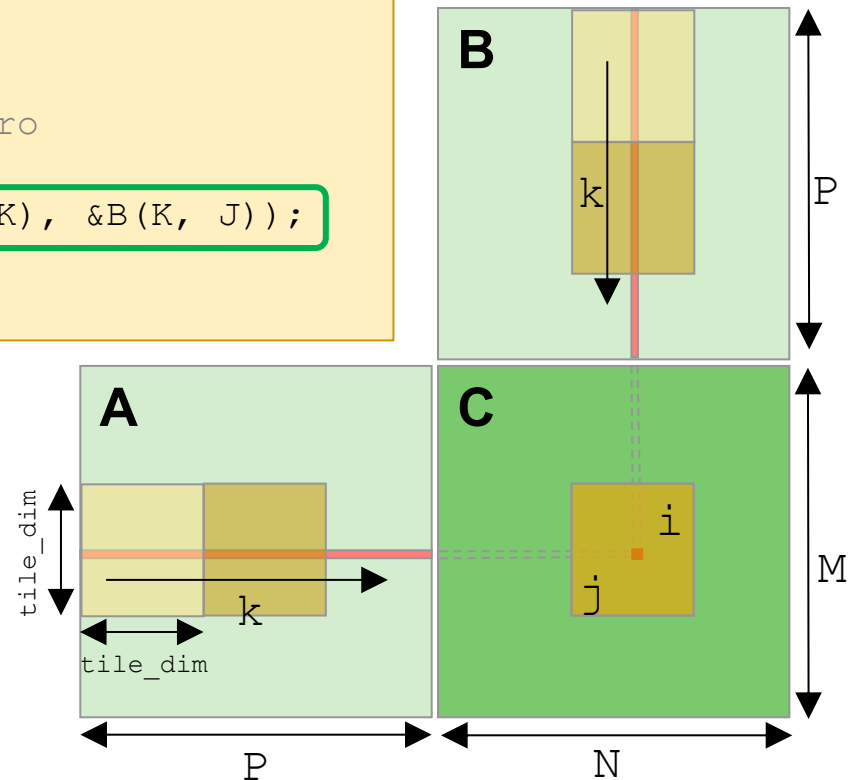
Tiled Matrix Multiplication (II)

- **Tiled implementation** operates on submatrices (tiles or blocks) that fit fast memories (cache, scratchpad, RF)

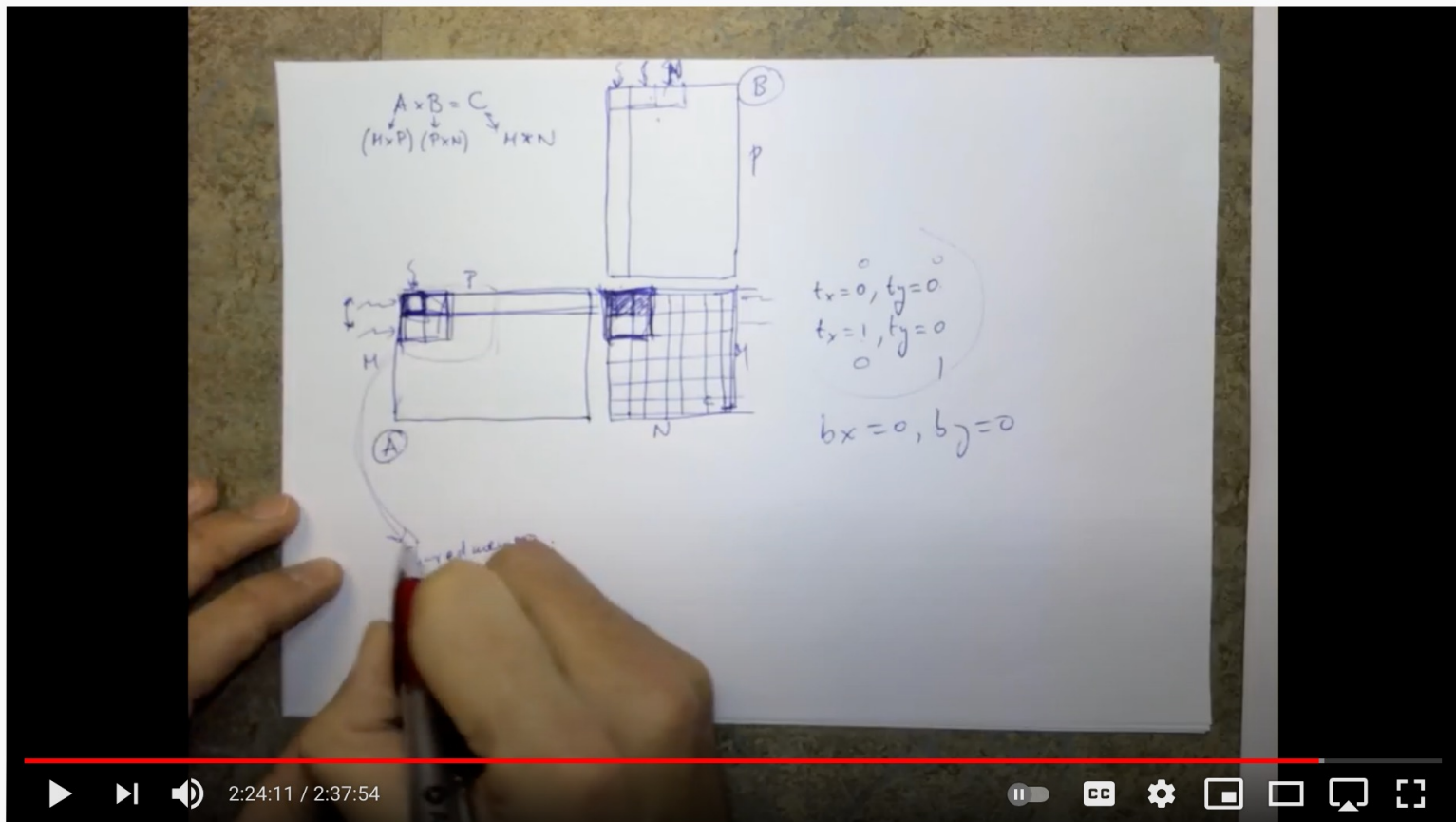
```
#define A(i,j) matrix_A[i * P + j]
#define B(i,j) matrix_B[i * N + j]
#define C(i,j) matrix_C[i * N + j]

for (I = 0; I < M; I += tile_dim){
    for (J = 0; J < N; J += tile_dim){
        Set_to_zero(&C(I, J)); // Set to zero
        for (K = 0; K < P; K += tile_dim)
            Multiply_tiles(&C(I, J), &A(I, K), &B(K, J));
    }
}
```

Multiply small submatrices (tiles or blocks)
of size `tile_dim` x `tile_dim`



Tiled Matrix Multiplication on GPUs



Computer Architecture - Lecture 9: GPUs and GPGPU Programming (ETH Zürich, Fall 2017)

14,426 views • Oct 23, 2017

225 2 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

SUBSCRIBED



Restructuring Data Layout (I)

```
struct Node {  
    struct Node* next;  
    int key;  
    char [256] name;  
    char [256] school;  
}  
  
while (node) {  
    if (node->key == input-key) {  
        // access other fields of node  
    }  
    node = node->next;  
}
```

Frequently accessed

Rarely accessed

Rarely accessed

Frequently accessed

- Assume pointer-based traversal (e.g., of a linked list)
- Assume a huge linked list (1B nodes) and unique keys
- Does the code on the left have poor or good cache hit rate?
- **Poor hit rate: Why?**
 - “Other fields” occupy most of the cache block even though they are rarely accessed!

Restructuring Data Layout (II)

```
struct Node {  
    struct Node* next;  
    int key;  
    struct Node-data* node-data;  
}  
  
struct Node-data {  
    char [256] name;  
    char [256] school;  
}  
  
while (node) {  
    if (node→key == input-key) {  
        // access node→node-data  
    }  
    node = node→next;  
}
```

- Idea: Separate rarely-accessed fields of a data structure and pack them into a separate data structure
- General idea: Separate frequently-accessed (hot) data from rarely-accessed (cold) data so that they are not in the same cache block
- Who should do this?
 - Programmer
 - Compiler
 - Profiling vs. dynamic
 - Hardware?
 - Who can determine what is frequently accessed?

Improving Basic Cache Performance

■ Reducing miss rate

- ❑ More associativity
- ❑ Alternatives/enhancements to associativity
 - Victim caches, hashing, pseudo-associativity, skewed associativity
- ❑ Better replacement/insertion policies
- ❑ Software approaches

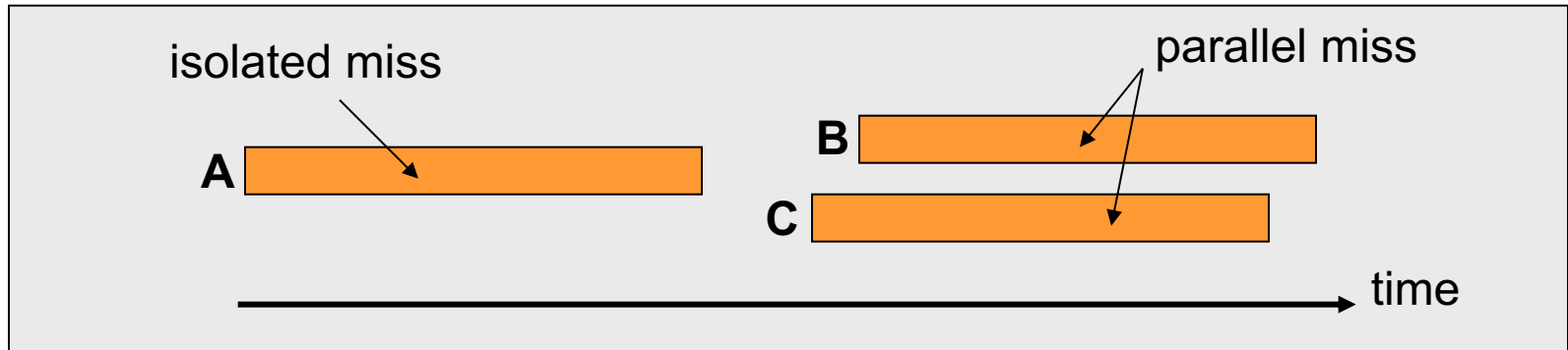
■ Reducing miss latency/cost

- ❑ Multi-level caches
- ❑ Critical word first
- ❑ Subblocking/sectoring
- ❑ Better replacement/insertion policies
- ❑ Non-blocking caches (multiple cache misses in parallel)
- ❑ Multiple accesses per cycle
- ❑ Software approaches

Miss Latency/Cost

- What is miss latency or miss cost affected by?
 - Where does the miss get serviced from?
 - What level of cache in the hierarchy?
 - Row hit versus row conflict in DRAM (bank/rank/channel conflict)
 - Queueing delays in the memory controller and the interconnect
 - Local vs. remote memory (chip, node, rack, remote server, ...)
 - ...
 - How much does the miss stall the processor?
 - Is it overlapped with other latencies?
 - Is the data immediately needed by the processor?
 - Is the incoming block going to evict a longer-to-refetch block?
 - ...

Memory Level Parallelism (MLP)



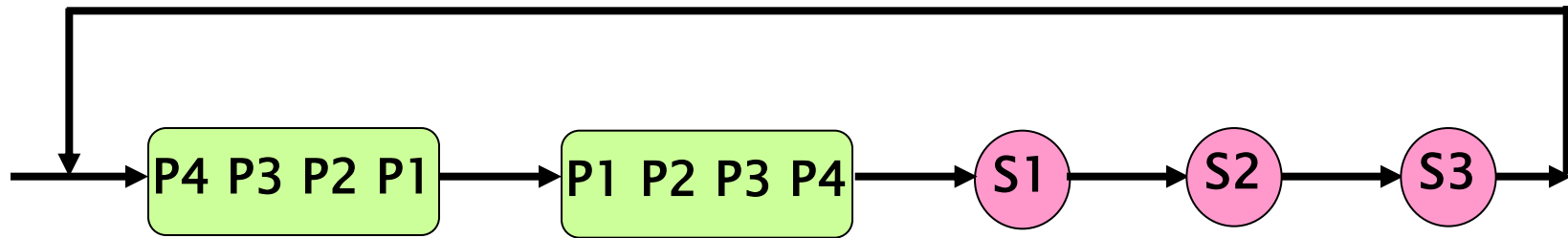
- ❑ Memory Level Parallelism (MLP) is the notion of generating and servicing multiple memory accesses in parallel [Glew'98]
- ❑ Several techniques improve MLP (e.g., out-of-order execution)
- ❑ MLP varies. Some misses are isolated and some parallel

How does this affect cache replacement?

Traditional Cache Replacement Policies

- ❑ Traditional cache management policies try to reduce miss count
- ❑ **Implicit assumption**: Reducing miss count reduces memory-related stall time
- ❑ Misses with varying cost/MLP **breaks** this assumption!
- ❑ Eliminating an isolated miss helps performance more than eliminating a parallel miss
- ❑ Eliminating a higher-latency miss could help performance more than eliminating a lower-latency miss

An Example



Misses to blocks P1, P2, P3, P4 can be serviced in parallel

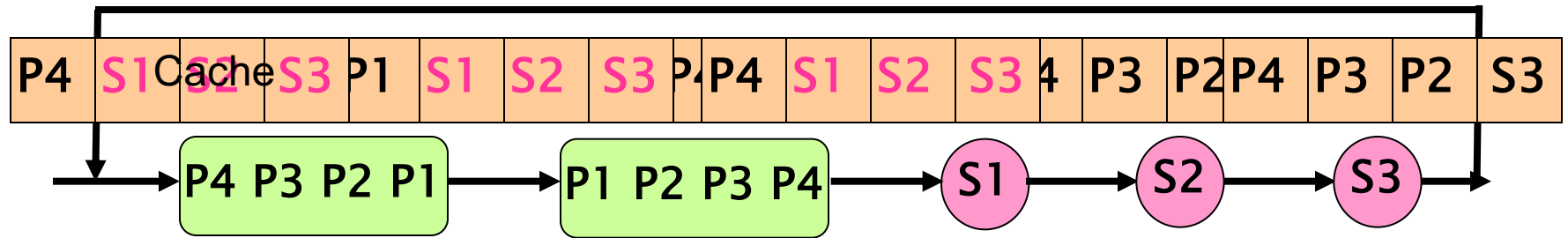
Misses to blocks S1, S2, and S3 are isolated (i.e., serviced serially)

Two replacement algorithms:

1. Minimizes miss count (Belady's OPT)
2. Reduces isolated miss (MLP-Aware)

For a fully associative cache containing 4 blocks

Fewest Misses $\neq$ Best Performance



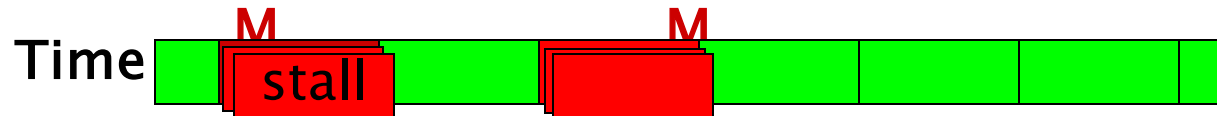
Hit/Miss H H H M H H H H M M M



Misses=4
Stalls=4

Belady's OPT replacement

Hit/Miss H M M H M M H H H



Saved
cycles

Misses=6
Stalls=2

MLP-Aware replacement

Recommended: MLP-Aware Cache Replacement

- How do we incorporate MLP/cost into replacement decisions?
- How do we design a hybrid cache replacement policy?
- Qureshi et al., "A Case for MLP-Aware Cache Replacement," ISCA 2006.

A Case for MLP-Aware Cache Replacement

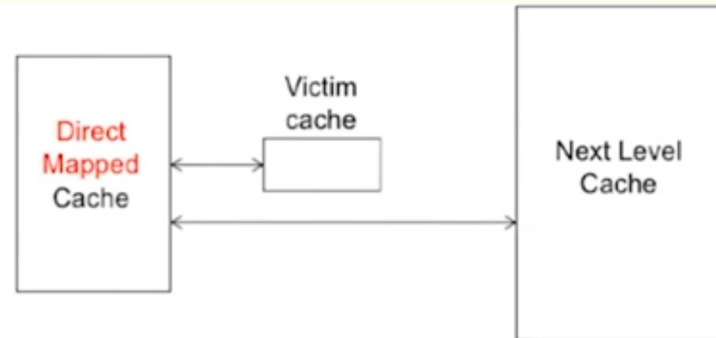
Moinuddin K. Qureshi Daniel N. Lynch Onur Mutlu Yale N. Patt
Department of Electrical and Computer Engineering
The University of Texas at Austin
{moin, lynch, onur, patt}@hps.utexas.edu

Improving Basic Cache Performance

- Reducing miss rate
 - ❑ More associativity
 - ❑ Alternatives/enhancements to associativity
 - Victim caches, hashing, pseudo-associativity, skewed associativity
 - ❑ Better replacement/insertion policies
 - ❑ Software approaches
 - ❑ ...
- Reducing miss latency/cost
 - ❑ Multi-level caches
 - ❑ Critical word first
 - ❑ Subblocking/sectoring
 - ❑ Better replacement/insertion policies
 - ❑ Non-blocking caches (multiple cache misses in parallel)
 - ❑ Multiple accesses per cycle
 - ❑ Software approaches
 - ❑ ...

Lectures on Cache Optimizations (I)

Victim Cache: Reducing Conflict Misses



- Jouppi, "Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers," ISCA 1990.
- Idea: Use a small fully-associative buffer (victim cache) to store recently evicted blocks
 - + Can avoid ping ponging of cache blocks mapped to the same set (if two cache blocks continuously accessed in nearby time conflict with each other)
 - Increases miss latency if accessed serially with L2; adds complexity

Computer Architecture - Lecture 3: Cache Management and Memory Parallelism (ETH Zürich, Fall 2017)

6,392 views • Sep 29, 2017

49 1 SHARE SAVE ...

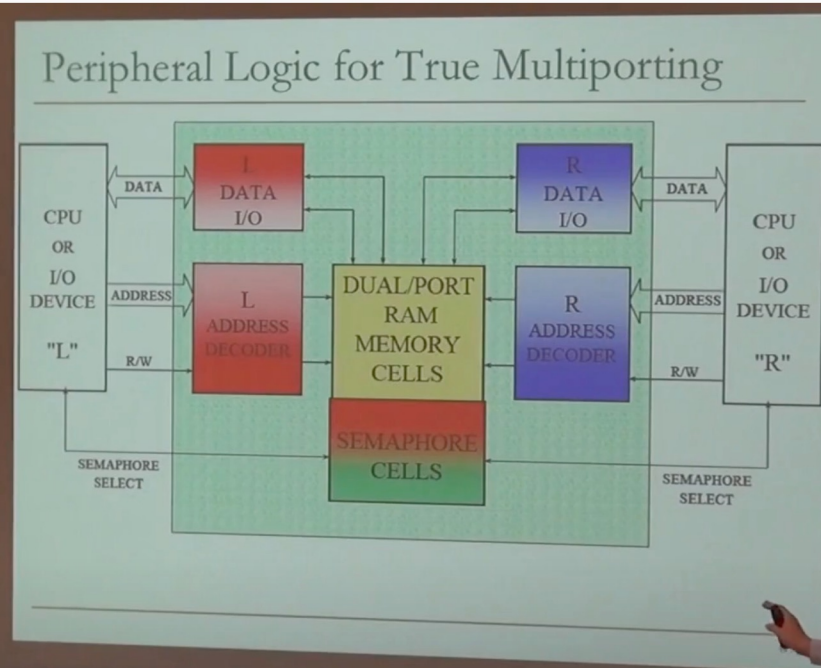


Onur Mutlu Lectures
16.3K subscribers

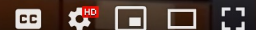
ANALYTICS

EDIT VIDEO

Lectures on Cache Optimizations (II)



1:18:05 / 1:28:10



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 4a: Cache Design (ETH Zürich, Fall 2018)

1,437 views • Sep 29, 2018

15 0 SHARE SAVE ...



Onur Mutlu Lectures
16.3K subscribers

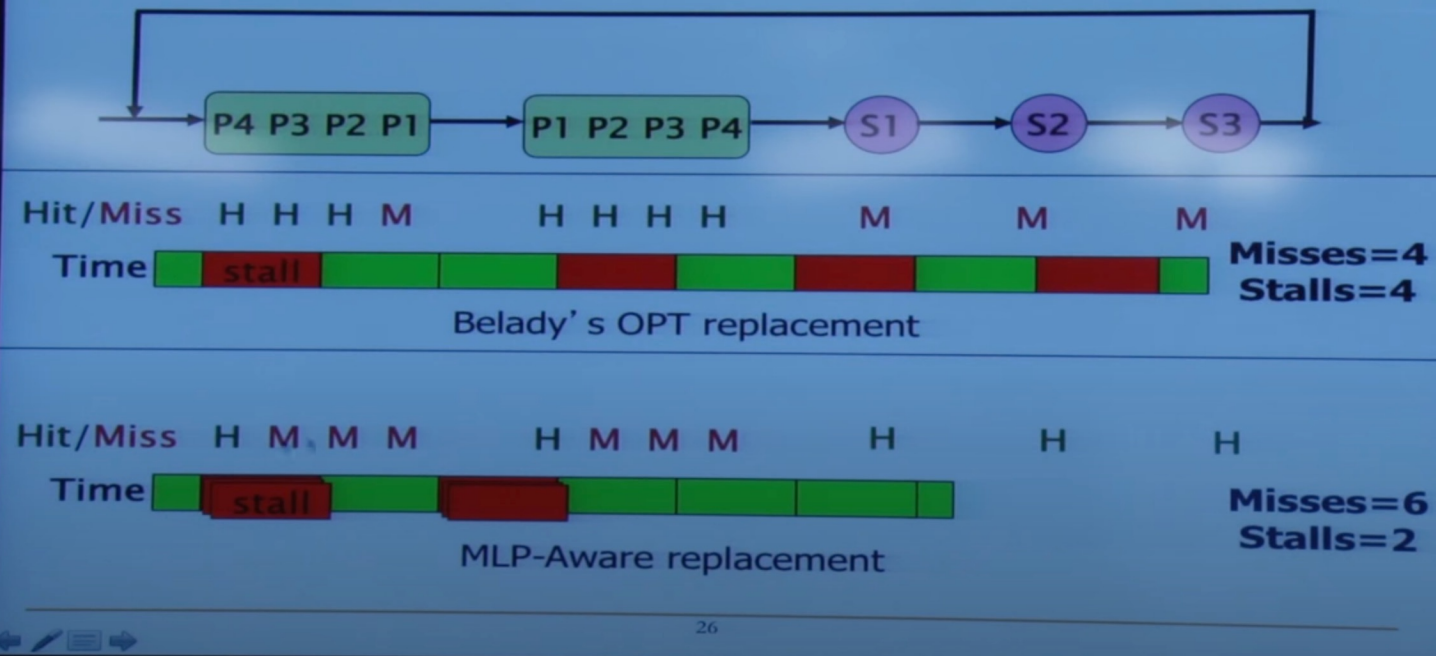
ANALYTICS

EDIT VIDEO

https://www.youtube.com/watch?v=55oYBm9cifl&list=PL5Q2soXY2Zi9JXe3ywQMhyIk_d5dl-TM7&index=6

Lectures on Cache Optimizations (III)

Fewest Misses $\neq$ Best Performance



Lecture 19. High Performance Caches - Carnegie Mellon - Comp. Arch. 2015 - Onur Mutlu

9,737 views • Mar 5, 2015

63 1 SHARE SAVE ...



Carnegie Mellon Computer Architecture
23.2K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Cache Optimizations

■ Computer Architecture, Fall 2017, Lecture 3

- ❑ Cache Management & Memory Parallelism (ETH, Fall 2017)
- ❑ https://www.youtube.com/watch?v=OyomXCHNJDA&list=PL5Q2soXY2Zi9OhoVQBX_YFIZywZXCPI4M_&index=3

■ Computer Architecture, Fall 2018, Lecture 4a

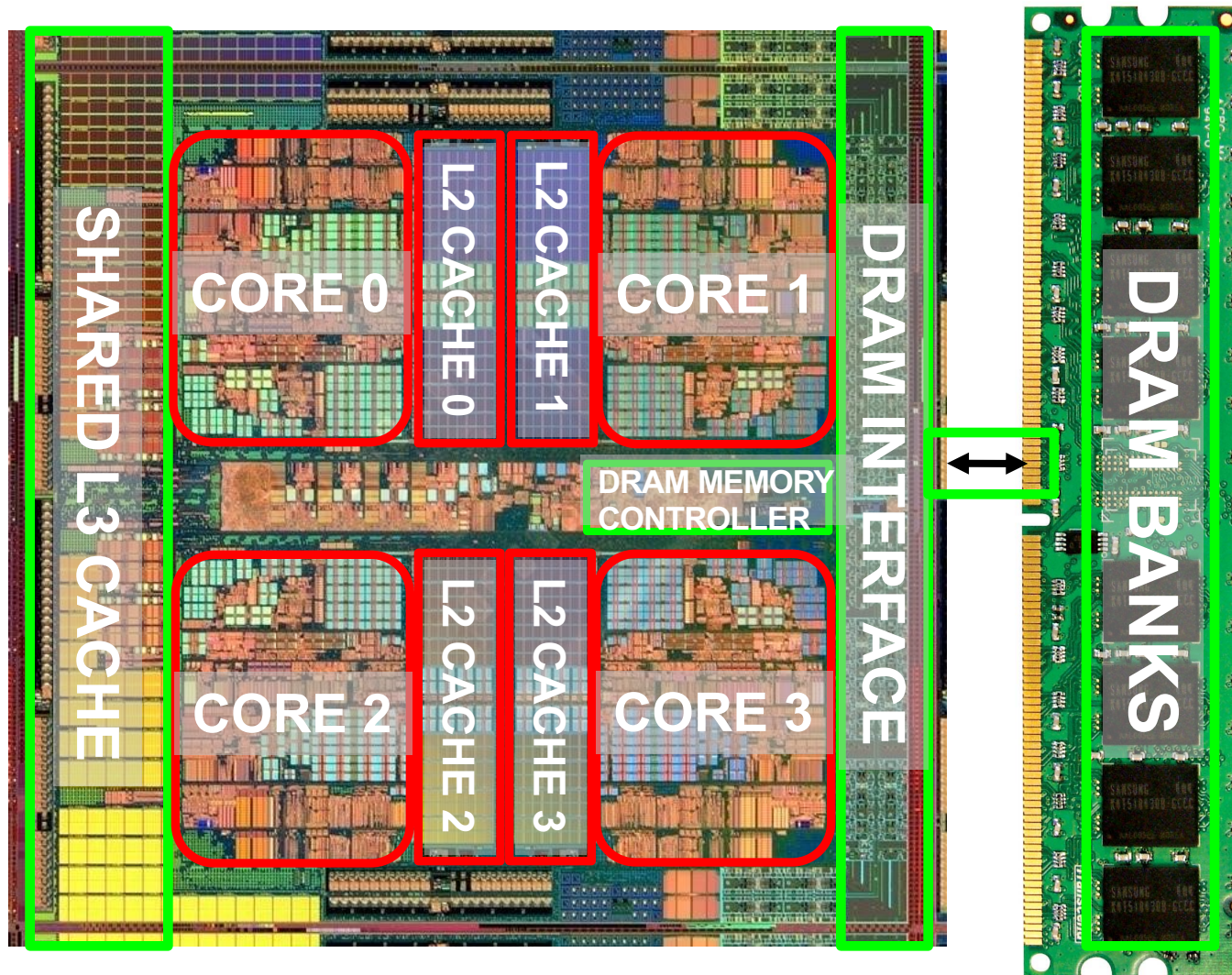
- ❑ Cache Design (ETH, Fall 2018)
- ❑ https://www.youtube.com/watch?v=55oYBm9cifI&list=PL5Q2soXY2Zi9JXe3ywQMh_ylk_d5dI-TM7&index=6

■ Computer Architecture, Spring 2015, Lecture 19

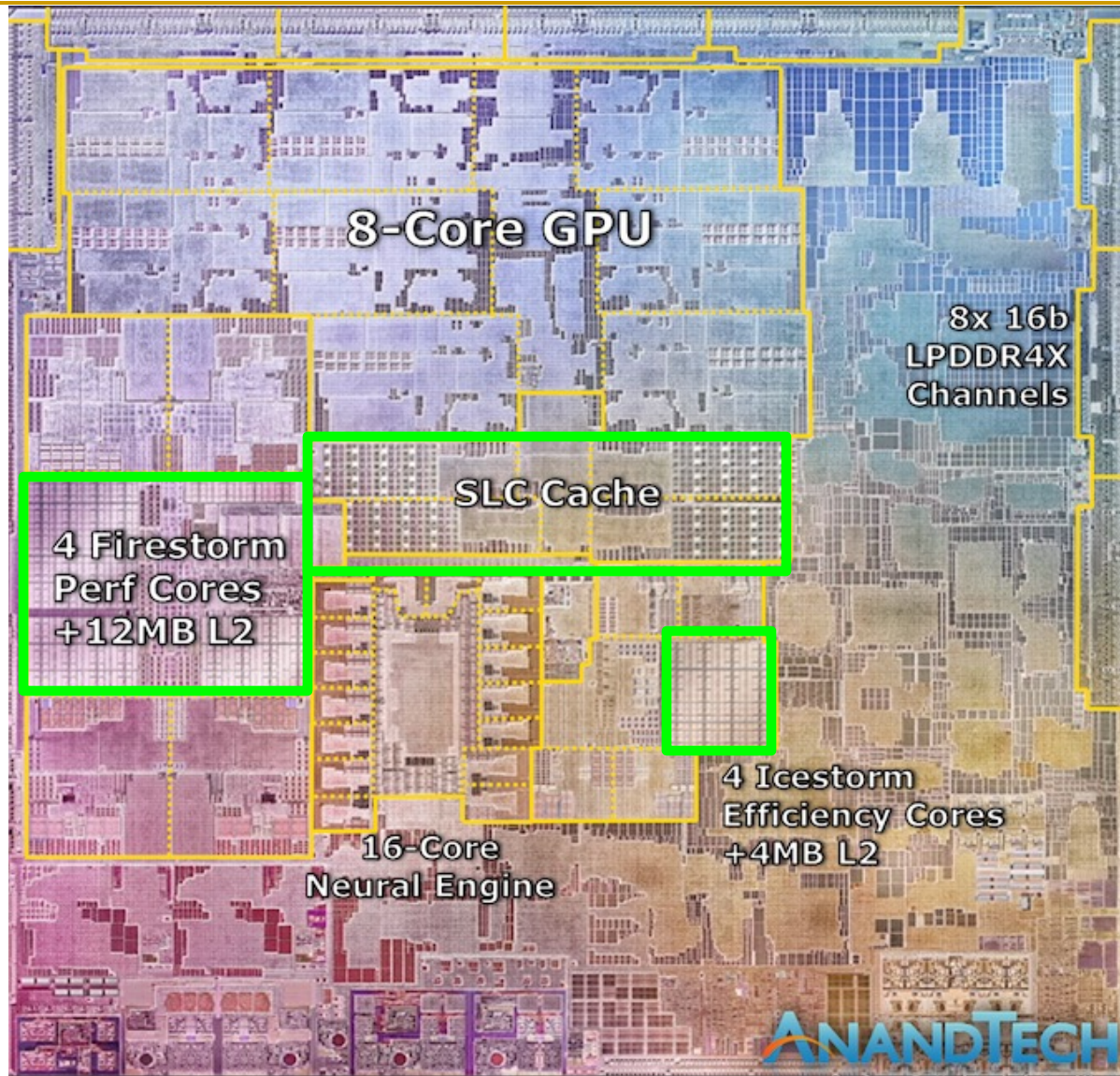
- ❑ High Performance Caches (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=jDHx2K9HxIM&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=21>

Multi-Core Issues in Caching

Caches in a Multi-Core System



Caches in a Multi-Core System



Apple M1,
2021

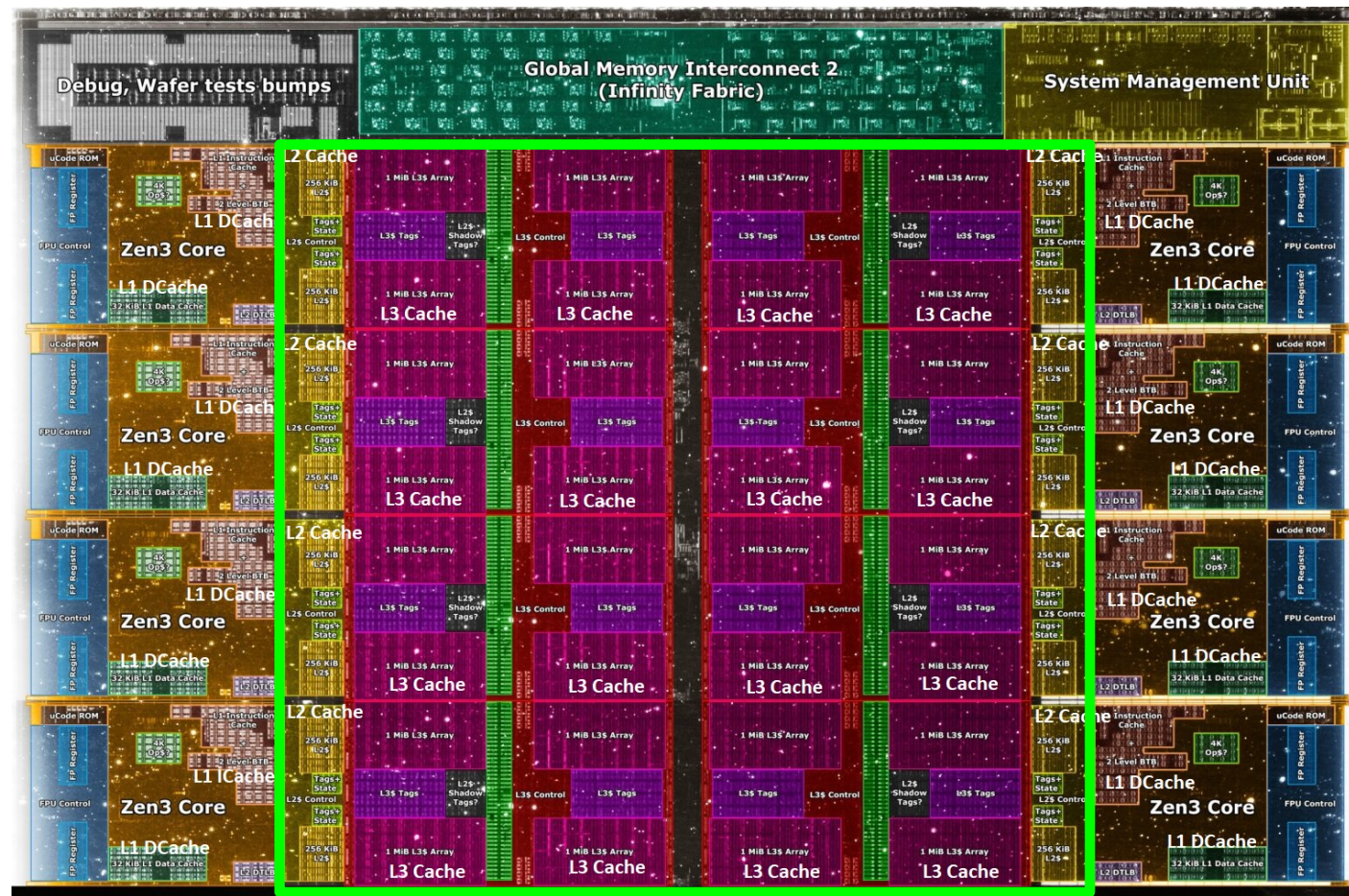
Caches in a Multi-Core System



10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>
Die shot interpretation by Locuza, October 2021

Intel Alder Lake,
2021

Caches in a Multi-Core System



Core Count:
8 cores/16 threads

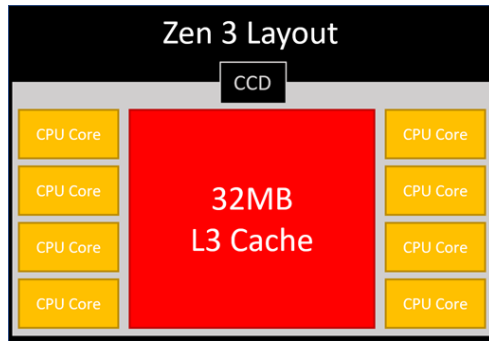
L1 Caches:
32 KB per core

L2 Caches:
512 KB per core

L3 Cache:
32 MB shared

AMD Ryzen 5000, 2020

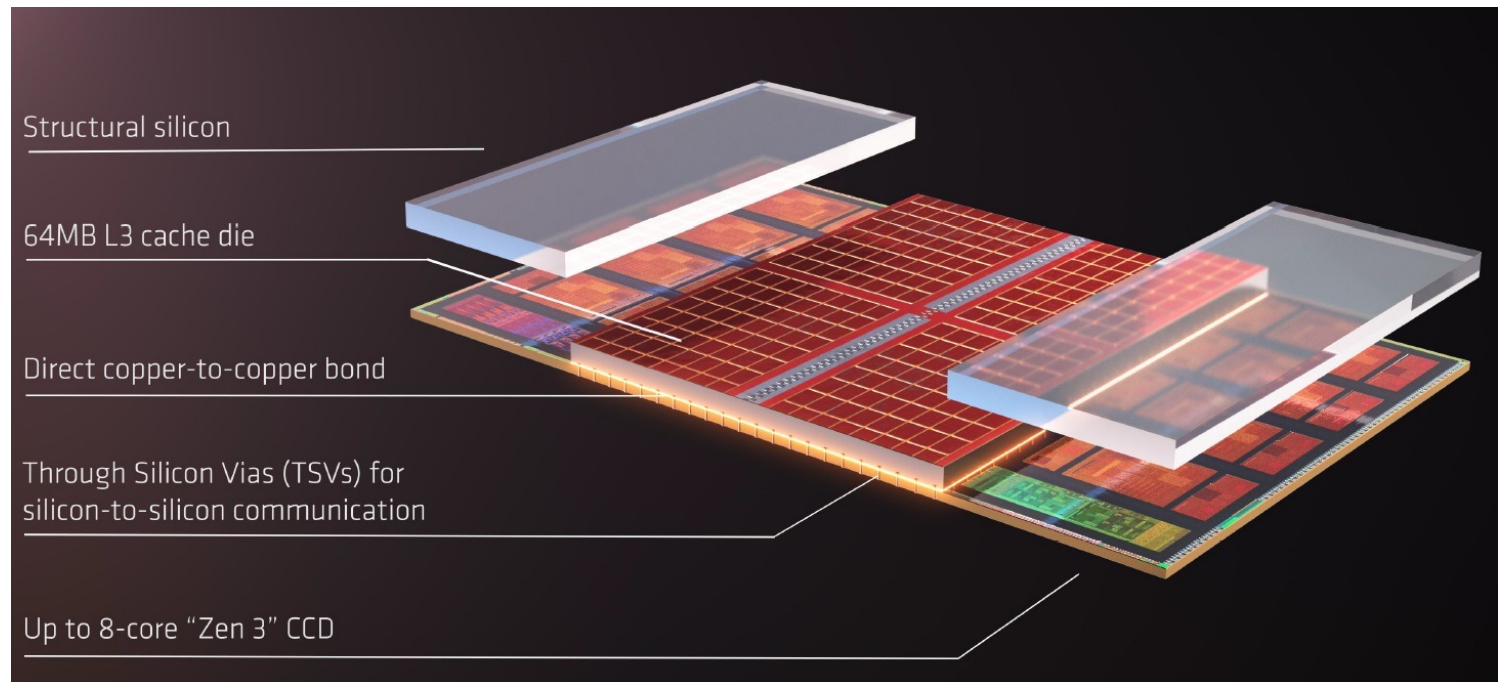
Caches in a Multi-Core System



<https://community.microcenter.com/discussion/5134/comparing-zen-3-to-zen-2>

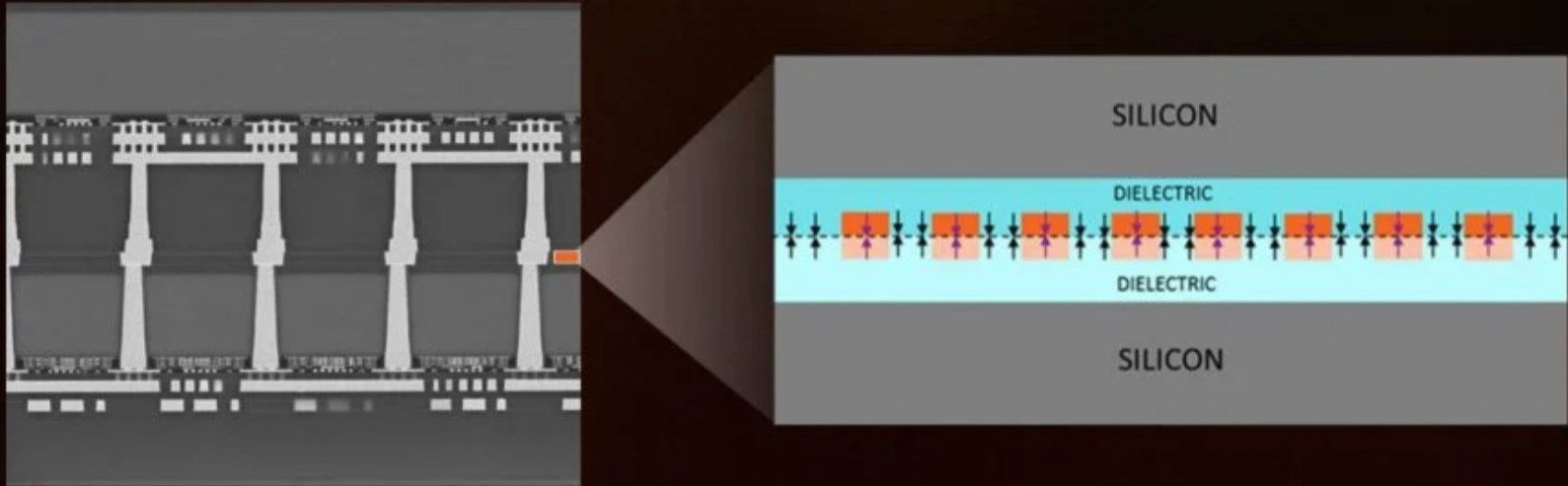
AMD increases the L3 size of their 8-core Zen 3 processors from 32 MB to 96 MB

- Additional 64 MB L3 cache die**
stacked on top of the processor die
- Connected using Through Silicon Vias (TSVs)
 - Total of 96 MB L3 cache



3D Stacking Technology: Example

AMD 3D V-CACHE™ HYBRID BONDING

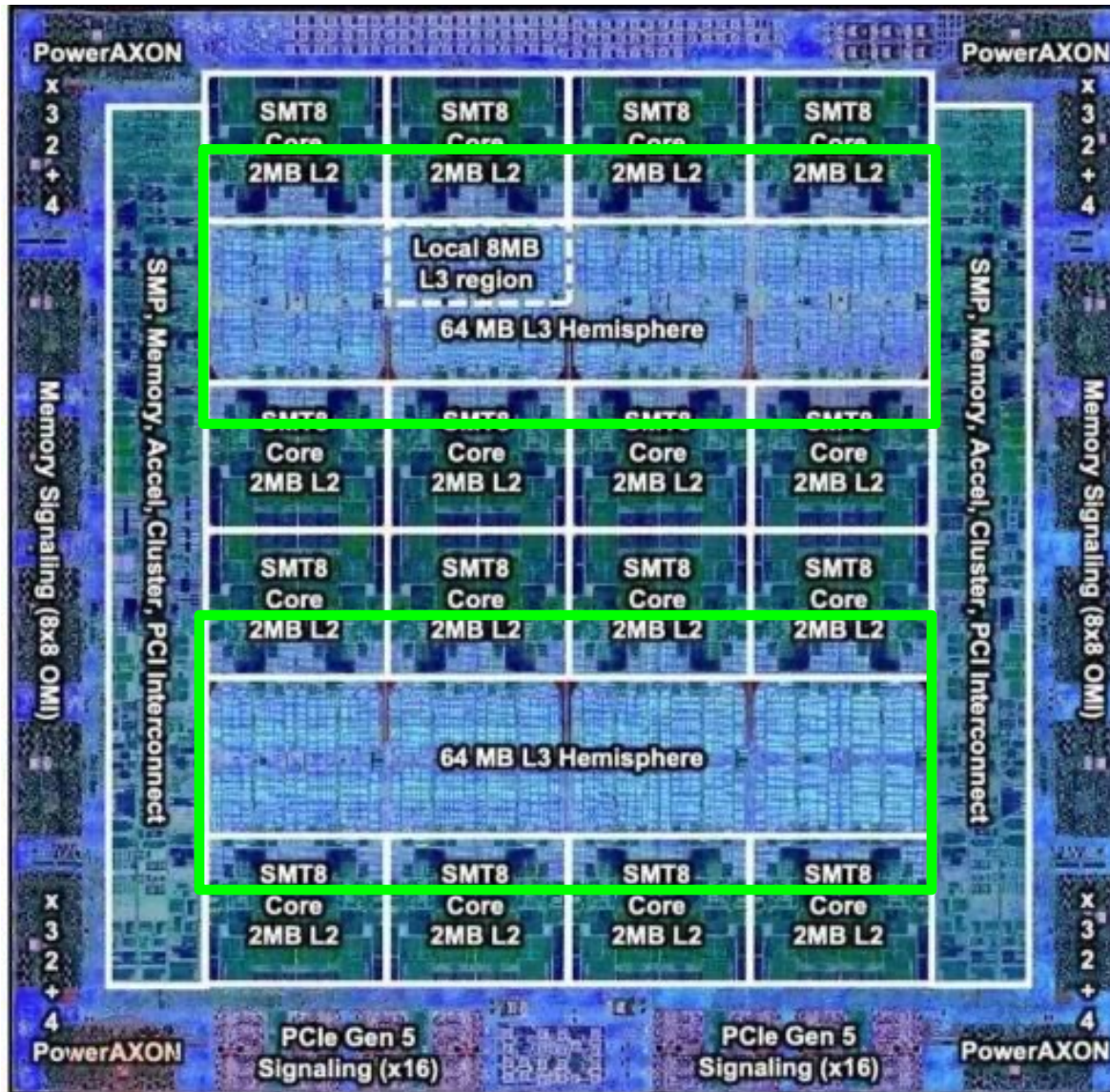


Direct Copper –Copper Bonding

AMD Ryzen 7 5800X3D: The 3D V-Cache in detail (4)

Source: AMD

Caches in a Multi-Core System



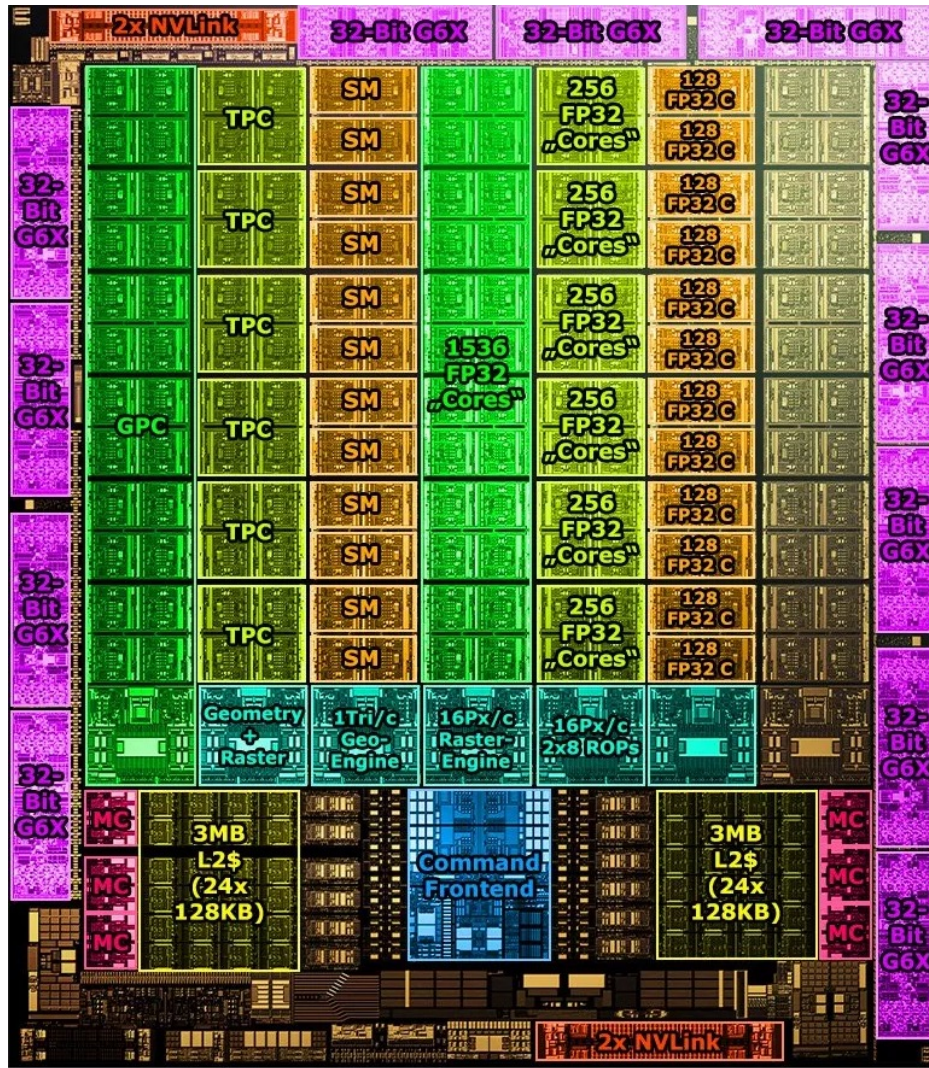
IBM POWER10,
2020

Cores:
15-16 cores,
8 threads/core

L2 Caches:
2 MB per core

L3 Cache:
120 MB shared

Caches in a Multi-Core System



Nvidia Ampere, 2020

Cores:

128 Streaming Multiprocessors

L1 Cache or Scratchpad:

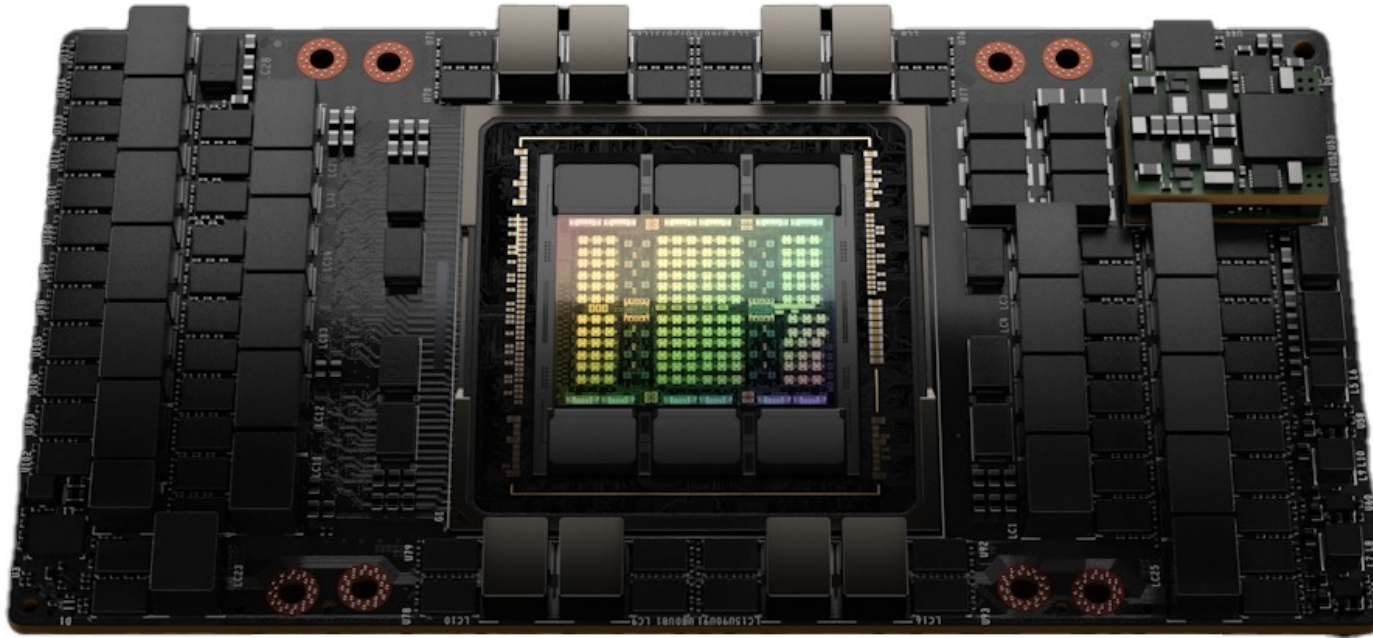
192KB per SM

Can be used as L1 Cache and/or Scratchpad

L2 Cache:

40 MB shared

Caches in a Multi-Core System



Nvidia Hopper,
2022

Cores:
144 Streaming
Multiprocessors

**L1 Cache or
Scratchpad:**
256KB per SM
Can be used as L1 Cache
and/or Scratchpad

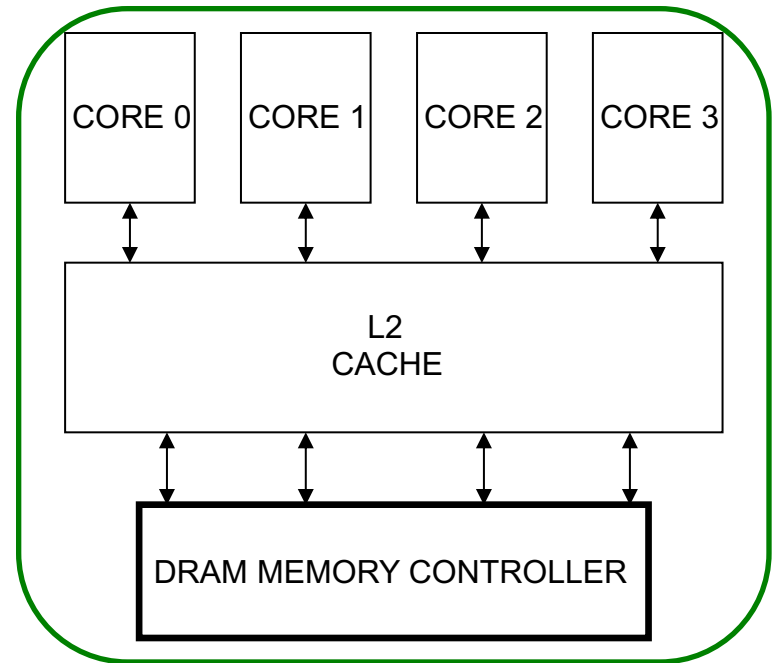
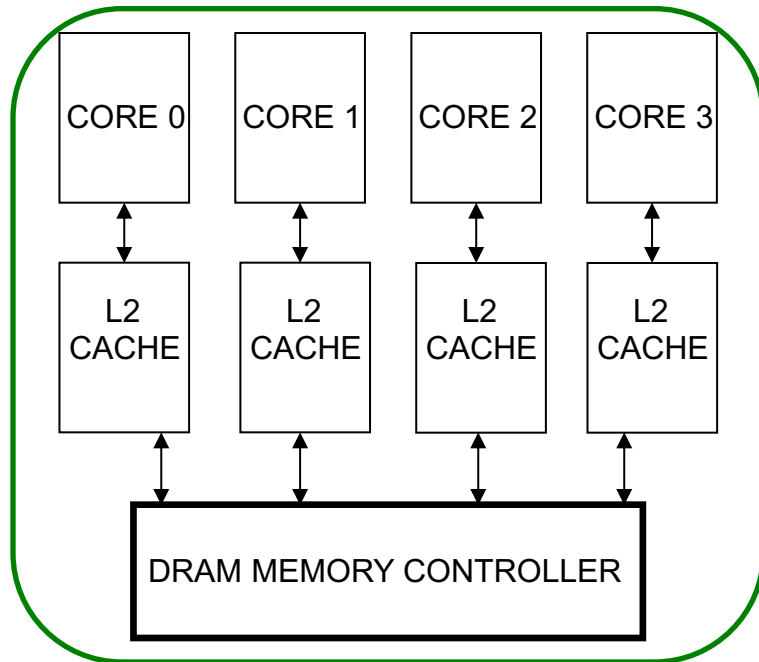
L2 Cache:
60 MB shared

Caches in Multi-Core Systems

- Cache efficiency becomes even more important in a multi-core/multi-threaded system
 - Memory bandwidth is at premium
 - Cache space is a limited resource across cores/threads
- How do we design the caches in a multi-core system?
- Many decisions and questions
 - Shared vs. private caches
 - How to maximize performance of the entire system?
 - How to provide QoS & predictable perf. to different threads in a shared cache?
 - Should cache management algorithms be aware of threads?
 - How should space be allocated to threads in a shared cache?
 - Should we store data in compressed format in some caches?
 - How do we do better reuse prediction & management in caches?

Private vs. Shared Caches

- **Private** cache: Cache belongs to one core (a shared block can be in multiple caches)
- **Shared** cache: Cache is shared by multiple cores



Resource Sharing Concept and Advantages

- Idea: Instead of dedicating a hardware resource to a hardware context, allow multiple contexts to use it
 - Example resources: functional units, pipeline, caches, buses, memory, interconnects, storage
 - Why?
-
- + Resource sharing improves utilization/efficiency → throughput
 - When a resource is left idle by one thread, another thread can use it; no need to replicate shared data
 - + Reduces communication latency
 - For example, data shared between multiple threads can be kept in the same cache in multithreaded processors
 - + Compatible with the shared memory programming model

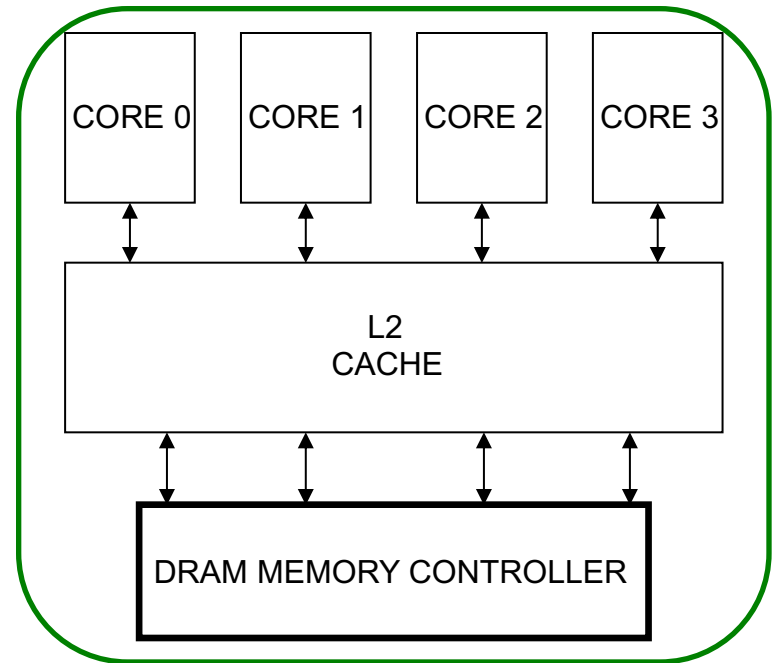
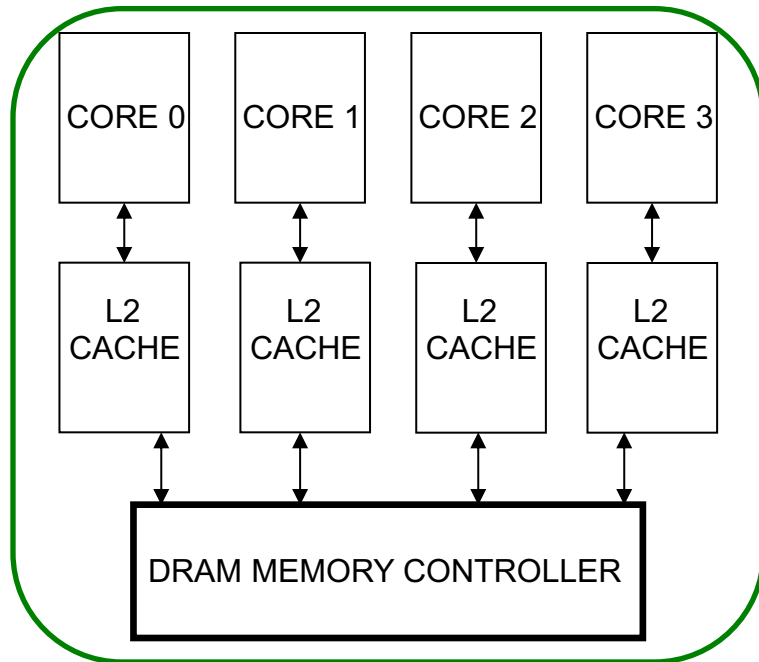
Resource Sharing Disadvantages

- Resource sharing results in **contention for resources**
 - When the resource is not idle, another thread cannot use it
 - If space is occupied by one thread, another thread needs to re-occupy it
- **Sometimes reduces each or some thread's performance**
 - Thread performance can be worse than when it is run alone
- **Eliminates performance isolation** → inconsistent performance across runs
 - Thread performance depends on co-executing threads
- Uncontrolled (free-for-all) sharing **degrades quality of service**
 - Causes unfairness, starvation

Need to efficiently and fairly utilize shared resources

Private vs. Shared Caches

- **Private** cache: Cache belongs to one core (a shared block can be in multiple caches)
- **Shared** cache: Cache is shared by multiple cores



Shared Caches Between Cores

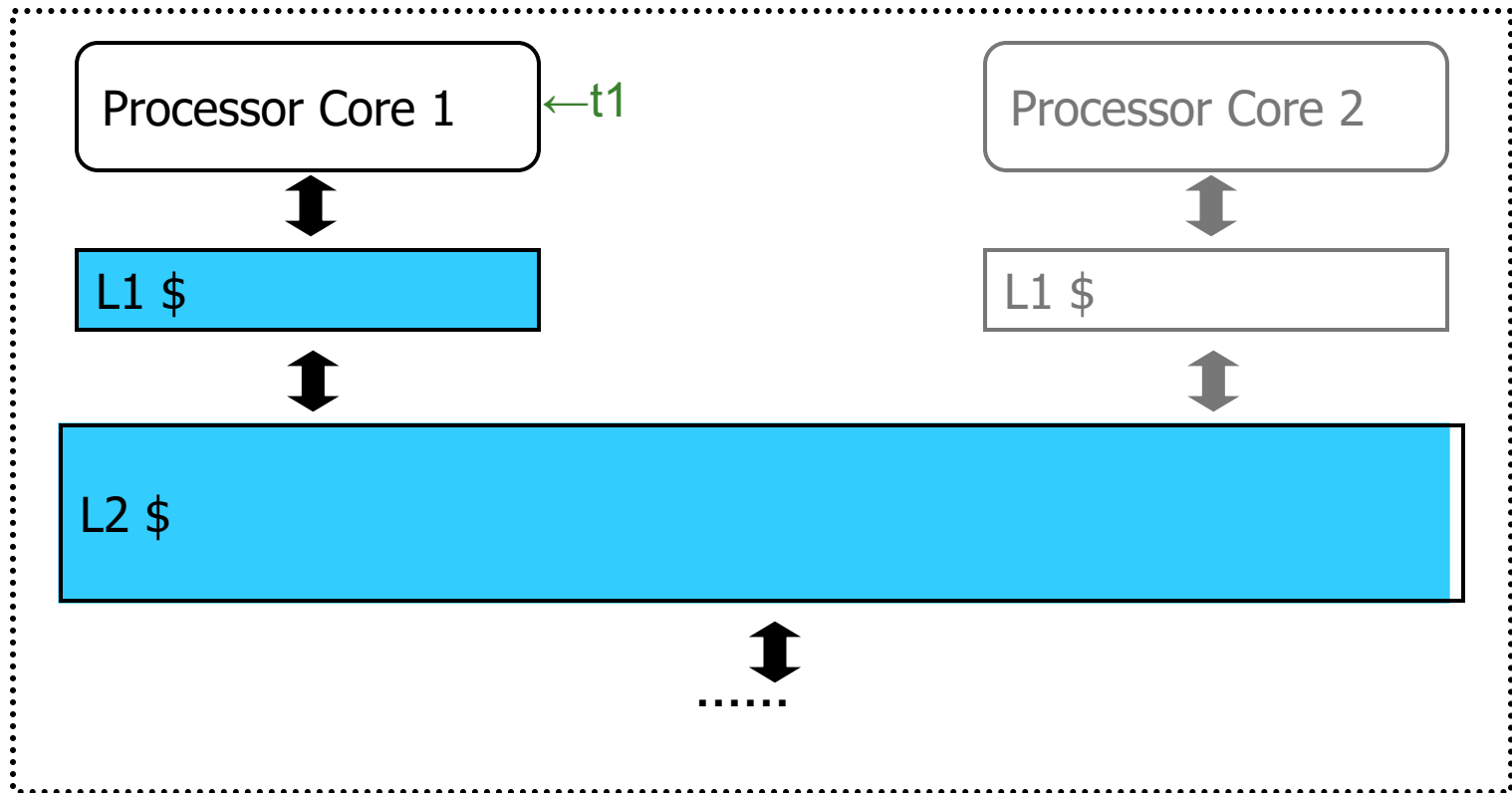
■ Advantages:

- High effective capacity
- Dynamic partitioning of available cache space
 - No fragmentation due to static partitioning
 - If one core does not utilize some space, another core can
- Easier to maintain coherence (a cache block is in a single location)

■ Disadvantages

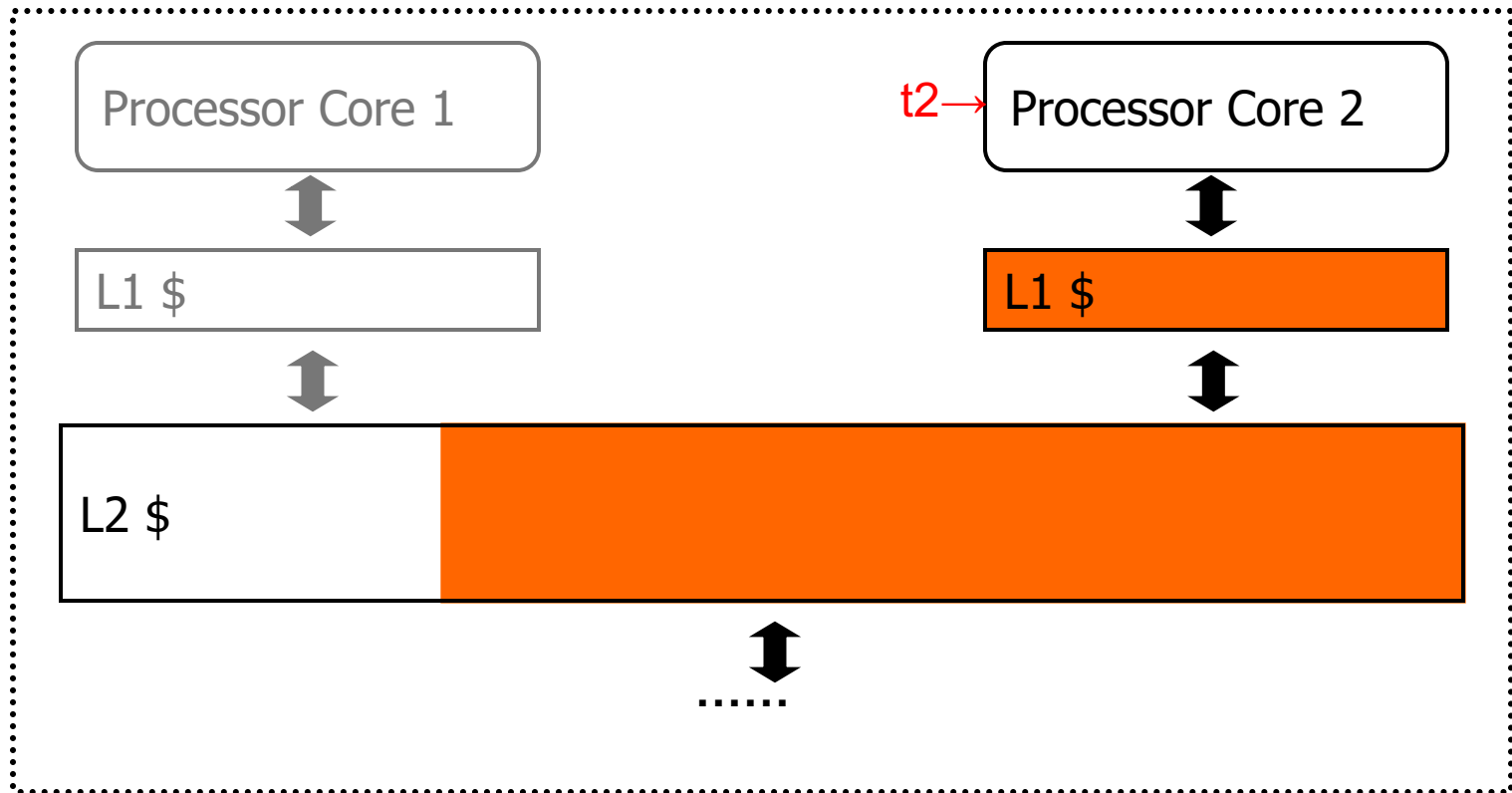
- Slower access (cache is not tightly coupled with the core)
- Cores incur conflict misses due to other cores' accesses
 - Misses due to inter-core interference
 - Some cores can destroy the hit rate of other cores
- Guaranteeing a minimum level of service (or fairness) to each core is harder (how much space, how much bandwidth?)

Example: One Problem with Shared Caches



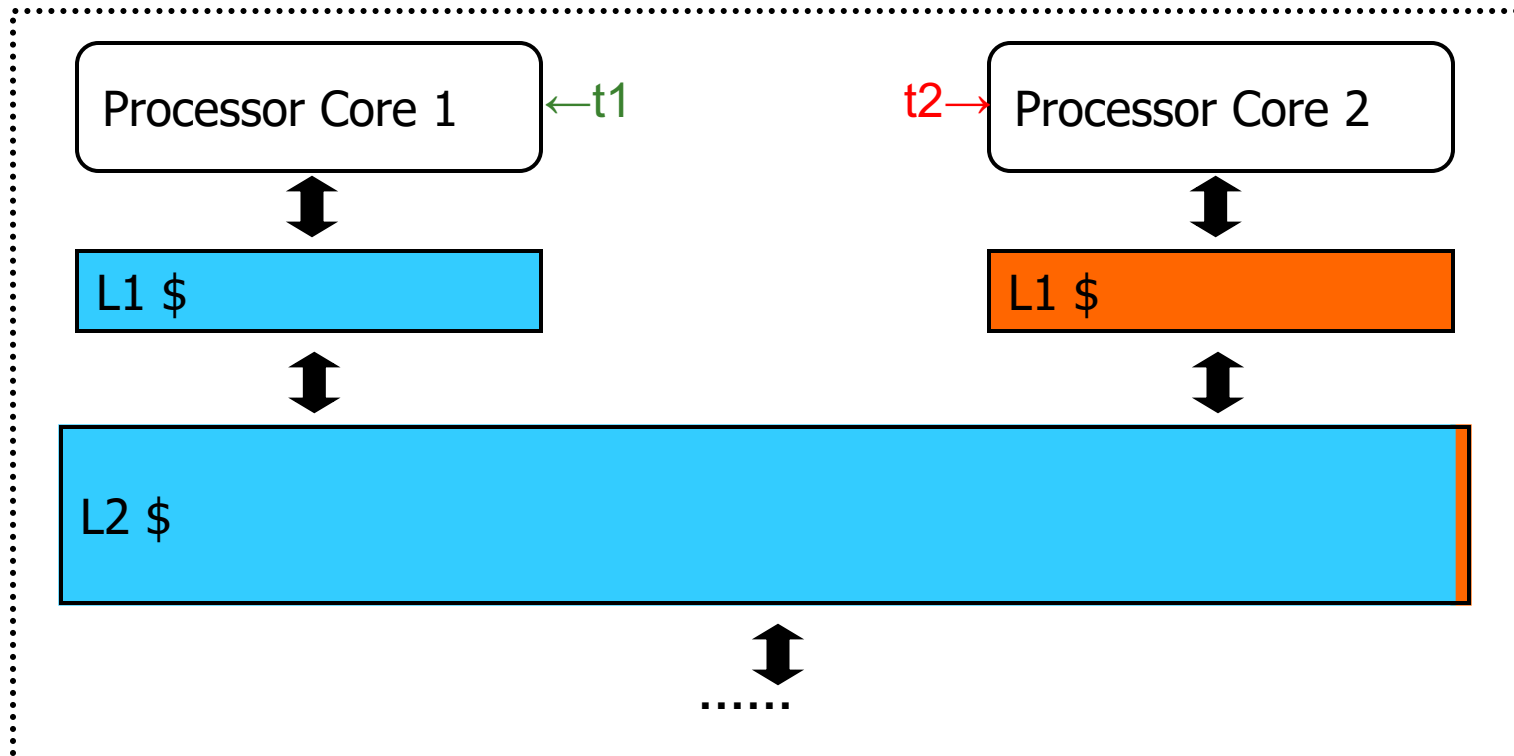
Kim et al., “Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture,” PACT 2004.

Example: One Problem with Shared Caches



Kim et al., “Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture,” PACT 2004.

Example: One Problem with Shared Caches



t2's performance is significantly reduced due to **unfair cache sharing**

Kim et al., "Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture," PACT 2004.

Resource Sharing vs. Partitioning

- Sharing improves resource utilization
 - Better utilization of space
- Partitioning provides performance isolation (predictable performance)
 - Dedicated space
- Can we get the benefits of both?
- Idea: Design shared resources such that they are efficiently utilized, controllable and partitionable
 - No wasted resource + QoS mechanisms for threads

Lectures on Multi-Core Cache Management

Computer Architecture

Lecture 15:

Multi-Core Cache Management

Prof. Onur Mutlu
ETH Zürich
Fall 2017
15 November 2017



0:35 / 2:33:03



Computer Architecture - Lecture 15: Multi-Core Cache Management (ETH Zürich, Fall 2017)

934 views • Nov 17, 2017

13 0 SHARE SAVE ...

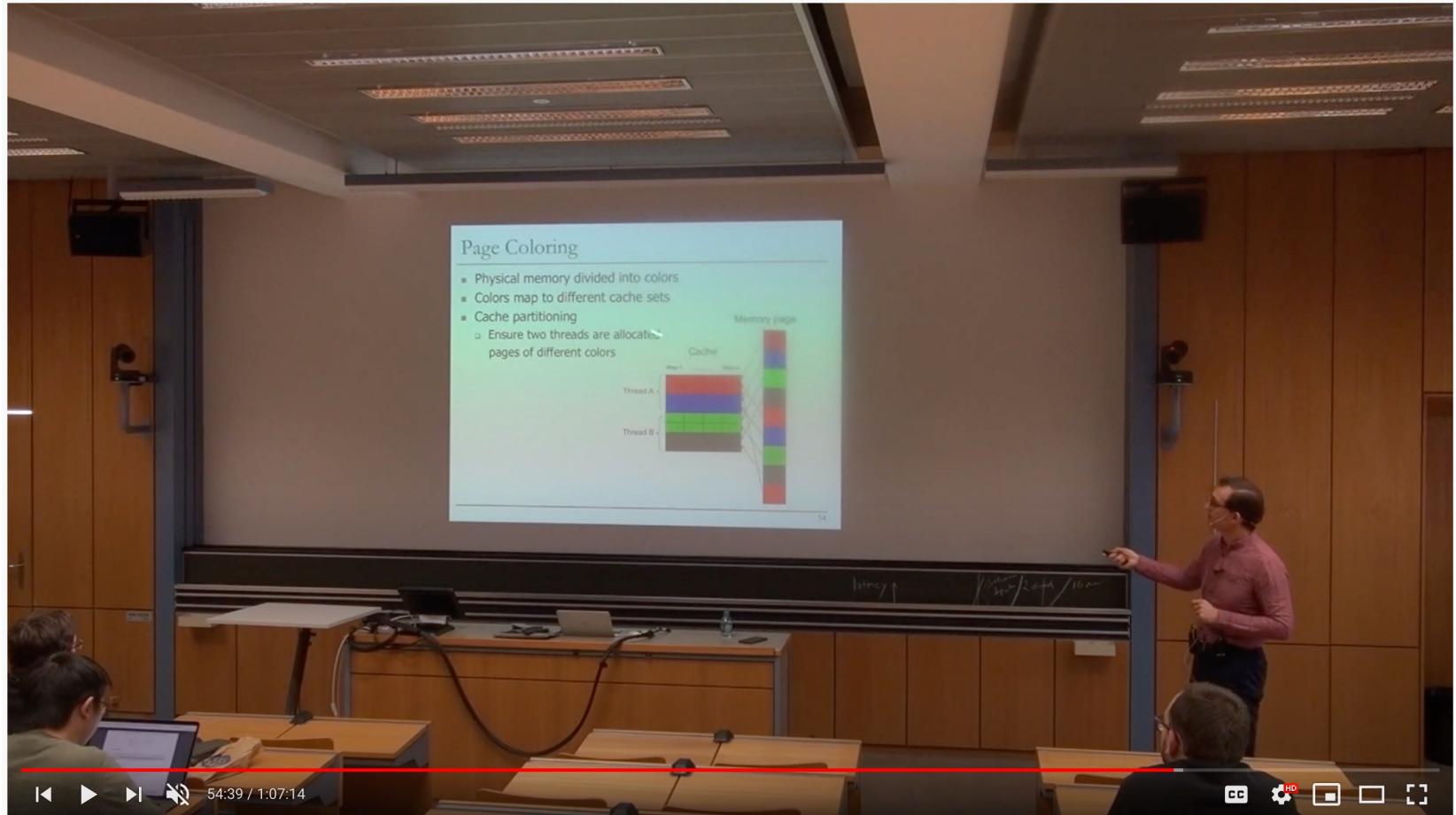


Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Multi-Core Cache Management



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 18b: Multi-Core Cache Management (ETH Zürich, Fall 2018)

744 views • Nov 23, 2018

12 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Multi-Core Cache Management

Approaches to Reuse Prediction

Use program counter or memory region information.

1. Group Blocks
2. Learn group behavior
3. Predict reuse

1. Same group $\nearrow$ same reuse behavior
2. No control over number of high-reuse blocks

77

ETH ZÜRICH HAUPTGEBÄUDE
Computer Architecture - Lecture 19a: Multi-Core Cache Management II (ETH Zürich, Fall 2018)

293 views • Dec 2, 2018

6 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Multi-Core Cache Management

- **Computer Architecture, Fall 2018, Lecture 18b**
 - Multi-Core Cache Management (ETH, Fall 2018)
 - https://www.youtube.com/watch?v=c9FhGRB3HoA&list=PL5Q2soXY2Zi9JXe3ywQMhylk_d5dI-TM7&index=29

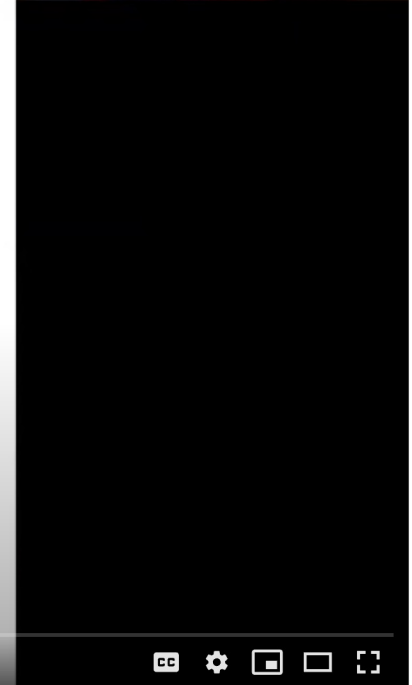
- **Computer Architecture, Fall 2018, Lecture 19a**
 - Multi-Core Cache Management II (ETH, Fall 2018)
 - https://www.youtube.com/watch?v=Siz86__PD4w&list=PL5Q2soXY2Zi9JXe3ywQMhylk_d5dI-TM7&index=30

- **Computer Architecture, Fall 2017, Lecture 15**
 - Multi-Core Cache Management (ETH, Fall 2017)
 - https://www.youtube.com/watch?v=7_Tqlw8gxOU&list=PL5Q2soXY2Zi9OhoVQBXyFIZywZXCPI4M_&index=17

Lectures on Memory Resource Management

QoS-Aware Memory Systems: Challenges

- How do we **reduce inter-thread interference**?
 - ❑ Improve system performance and core utilization
 - ❑ Reduce request serialization and core starvation
- How do we **control inter-thread interference**?
 - ❑ Provide mechanisms to enable system software to enforce QoS policies
 - ❑ While providing high system performance
- How do we **make the memory system configurable/flexible**?
 - ❑ Enable flexible mechanisms that can achieve many goals
 - Provide fairness or throughput when needed
 - Satisfy performance guarantees when needed



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 11b: Memory Interference and QoS (ETH Zürich, Fall 2020)

735 views • Oct 31, 2020

14 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

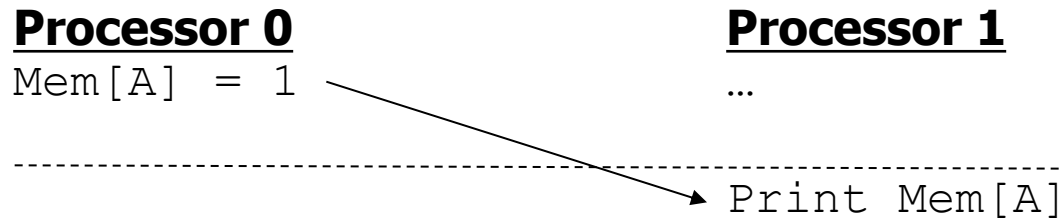
Lectures on Memory Resource Management

- **Computer Architecture, Fall 2020, Lecture 11a**
 - Memory Controllers (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=TeG773OgiMQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=20>
- **Computer Architecture, Fall 2020, Lecture 11b**
 - Memory Interference and QoS (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=0nnI807nCkc&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=21>
- **Computer Architecture, Fall 2020, Lecture 13**
 - Memory Interference and QoS II (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=Axye9VqQT7w&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=26>
- **Computer Architecture, Fall 2020, Lecture 2a**
 - Memory Performance Attacks (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=VJzZbwgBfy8&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=2>

Cache Coherence

Shared Memory Model

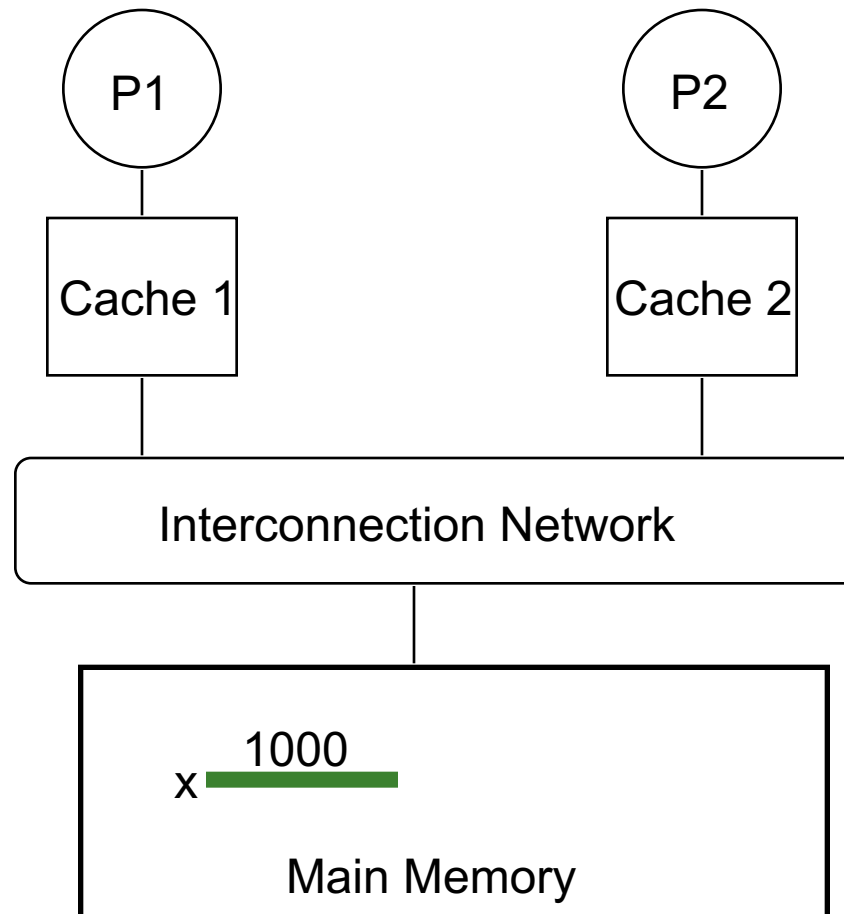
- Threads in parallel programs communicate through *shared memory*
- Thread 0 writes to an address, followed by Thread 1 reading
 - This implies communication between the two



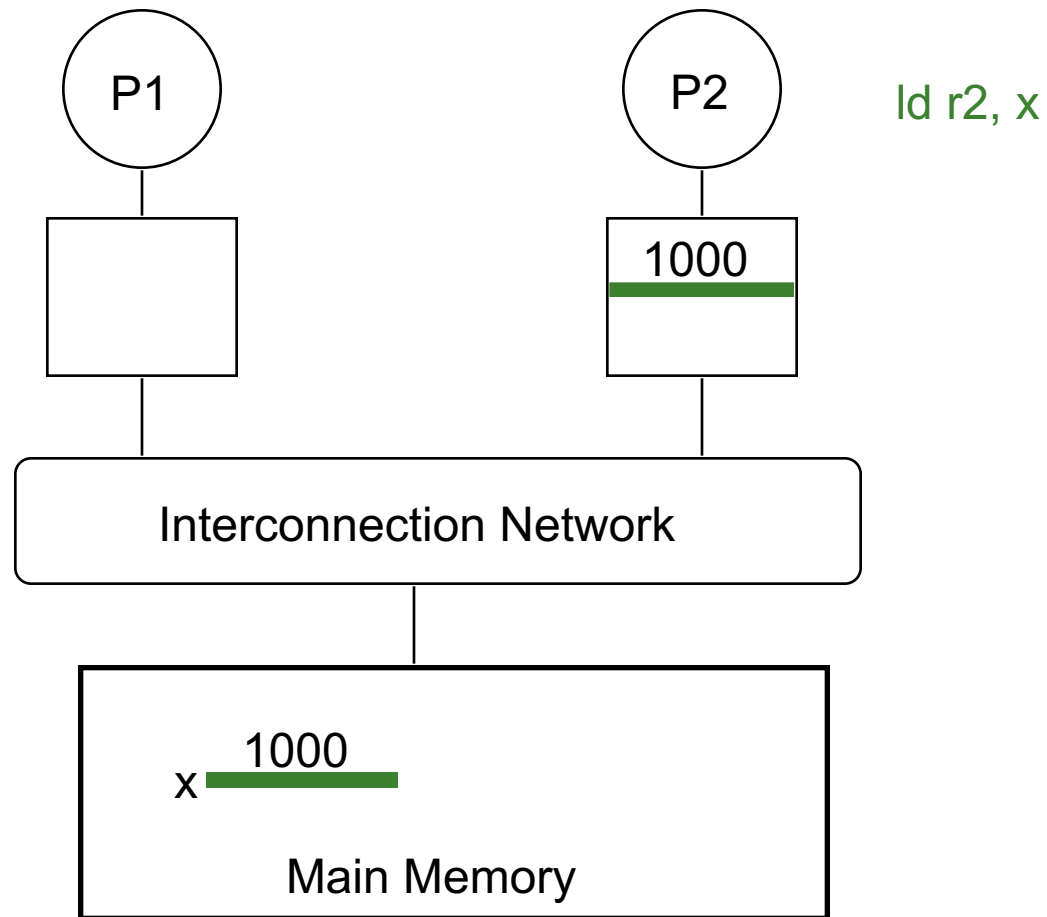
- Each read should receive the value last written by any processor
 - This requires synchronization between threads (i.e., what does last written mean?)
- What if Mem[A] is cached (at either processor)?

Cache Coherence

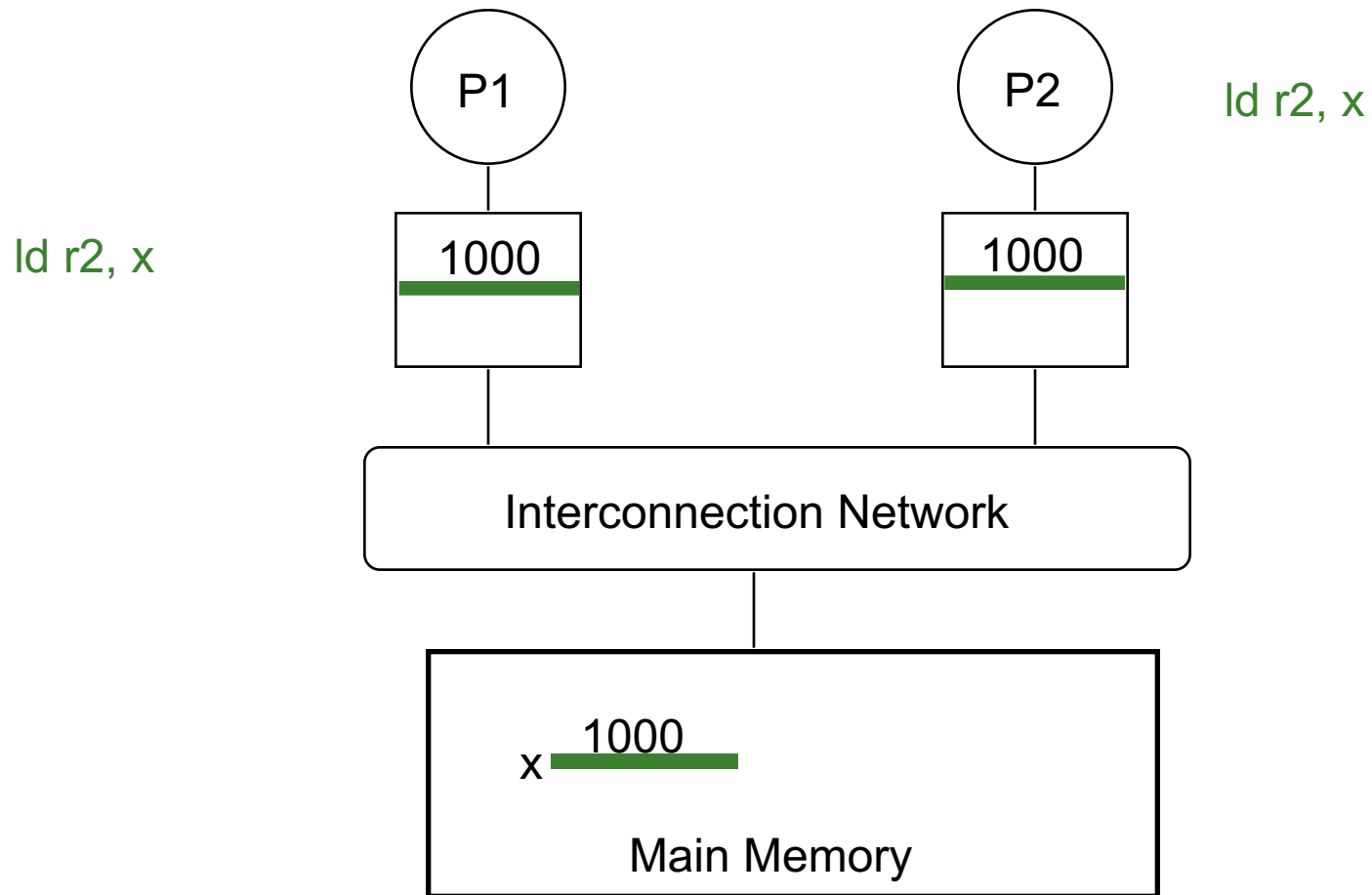
- Basic question: If multiple processors cache the same block, how do they ensure they all see a consistent state?



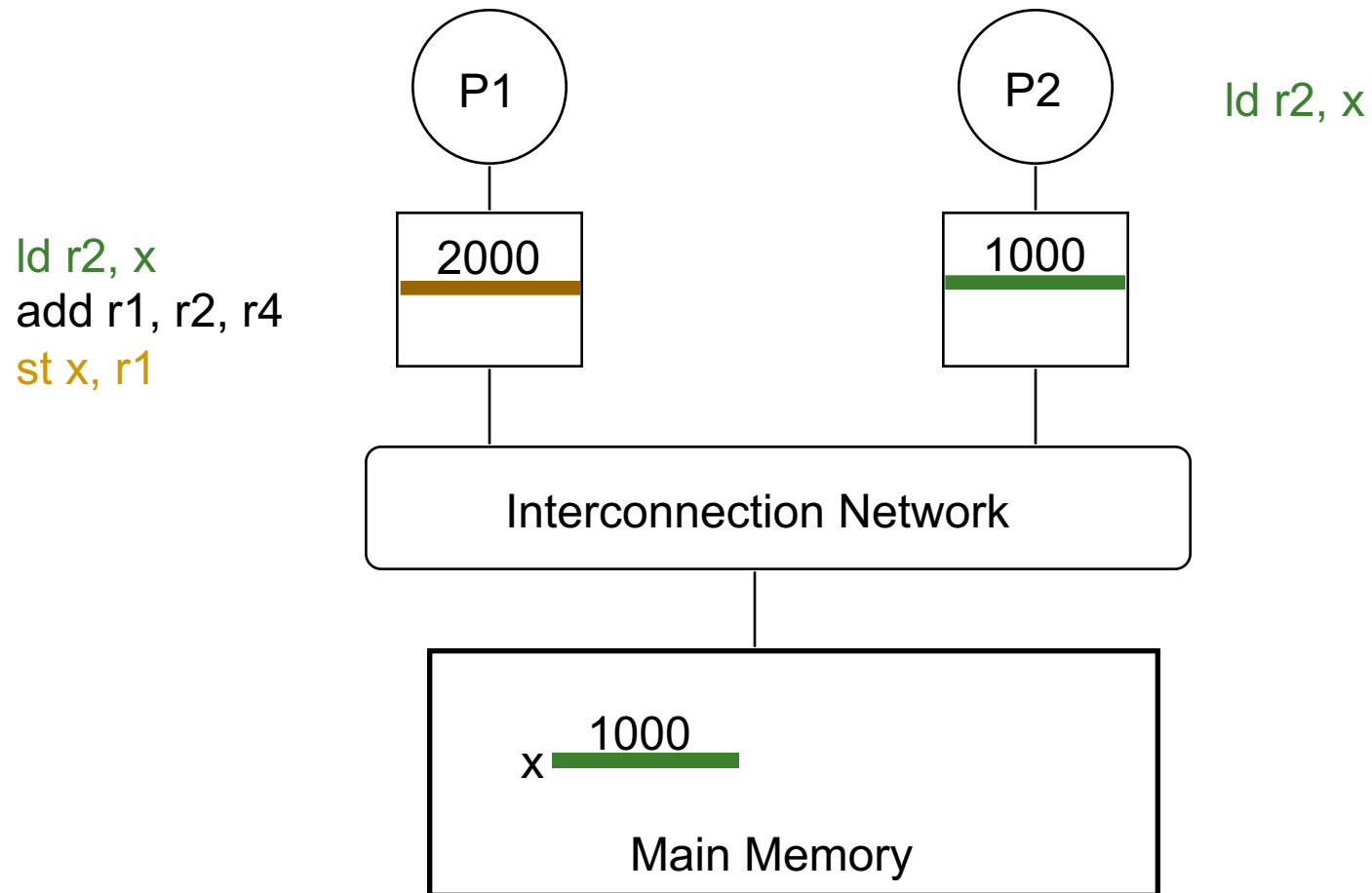
The Cache Coherence Problem



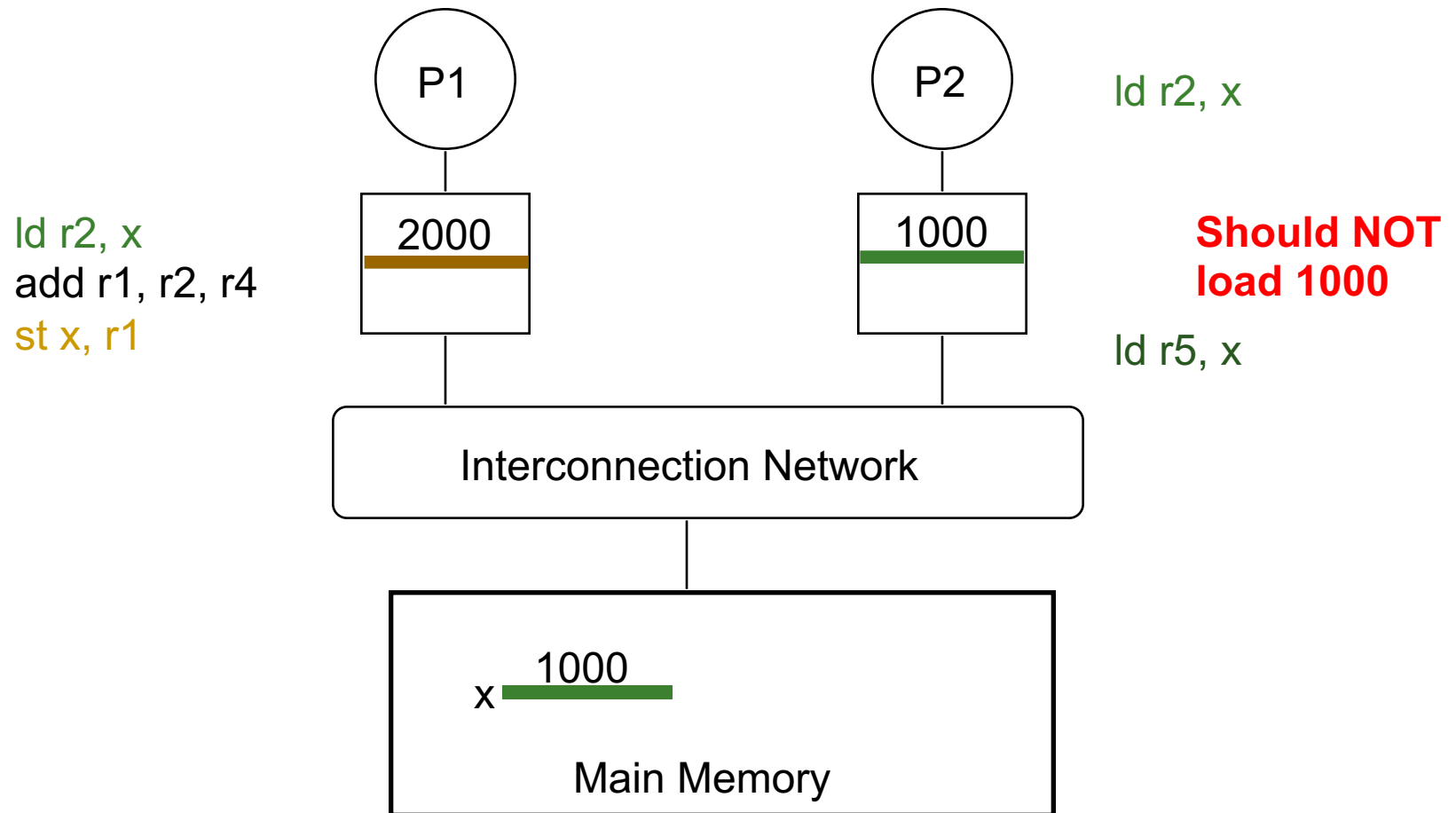
The Cache Coherence Problem



The Cache Coherence Problem



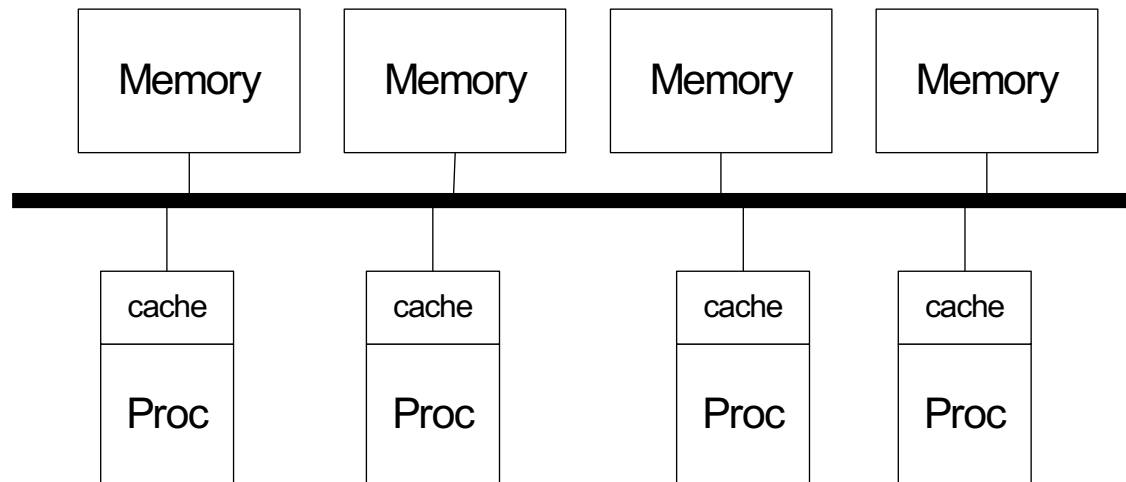
The Cache Coherence Problem



Broadcast-Based Hardware Cache Coherence

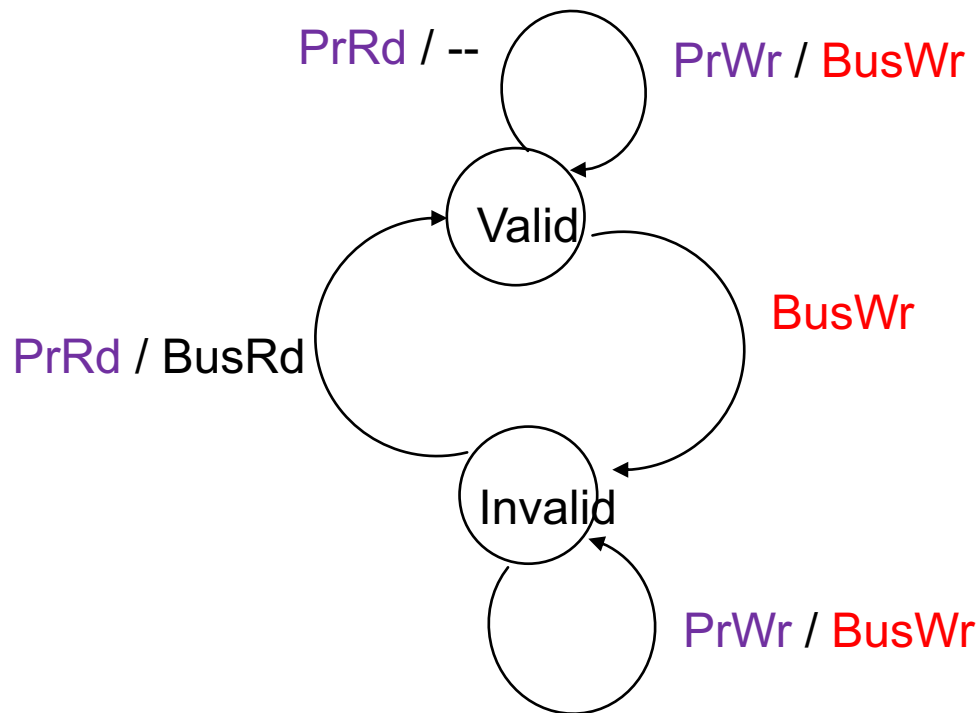
■ Basic idea:

- A processor/cache broadcasts its write/update to a cache block to all other processors
- Another processor/cache that has the block either invalidates or updates its local copy of the block



A Very Simple Coherence Scheme (VI)

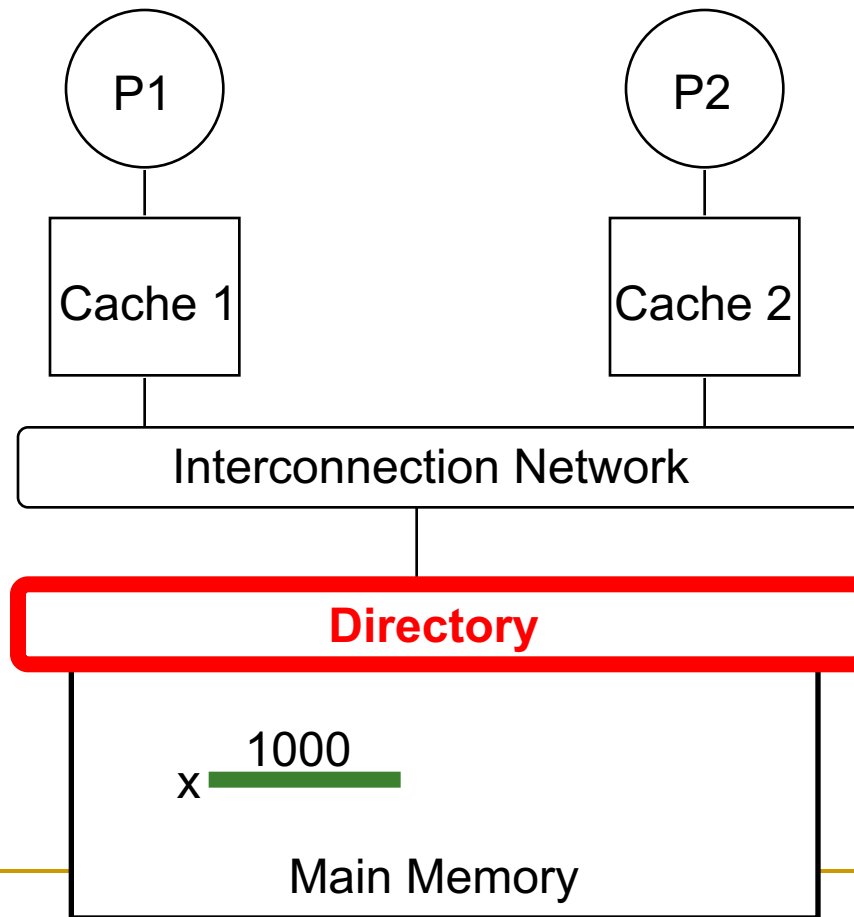
- Idea: All caches “snoop” (observe) each other’s write/read operations. If a processor writes to a block, all others invalidate the block.
- A simple protocol:



- Assumption: Write-through, no-write-allocate cache
- Actions of the local processor on the cache block: PrRd, PrWr
- Actions that are broadcast on the bus for the block: BusRd, BusWr

Directory-Based Cache Coherence

- Idea: A central directory keeps track of which caches contain every possible cache block. Every cache consults this directory to ensure data is kept coherent.

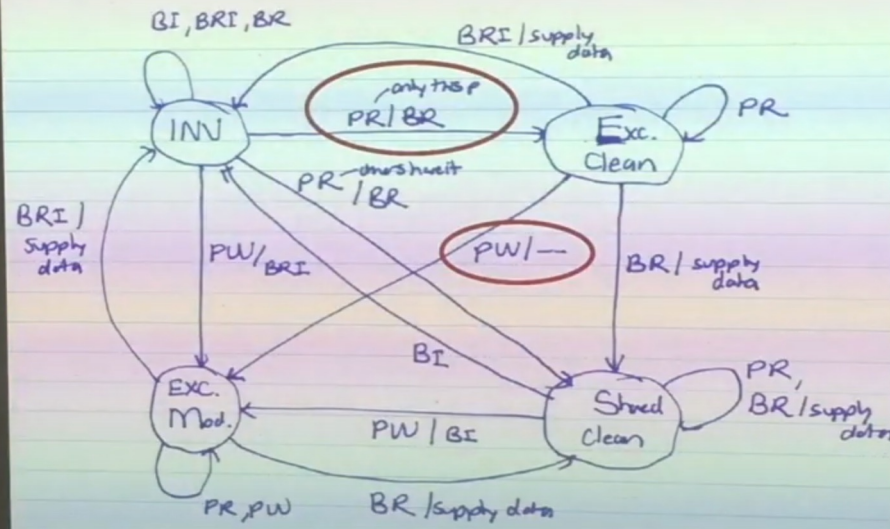


Directory-Based Cache Coherence

- Idea: A central directory keeps track of which caches contain every possible cache block. Every cache consults this directory to ensure data is kept coherent.
- An example mechanism:
 - For each cache block in memory, store $P+1$ bits in directory
 - One bit for each cache, indicating whether the block is in cache
 - Exclusive bit: indicates that a cache has the only copy of the block and can update it without notifying others
 - On a read: set the cache's bit in the directory entry for the block and arrange the supply of the block into the cache
 - On a write: invalidate the block in all caches that have the block and reset their bits in directory entry for the block
 - Have an "exclusive bit" associated with each block in each cache (so that the cache can update the exclusive block silently)

Lecture on Cache Coherence

MESI State Machine



Computer Architecture - Lecture 21: Cache Coherence (ETH Zürich, Fall 2020)

1,419 views • Dec 4, 2020



Onur Mutlu Lectures
16.3K subscribers

27 0 SHARE SAVE ...

ANALYTICS

EDIT VIDEO

Lecture on Memory Ordering & Consistency

time

0

P_1 : A (Set $F_1=1$)

- $F_1=1$ complete from P_1 's view
- A sent to memory

P_2 : X (Set $F_2=1$)

- $F_2=1$ complete from P_2 's view
- X sent to memory

1

P_1 : B (test $F_2=0$)

- B (req F_2) sent to mem.
- B (load F_2) started

P_2 : Y (test $F_1=0$)

- Y (req F_1) sent to mem.
- Y (load F_1) started

50

Mem. sends $F_2(0)$ to P_1

Mem. sends $F_1(0)$ to P_2

51

P_1 is in Critical Section()

P_2 is in Critical Section()

100

Mem. completes A

Mem. completes X

$F_1=1$ in memory TOO LATE!

$F_2=1$ in memory TOO LATE!

For P_1 :
A appeared to happen before X

For P_2 :
X appeared to happen before A

P_1 's VIEW

$A \rightarrow B \rightarrow X$

$A \rightarrow X$

P_2 's VIEW

$X \rightarrow Y \rightarrow A$

$X \rightarrow A$

BOTH CANNOT BE CORRECT!
(from memory's perspective)

P_1 and P_2 saw an inconsistent order of operations in memory

Computer Architecture - Lecture 20: Memory Ordering (Memory Consistency) (ETH Zürich, Fall 2020)

976 views • Dec 4, 2020

22 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lecture on Cache Coherence & Consistency

■ Computer Architecture, Fall 2020, Lecture 21

- ❑ Cache Coherence (ETH, Fall 2020)
- ❑ <https://www.youtube.com/watch?v=T9WlyezeaII&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=38>

■ Computer Architecture, Fall 2020, Lecture 20

- ❑ Memory Ordering & Consistency (ETH, Fall 2020)
- ❑ <https://www.youtube.com/watch?v=Suy09mzTbiQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=37>

■ Computer Architecture, Spring 2015, Lecture 28

- ❑ Memory Consistency & Cache Coherence (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=JfjT1a0vi4E&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=32>

■ Computer Architecture, Spring 2015, Lecture 29

- ❑ Cache Coherence (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=X6DZchnMYcw&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=33>